AGH Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie

Wydział Elektrotechniki, Automatyki, Informatyki i Inżynii Biomedycznej

Praca magisterska

Aleks Piszczek

kierunek studiów: **Elektrotechnika**

Zastosowanie uczenia maszynowego do estymacji stanu systemu elektroenergetycznego

Opiekun: **dr inż. Krzysztof Chmielowiec**

Kraków, 2025

Spis treści

Spis treści **i**

1 Wstęp **1**

2 Definicja i znaczenie estymacji stanu w kontekście zarządzania systemami elektroenergetycznymi **3**

2.1 Rys historyczny 3

2.2 Definicja matematyczna 4

3 Przegląd klasycznych metod estymacji stanu **7**

3.1 Metoda najmniejszych kwadratów (LS) 7

3.2 Metoda ważonych najmniejszych kwadratów (WLS) 8

3.3 Metoda iteracyjna (Gauss-Newton / Newton-Raphson) 9

3.4 Detekcja błędów (Bad Data Detection) 10

3.5 Metody rozproszone i dekompozycyjne 11

3.6 Podsumowanie 12

4 Porównanie metod estymacji stanu na przykładzie sieci testowej **13**

4.1 Wprowadzenie do problemu estymacji 13

4.2 Metodyka Eksperymentu 13

4.3 Implementacja i wyniki estymacji 15

4.3.1 Definicja sieci referencyjnej i obliczenie stanu rzeczywistego. . . 15

4.4 Analiza i dyskusja wyników 19

4.5 Wnioski 20

5 Wprowadzenie do sztucznych sieci neuronowych **21**

5.1 Rys historyczny 21

5.2 Podstawy matematyczne 22

5.3 Proces uczenia: propagacja wsteczna i optymalizacja gradientowa 23

5.4 Metodyka oceny i walidacji modeli 25

5.4.1 Podział zbioru danych. 25

5.4.2 Metody oceny jakości modelu 26

5.5 Zastosowanie sieci neuronowych w zadaniu estymacji stanu 27

6 Przegląd technik ML stosowanych w modelowaniu systemów elektroenergetycznych **29**

6.1 Problem estymacji stanu jako zadanie uczenia nadzorowanego 29

6.2 Wielowarstwowe perceptrony (MLP) 30
6.3 Modele oparte na zespołach drzew decyzyjnych 31
6.3.1 Lasy Losowe (ang. *Random Forest*). 31
6.3.2 Gradient Boosting Machines. 31
6.4 Ograniczenia standardowych architektur i potrzeba uwzględnienia topologii 32

7 Wprowadzenie do grafowych sieci neuronowych i ich potencjału w analizie topologii sieci 34
7.1 Mechanizm przekazywania wiadomości (Message Passing) 35

8 Modelowanie systemu elektroenergetycznego w środowisku Power Factory 40
8.1 Charakterystyka i struktura danych uczących 40
8.2 Środowisko PowerFactory 41
8.3 Metodyka budowy modelu sieci 42

9 Generowanie zbioru uczącego dla modelu GNN na przykładzie sieci IEEE 39-Bus 47
9.1 Metodyka generowania danych. 49
9.2 Implementacja skryptu automatyzującego 50

10 Implementacja grafowej sieci neuronowej 53
10.1 Przygotowanie środowiska i bibliotek 53
10.2 Przetwarzanie i normalizacja danych 54
10.2.1 Formułowanie danych do formatu grafowego 54
10.2.2 Podział zbioru danych i skalowanie 55
10.3 Definicja architektury modelu GNN 56
10.3.1 Pełna architektura estymatora stanu 57
10.4 Trening i walidacja modelu 58
10.4.1 Wczesne zatrzymanie (Early Stopping) 59

11 Ocena jakości zaimplementowanej seci neuronowej 60
11.1 Analiza wydajności obliczeniowej 60

12 Wnioski 67

13 Załączniki 70
13.1 Załącznik A 70
13.2 Załącznik B 76
13.3 Załącznik C 79

Literatura **89**

1 Wstęp

Sieci elektroenergetyczne każdego rodzaju są strukturami krytycznymi, których niezawodna i bezpieczna praca warunkuje poprawne funkcjonowanie całej gospodarki kraju oraz życia społecznego. W obliczu potrzeby zapewnienia odpowiedniego zapotrzebowania na energię elektryczną, a także odpowiednich jej parametrów, operatorzy systemów dystrybucyjnych stają przed licznymi wyzwaniami, obecnie są to przede wszystkim: dynamiczne zmiany obciążeń, rosnący udział źródeł odnawialnych oraz konieczność integracji systemów rozproszonych [1]. Ciągły rozwój sieci dystrybucyjnych, rozdzielczych czy przesyłowych wymagany w celu pokonania wyżej wymienionych trudności, warunkuje wzrost ilości informacji niezbędnej do racjonalnego sterowania ich pracą. Tradycyjne systemy stosowane do prowadzenia ruchu sieci nie są przystosowane do gromadzenia i przetwarzania tak dużej ilości danych i nie umożliwiają dyspozytorom efektywnego wykorzystania dostępnej informacji. Konieczne więc staje się stosowanie coraz to nowszych narzędzi i metod, które nie tylko pozwalają na bieżące monitorowanie parametrów sieci, a także na przewidywanie potencjalnych zakłóceń i awarii [2].

W kontekście rozwiązania powyższych problemów, już od wielu dekad podstawowym narzędziem służącym do odwzorowywania stanu sieci staje się "estymacja stanu systemu elektroenergetycznego"[3]. Należy jednak wziąć pod uwagę fakt, że jeszcze do niedawna systemy elektroenergetyczne były stosunkowo słabo opomiarowane - proces estymacji stanu, w którym znajdowała się sieć, opierał się na ograniczonej ilości informacji oraz algorytmach bazujących na małej ilości zmiennych. Choć podejście to zapewniało akceptowalną dokładność dla ówczesnych, mniej złożonych systemów, w praktyce dawało jedynie przybliżony obraz funkcjonowania sieci i ograniczało możliwości szybkiego reagowania w sytuacjach awaryjnych. Obecnie sytuacja ta uległa całkowitej zmianie, rozwój technologii pomiarowych, w tym przede wszystkim wdrażanie inteligentnych liczników energii AMI (Advanced Metering Infrastructure), urządzeń PMU (Phasor Measurement Units) oraz rozbudowanych systemów SCADA (Supervisory Control and Data Acquisition), spowodował gwałtowny wzrost liczby dostępnych danych [4]. Operatorzy dysponują więc ogromnymi wolumenami informacji, których skala i różnorodność przekraczają możliwości klasycznych (obecnie stosowanych) metod przetwarzania. Dane te są jednak kluczowe dla poprawy dokładności estymacji stanu, wykrywania anomalii czy prognozowania obciążeń [5]. W efekcie powstaje potrzeba stosowania nowych podejść, zdolnych do efektywnej analizy dużych i dynamicznych zbiorów danych.

W obliczu nowoczesnych trudności coraz większe zainteresowanie budzą metody oparte na uczeniu maszynowym, które w ostatnich latach znalazły szerokie zastosowanie w różnych obszarach inżynierii i analizy danych. Ich przewagą nad tradycyjnymi podej-

ściami jest zdolność do automatycznego wydobywania ukrytych wzorców i zależności z dużych oraz niejednorodnych zbiorów informacji, bez konieczności tworzenia szczegółowych modeli matematycznych. W kontekście systemów elektroenergetycznych oznacza to możliwość budowy narzędzi wspierających estymację stanu, które nie tylko lepiej wykorzystują dostępne dane pomiarowe, lecz także są bardziej odporne na brak informacji czy pojawienie się błędów w sygnałach pomiarowych [6]. Uczenie maszynowe staje się tym samym obiecującym kierunkiem badań, pozwalającym na opracowanie metod estymacji dostosowanych do współczesnych, silnie opomiarowanych i dynamicznie zmieniających się sieci elektroenergetycznych.

2 Definicja i znaczenie estymacji stanu w kontekście zarządzania systemami elektroenergetycznymi

2.1 Rys historyczny

Pojęcie estymacji stanu w systemach elektroenergetycznych po raz pierwszy zostało sformułowane na przełomie lat 60. i 70. XX wieku. Fundamentalną pracą, która zapoczątkowała ten obszar badań, były publikacje F.C. Schweppe [3], w których zdefiniowano estymację stanu jako "proces matematycznego wyznaczania najbardziej prawdopodobnych wartości zmiennych opisujących stan systemu, na podstawie nadmiarowych i obarczonych błędami pomiarów". W tamtym okresie podstawowym celem estymatora było zastąpienie pojedynczych, często zawodnych pomiarów spójnym obrazem pracy systemu, umożliwiającym operatorowi prowadzenie obliczeń rozpływu mocy i analizy bezpieczeństwa.

Pierwsze metody opierały się na klasycznej teorii estymacji statycznej i były formułowane w postaci problemu najmniejszych kwadratów. Szczególną rolę odegrało uogólnienie tej idei w postaci metody ważonych najmniejszych kwadratów (WLS - Weighted Least Square), która pozwalała uwzględniać różną dokładność poszczególnych pomiarów. Już w latach 70. i 80. ugruntowało się podejście, zgodnie z którym estymacja stanu jest centralnym modułem systemów EMS (Energy Management System), integrującym dane pomiarowe i dostarczającym spójnego wektora stanu jako podstawy dalszych aplikacji - takich jak analiza bezpieczeństwa, wykrywanie awarii czy optymalizacja pracy systemu. [7].

W klasycznym ujęciu estymacja stanu była zatem zdefiniowana przede wszystkim jako narzędzie rekonstrukcji wewnętrznych stanów systemu (np. kątów napięć w węzłach), którego rolą było wypełnienie luk wynikających z ograniczonego opomiarowania sieci. Systemy elektroenergetyczne w drugiej połowie XX wieku charakteryzowały się bowiem stosunkowo niewielką ilością urządzeń pomiarowych, pomiary dostępne w systemach SCADA miały ograniczoną częstotliwość i dokładność. W rezultacie proces estymacji stanu musiał kompensować braki informacji poprzez wykorzystanie redundancji i modeli matematycznych, jednak zarówno od samego początku funkcjonowania przez kolejne lata aż do najnowszych czasów proces estymacji stanu miał kluczowe znaczenie dla monitorowania pracy w sieci.

Zmiana paradygmatu nastąpiła w ostatnich dwóch dekadach, wraz z gwałtownym rozwojem technologii pomiarowych i wprowadzeniem do eksploatacji nowych urządzeń, jak wspomniane w rozdziale 1 AMI, oraz synchroniczne rejestratory fazorów PMU. W przeciwieństwie do tradycyjnych systemów SCADA, PMU umożliwiają pomiary synchroniczne z wysoką częstotliwością próbkowania oraz dokładnością czasową dzięki synchronizacji

GPS, co znacząco zwiększyło ilość i jakość dostępnych danych [8]. Rezultatem tych zmian jest przejście od sytuacji niedoboru informacji do jej nadmiaru, który klasyczne algorytmy estymacji nie zawsze są w stanie przetworzyć.

Współcześnie estymacja stanu rozumiana jest zatem znacznie szerzej niż w jej klasycznym ujęciu. Oprócz podstawowej funkcji rekonstrukcji stanu, postrzegana jest jako platforma integracji i analizy wielkoskalowych, heterogenicznych danych pomiarowych, która musi radzić sobie z problemami jakości danych, np. brakami, błędami systematycznymi, czy w kontekście panującej wojny hybrydowej atakami typu bad data injection [9]. Stała zostaje natomiast dynamiczna zmiana warunków pracy systemów wynikająca z rozwoju technologii i zapotrzebowania użytkowników. Coraz większe znaczenie zyskują też metody hybrydowe i inteligentne, które integrują klasyczne modele fizyczne z technikami uczenia maszynowego, umożliwiając lepsze odwzorowanie nieliniowych zależności i adaptację do zmiennych warunków pracy [6].

W efekcie, o ile w latach 70 i 80 estymacja stanu była przede wszystkim odpowiedzią na ograniczoną dostępność pomiarów, o tyle obecnie jej rozwój wynika z odwrotnej sytuacji - ogromnej ilości i różnorodności danych pomiarowych, które wymagają nowych metod analizy i przetwarzania. Ta ewolucja definicji i znaczenia estymacji stanu odzwierciedla szersze przemiany w elektroenergetyce: od prostych, scentralizowanych systemów opartych na dużych jednostkach wytwórczych do złożonych, dynamicznych i rozproszonych sieci XXI wieku, w których bezpieczeństwo i niezawodność muszą być zapewniane przez zaawansowaną analizę i inteligentne algorytmy.

2.2 Definicja matematyczna

Jak wspomniano w punkcie 2.1, estymacja stanu (ang. State Estimation SE) polega na odtworzeniu rzeczywistego obrazu pracy sieci na podstawie dostępnych pomiarów oraz matematycznego modelu systemu. Ponieważ w praktyce nie jesteśmy jednak w stanie zmierzyć wszystkiego (lub pomiary są obarczone błędami), potrzebny jest algorytm, który na podstawie dostępnych danych w sposób najbardziej wiarygodny będzie oddawał stan systemu elektroenergetycznego.

W klasycznym modelu estymacji stanu wybór zmiennych stanu opiera się na fundamentalnych zasadach działania sieci elektroenergetycznej oraz możliwościach matematycznego odwzorowania przepływów mocy.

Wektor stanu w SE można zapisać jako:

$$x = \Big[\theta_1, \ldots, \theta_n\,, \quad V_1, \ldots, V_n\Big]^T \tag{1}$$

gdzie:

- θ_i oznaczają kąty fazowe napięć w poszczególnych węzłach,

- V_i odpowiadają wartościom skutecznym napięć w poszczególnych węzłach,

Uzasadnienia wyboru tych konkretnych zmiennych można szukać w fizycznych podstawach obliczeń przepływu mocy czynnej i biernej w SE, przepływ ten zależy bezpośrednio od napięć w węzłach oraz różnic fazowych między nimi. Zgodnie z równaniem przesyłowym linii i, j:

$$P_{ij} = V_i V_j \left(G_{ij} \cos(\theta_i - \theta_j) + B_{ij} \sin(\theta_i - \theta_j)\right) \tag{2}$$

$$Q_{ij} = V_i V_j \left(G_{ij} \sin(\theta_i - \theta_j) - B_{ij} \cos(\theta_i - \theta_j)\right) \tag{3}$$

gdzie G_{ij} i B_{ij} to odpowiednio przewodność i susceptancja linii.

Widać wyraźnie, że przepływy mocy zależą tylko od modułów napięć V i różnic kątów $\theta_i - \theta_j$.[1] Zatem te dwie grupy zmiennych w pełni określają stan sieci w sensie elektroenergetycznym – każda inna wielkość może być wyliczona z nich i znanych parametrów linii.

W praktyce, w systemie elektroenergetycznym o n węzłach, jeden lub więcej z nich (w liczbie k) pełni rolę węzła odniesienia *(ang. slack bus)*. Jego kąt fazowy jest z definicji ustalony (zazwyczaj na $0°$) i nie podlega estymacji, służąc jako punkt odniesienia dla pozostałych kątów w sieci. Dlatego, choć sieć ma n kątów fazowych, estymacji podlega jedynie $n-k$ z nich. Liczba modułów napięć do wyznaczenia pozostaje równa n. Całkowita liczba zmiennych stanu, które należy wyznaczyć, wynosi zatem:

$$\underbrace{(n-k)}_{\text{kąty fazowe do wyznaczenia}} + \underbrace{n}_{\text{moduły napięć do wyznaczenia}} = 2n - k \tag{4}$$

Aby oszacować wartości tych $2n - k$ zmiennych, wykorzystuje się zbiór pomiarów dokonywanych w różnych punktach sieci. Pomiary te, z natury obarczone pewnym błędem, tworzą tzw. wektor pomiarów z, który może zawierać takie wielkości jak:

- wartości modułów napięć w wybranych węzłach,
- przepływy mocy czynnej i biernej w gałęziach (liniach i transformatorach),
- bezpośrednie pomiary fazorów napięć i prądów z synchronicznych rejestratorów fazorów (PMU).

Zależność pomiędzy wektorem pomiarów z a stanem systemu opisuje nieliniowy układ równań:

[1] Parametry linii G_{ij} i B_{ij} również wpływają na wartości przepływów mocy, jednak są one stałe dla danej sieci, wynikające z fizyki i konstrukcji linii (materiał przewodnika, długość, konfiguracja, częstotliwość sieci itp.), więc w estymacji stanu nie traktuje się ich jako zmiennych sterowalnych.

$$z = h(x) + e \tag{5}$$

gdzie:

- $h(x)$ - to wektor funkcji nieliniowych odwzorowujących prawa Kirchhoffa i model sieci,
- e - to wektor błędów pomiarowych, który przyjmuje się za zmienną losową o rozkładzie normalnym i znanej macierzy kowariancji R.

Z równania (4) bezpośrednio wynika więc informacja, że istnieje minimalna liczba pomiarów, które należy wykonać, aby estymacja stanu sprowadzona do układu równań miała rozwiązanie. W praktyce jednak zależy nam na spełnieniu warunku

$$m > 2n - k \tag{6}$$

gdzie m to liczba pomiarów, prowadzi to do powstania układu nadokreślonego, w którym liczba pomiarów przewyższa liczbę niewiadomych. Takie podejście niesie za sobą kilka korzyści. Większa liczba pomiarów minimalizować będzie wpływ błędów losowych pomiarów. W przypadku dostarczania przez urządzenia pomiarowe wadliwych danych, możliwa będzie łatwiejsza detekcja i korekcja błędu; układ nadokreślony charakteryzuje się też lepszą kondycją numeryczną w porównaniu do określonego, co będzie zmniejszało ryzyko niestabilności w procesie iteracyjnego rozwiązywania równania. Empiryczne doświadczenia pokazują, że rozsądna liczba nadmiarowych pomiarów to $m = 4n$ [10]

Idea estymacji polega więc na znalezieniu takiego wektora stanu x, który najlepiej odwzorowuje rzeczywisty stan sieci na podstawie dostępnych pomiarów, czyli minimalizuje różnicę między wartościami zmierzonymi a wartościami obliczonymi z modelu sieci. W praktyce oznacza to, że szukamy takiego x, dla którego funkcja kosztu przyjmuje wartość najmniejszą, co pozwala uzyskać spójny i wiarygodny obraz parametrów SE.

3 Przegląd klasycznych metod estymacji stanu

3.1 Metoda najmniejszych kwadratów (LS)

Zasadniczą ideą metody najmniejszych kwadratów (LS - Least Squares) jest minimalizacja sumy kwadratów różnic pomiędzy wartościami rzeczywistymi pomiarów a wartościami przewidywanymi przez model matematyczny sieci elektroenergetycznej. Formalnie, dla danego zestawu pomiarów z oraz nieliniowej funkcji $h(x)$ odwzorowującej zależności fizyczne opisujące pracę systemu elektroenergetycznego, definiuje się funkcję kosztu postaci:

$$J(x) = \sum_{i=1}^{m} \big(z_i - h_i(x)\big)^2 \tag{7}$$

gdzie m oznacza liczbę dostępnych pomiarów, z_i jest wartością i-tego pomiaru, natomiast $h_i(x)$ jest wartością teoretyczną tego samego pomiaru obliczoną na podstawie modelu dla wektora stanu x. Proces estymacji polega na takim doborze x, aby funkcja $J(x)$ osiągnęła minimum.

Podstawowym założeniem metody LS jest traktowanie wszystkich pomiarów jako równoważnych, tzn. zakłada się, że każdy pomiar jest obarczony błędem o tym samym rozkładzie, zwykle przyjmowanym jako normalny, z wartością oczekiwaną równą zero i znaną wariancją. W praktyce oznacza to, że LS nie odróżnia pomiaru napięcia w stacji elektroenergetycznej od pomiaru przepływu mocy w linii – każdy z nich jest brany pod uwagę z tą samą wagą. Dzięki temu problem estymacji przyjmuje bardzo prostą postać matematyczną, co zadecydowało o jego popularności w początkowych latach rozwoju systemów zarządzania siecią.

W najprostszym przypadku, gdzie zależności $h(x)$ są liniowe, rozwiązanie można zapisać w postaci analitycznej jako:

$$\hat{x} = (H^T H)^{-1} H^T z \tag{8}$$

gdzie H jest macierzą czułości (tzw. macierzą Jacobiego[2]) odwzorowania pomiarowego. Taka postać rozwiązania wskazuje na duże walory metody LS, ponieważ w prosty sposób łączy dane pomiarowe z modelem sieci.

Metoda LS w kontekście elektroenergetyki stanowiła fundament dla późniejszego rozwoju badań nad bardziej zaawansowanymi metodami, w szczególności metodą ważonych najmniejszych kwadratów, która obecnie wyparła swoją poprzedniczkę. Niemniej jednak, nawet w najprostszej formie, LS pozwalała na uzyskanie spójnego wektora stanu, dzięki czemu mogła być podstawą dalszych analiz czy planowania rozpływów mocy.

[2]Macierz Jacobiego to macierz pochodnych cząstkowych funkcji pomiarowej względem wektora stanu.

Największą zaletą metody LS bezsprzecznie zawsze była jej prostota i niewielkie wymagania obliczeniowe. W latach 70 i 80, gdy możliwości komputerów wykorzystywanych w centrach dyspozytorskich były ograniczone, miało to ogromne znaczenie praktyczne. Algorytm LS mógł być wykonywany w czasie rzeczywistym, co pozwalało na bieżącą ocenę stanu sieci. Również transparentność metody, wynikająca z jej matematycznej prostoty, czyniła ją łatwą do analizy i interpretacji przez inżynierów systemowych.

Niestety ograniczenia LS ujawniły się bardzo szybko. Ponieważ wszystkie pomiary traktowane są w niej jako jednakowo dokładne, metoda ta nie potrafiła uwzględnić faktu, że w rzeczywistych systemach elektroenergetycznych istnieje duże zróżnicowanie jakości danych. Pomiary napięć w punktach z transformatorami były dużo bardziej dokładne, niż dane przesyłane przez stare urządzenia telemetryczne w liniach przesyłowych. Ignorowanie tego aspektu powodowało, że estymata stanu mogła być zdominowana przez dużą liczbę mniej dokładnych i mniej istotnych pomiarów. Ponadto LS była bardzo wrażliwa na występowanie tzw. błędów odstających. Pojedynczy pomiar zawierający poważny błąd również mógł istotnie zniekształcić wynik, ponieważ funkcja kosztu kwadratowo wzmacnia wpływ dużych odchyleń. W systemach o niskiej redundancji pomiarowej oznaczało to niekiedy brak możliwości uzyskania wiarygodnego obrazu pracy sieci.

Pomimo tych słabości, metoda najmniejszych kwadratów była pierwszym krokiem w kierunku automatyzacji monitorowania systemów elektroenergetycznych i stworzyła fundament teoretyczny dla całej późniejszej dziedziny estymacji stanu.

3.2 Metoda ważonych najmniejszych kwadratów (WLS)

Rozwinięciem klasycznej metody LS, które znalazło praktyczne zastosowanie w rzeczywistych systemach elektroenergetycznych, jest metoda ważonych najmniejszych kwadratów (WLS – Weighted Least Squares). Jej zasadnicza różnica polega na tym, że poszczególnym pomiarom przypisuje się wagi zależne od ich wiarygodności, zwykle odwrotnie proporcjonalne do wariancji błędu pomiarowego. Dzięki temu dokładniejsze pomiary mają większy wpływ na wynik estymacji, natomiast dane obarczone dużą niepewnością wnoszą proporcjonalnie mniej informacji. Funkcja kosztu w WLS przyjmuje postać

$$J(x) = \big(z - h(x)\big)^T R^{-1} \big(z - h(x)\big), \tag{9}$$

gdzie R jest macierzą kowariancji błędów pomiarowych, zwykle przyjmowaną jako diagonalna, jeśli zakłada się brak korelacji pomiędzy poszczególnymi pomiarami. W przeciwieństwie do metody LS, w której wszystkie składniki sumy były traktowane jednakowo, w WLS każdy składnik jest ważony, co pozwala na lepsze odzwierciedlenie rzeczywistości.

Rozwiązanie analityczne w przypadku liniowym ma postać:

$$\hat{x} = (H^T R^{-1} H)^{-1} H^T R^{-1} z, \tag{10}$$

Takie podejście nie tylko poprawia dokładność otrzymywanej estymaty stanu, lecz także zwiększa jej odporność na mniej wiarygodne pomiary, które w klasycznym LS mogłyby zdominować rozwiązanie. Z tego względu metoda WLS bardzo szybko stała się standardem stosowanym w systemach zarządzania siecią elektroenergetyczną i do dziś stanowi podstawę większości implementacji estymacji stanu w praktyce przemysłowej.

3.3 Metoda iteracyjna (Gauss-Newton / Newton-Raphson)

W praktyce jednak zależności pomiędzy pomiarami a stanem systemu są silnie nieliniowe, ponieważ wynikają z równań rozpływu mocy, w których moc czynna i bierna są funkcjami wartości napięć oraz różnic kątów fazowych między poszczególnymi węzłami. Z tego względu rozwiązanie problemu w rzeczywistych systemach wymaga zastosowania podejścia iteracyjnego. Najczęściej wykorzystuje się w tym celu metodę Gaussa–Newtona, w której w każdej iteracji dokonuje się liniowego przybliżenia nieliniowej funkcji $h(x)$,

1. Na podstawie aktualnego przybliżenia wektora stanu x^k oblicza się wartości przewidywanych pomiarów $h(x^k)$ oraz reszty:

$$r = z - h(x^k).$$

2. Oblicza się macierz Jacobiego H, która opisuje, jak zmiana poszczególnych elementów wektora stanu wpływa na przewidywane pomiary:

$$H_{ij} = \frac{\partial h_i(x)}{\partial x_j}.$$

3. Wyznacza się poprawkę wektora stanu Δx poprzez rozwiązanie układu liniowego w postaci najmniejszych kwadratów:

$$\Delta x = (H^T H)^{-1} H^T r.$$

4. Aktualizuje się wektor stanu:

$$x^{k+1} = x^k + \Delta x.$$

5. Powtarza się procedurę iteracyjnie, aż zmiana Δx będzie wystarczająco mała lub dalsze iteracje nie zmniejszą znacząco wartości funkcji kosztu.

Więc dla metody LS wektor stanu aktualizuje się według schematu:

$$x^{k+1} = x^k + \Delta x, \tag{11}$$

$$\Delta x = (H^T H)^{-1} H^T \big(z - h(x^k)\big). \tag{12}$$

Natomiast w nieliniowym przypadku WLS procedura iteracyjna przebiega analogicznie jak w metodzie LS, z tym że w każdym kroku wykorzystywane są macierze H i R (równanie 10).

3.4 Detekcja błędów (Bad Data Detection)

W praktyce systemy pomiarowe w sieciach elektroenergetycznych są narażone na występowanie błędnych lub niewiarygodnych danych pomiarowych, które mogą wynikać z awarii czujników, zakłóceń transmisji czy błędów kalibracji. Obecność takich tzw. „bad data" może znacząco zniekształcić wyniki estymacji stanu, prowadząc do nieprawidłowych decyzji w zarządzaniu siecią.

Detekcja błędów w estymacji stanu opiera się na analizie reszt pomiarowych, tj. różnic między wartościami zmierzonymi z a wartościami przewidywanymi przez model $h(x)$:

$$r = z - h(x).$$

W metodzie BDD identyfikuje się pomiary, których reszty są znacząco większe niż oczekiwane odchylenia losowe. Najczęściej stosuje się do tego celu test największej znormalizowanej reszty (Largest Normalized Residual LNR [11]) lub test χ^2, które pozwalają wskazać pomiary odstające i ocenić spójność całego zestawu danych.

Test LNR jest jedną z podstawowych metod wykrywania błędnych danych pomiarowych („bad data") w systemach estymacji stanu. Aby uwzględnić różną dokładność poszczególnych pomiarów, reszty normalizuje się przez ich oczekiwane odchylenie standardowe σ_i:

$$r_i^{\text{norm}} = \frac{r_i}{\sigma_i}.$$

Następnie wyznacza się największą znormalizowaną resztę w zbiorze pomiarów:

$$r_{\max}^{\text{norm}} = \max_i |r_i^{\text{norm}}|.$$

Jeżeli wartość $r_{\max}^{\text{norm}}$ przekracza ustalony próg krytyczny, pomiar odpowiadający tej reszcie jest uznawany za błędny i może zostać odrzucony lub poddany dalszej weryfikacji. Takie podejście pozwala na skuteczne wykrywanie pojedynczych pomiarów odstających, które w klasycznej estymacji stanu mogłyby znacząco zniekształcić wynik, przy jednoczesnym uwzględnieniu różnej jakości danych pochodzących z różnych typów czujników w systemie elektroenergetycznym.

Test χ^2 stanowi kolejną klasyczną metodę wykrywania błędnych danych pomiarowych. Jego istota polega na ocenie ogólnej spójności wszystkich pomiarów z modelem systemu poprzez analizę funkcji kosztu najmniejszych kwadratów. W przypadku WLS przyjmuje ona postać jak w równaniu 9.

Test χ^2 polega na porównaniu wartości funkcji kosztu $J(x)$ dla estymowanego wektora stanu $\hat{x}$ z wartością krytyczną τ wyznaczoną na podstawie rozkładu χ^2 dla odpowiedniej liczby stopni swobody i ustalonego poziomu ufności:

$$J(\hat{x}) \leq \tau.$$

Jeżeli wartość funkcji kosztu przekracza próg krytyczny τ, oznacza to, że przynajmniej jeden z pomiarów jest niespójny z pozostałymi danymi i może być traktowany jako „bad data". Test χ^2 umożliwia więc wykrycie zarówno pojedynczych pomiarów odstających, jak i sytuacji, w których cały zestaw pomiarów jest niespójny z modelem systemu. Dzięki temu stanowi skuteczne narzędzie do zwiększenia dokładności i wiarygodności estymacji stanu w praktycznych systemach elektroenergetycznych.

3.5 Metody rozproszone i dekompozycyjne

Wraz z rosnącą złożonością i rozmiarem współczesnych systemów elektroenergetycznych, tradycyjne, scentralizowane podejście do estymacji stanu napotyka na fundamentalne ograniczenia. Skalowalność obliczeniowa staje się wyzwaniem, ponieważ wymiary macierzy Jacobiego oraz macierzy kowariancji rosną nieliniowo z liczbą węzłów, co prowadzi do drastycznego wzrostu czasu obliczeń i wymagań pamięciowych. Co więcej, scentralizowane podejście wymaga przesyłania ogromnych ilości danych pomiarowych z całego systemu do jednego centrum dyspozytorskiego, co rodzi problemy z niezawodnością i bezpieczeństwem komunikacji.

Odpowiedzią na te wyzwania są metody rozproszone i dekompozycyjne. Ich fundamentalna idea polega na rezygnacji z rozwiązywania jednego, wielkiego problemu estymacji na rzecz jego podziału na szereg mniejszych, wzajemnie powiązanych podproblemów. System elektroenergetyczny jest dzielony na kilka lub kilkanaście mniejszych obszarów (podsystemów), a dla każdego z nich prowadzona jest lokalna estymacja stanu. Następnie, poprzez zdefiniowany protokół komunikacji i koordynacji, lokalne rozwiązania są iteracyjnie uzgadniane, aby ostatecznie zbiec do spójnego, globalnego rozwiązania dla całego systemu [12].

Podejście to niesie za sobą szereg korzyści. Przede wszystkim, znacząco poprawia skalowalność i wydajność obliczeniową, ponieważ mniejsze problemy mogą być rozwiązywane równolegle przez różne jednostki obliczeniowe. Zwiększa również odporność i bezpieczeństwo systemu – awaria lub cyberatak w jednym obszarze niekoniecznie prowadzi do załamania całego procesu estymacji. Niemniej jednak, metody te wprowadzają nowe wyzwania, głównie związane z koniecznością zapewnienia niezawodnej komunikacji między obszarami oraz z matematyczną gwarancją zbieżności lokalnych algorytmów do poprawnego globalnego optimum.

Chociaż metody rozproszone i dekompozycyjne stanowią obecnie jeden z najważniejszych i najszybciej rozwijających się trendów w dziedzinie estymacji stanu, ich szczegółowa analiza wykracza poza zakres niniejszej pracy. Zostały one wspomniane w celu przedstawienia pełnego obrazu aktualnego stanu wiedzy i wskazania kierunków, w jakich ewoluują klasyczne, scentralizowane algorytmy [13].

3.6 Podsumowanie

W niniejszym rozdziale dokonano przeglądu metod estymacji stanu systemów elektroenergetycznych. Przedstawiono ewolucję od historycznie pierwszej i najprostszej metody najmniejszych kwadratów, do jej znacznie bardziej odpornego i powszechnie stosowanego w przemyśle następcy – metody ważonych najmniejszych kwadratów. Podkreślono, że kluczową przewagą WLS jest zdolność do uwzględniania różnej jakości i wiarygodności danych pomiarowych poprzez mechanizm wag, co jest niezbędne w rzeczywistych zastosowaniach.

Omówiono również fakt, że nieliniowość równań opisujących systemy elektroenergetyczne wymusza stosowanie metod iteracyjnych, takich jak algorytm Gaussa-Newtona, w celu znalezienia rozwiązania. Ponadto, przedstawiono standardowe techniki detekcji błędów grubych (bad data detection), takie jak test największej znormalizowanej reszty oraz test χ^2, które stanowią integralną część nowoczesnych estymatorów i pozwalają na identyfikację oraz eliminację niewiarygodnych pomiarów, chroniąc estymatę przed zniekształceniem.

Na koniec, zarysowano nowoczesny trend rozwoju w postaci metod rozproszonych i dekompozycyjnych, które stanowią odpowiedź na wyzwania związane ze skalowalnością i bezpieczeństwem w coraz bardziej rozległych i złożonych sieciach. Mimo iż stanowią one obiecujący kierunek badań, to właśnie scentralizowany estymator WLS, wzbogacony o mechanizmy detekcji błędów, pozostaje do dziś przemysłowym standardem i punktem odniesienia dla wszystkich nowych technik.

4 Porównanie metod estymacji stanu na przykładzie sieci testowej

4.1 Wprowadzenie do problemu estymacji

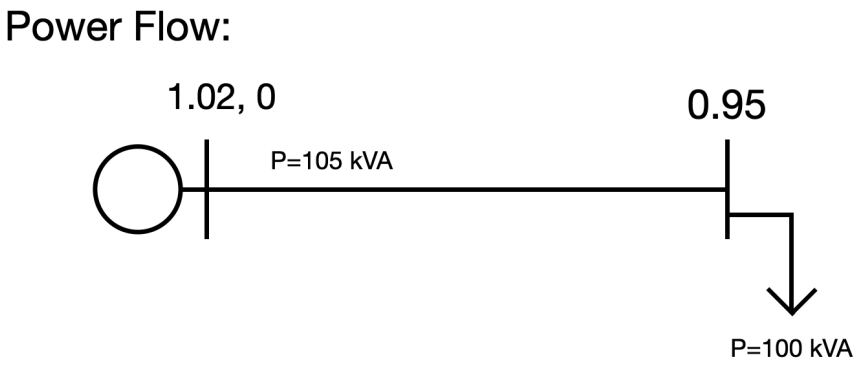
Celem niniejszego podrozdziału jest praktyczne zilustrowanie oraz porównanie fundamentalnych metod estymacji stanu: klasycznej metody najmniejszych kwadratów oraz jej udoskonalenia, metody ważonych najmniejszych kwadratów. Przeprowadzona analiza posłuży jako metodyczny punkt odniesienia, pokazując działanie i skuteczność standardowego podejścia do estymacji. W dalszych rozdziałach nowo opracowane modele oparte na uczeniu maszynowym będą porównywane z koncepcyjną i wydajnościową charakterystyką tego rozwiązania.

Do zilustrowania problemu posłużono się prostym, dwuwęzłowym modelem sieci elektroenergetycznej, przedstawionym na Rysunku 1. Składa się on z węzła bilansującego (ang. *slack bus*), reprezentującego punkt zasilania o znanym napięciu, oraz węzła odbiorczego (ang. *load bus*), do którego przyłączone jest obciążenie. Oba węzły połączone są jedną linią przesyłową. Taki model pozwala w czytelny sposób ukazać fundamentalną różnicę między problemem obliczeń rozpływu mocy a problemem estymacji stanu.

W przypadku analizy rozpływu mocy (rys. 1) dysponujemy kompletem informacji wejściowych: znamy zarówno parametry zasilania w węźle odniesienia (V, θ), jak i dokładną wartość obciążenia na końcu linii (P). Celem jest analityczne *obliczenie* wynikowego stanu systemu, np. wartości napięcia w węźle odbiorczym. Z kolei w problemie estymacji stanu (rys. 2) sytuacja jest inna. Dysponujemy określonym zestawem pomiarów, obejmującym zarówno dane z węzła zasilającego, jak i z węzła odbiorczego. Kluczowym elementem scenariusza jest założenie, że jeden z tych pomiarów jest celowo obarczony błędem grubym, co symuluje sytuację otrzymania danych o niskiej wiarygodności. Zadaniem estymatora jest wykorzystanie tego redundantnego i częściowo błędnego zestawu danych do odfiltrowania nieprawidłowej informacji i oszacowania najbardziej prawdopodobnego stanu całej sieci. W kolejnych podrozdziałach przeprowadzony zostanie eksperyment numeryczny, który na podstawie tego modelu porówna zdolność metod LS i WLS do rekonstrukcji nieznanego stanu systemu.

4.2 Metodyka Eksperymentu

W celu przeprowadzenia porównania estymatorów, stworzono syntetyczne środowisko testowe w bibliotece `pandapower`[14]. Kolejne etapy przygotowania eksperymentu stanowiły: budowa modelu referencyjnego, generowanie danych pomiarowych oraz definicja scenariusza testowego.



Rysunek 1: Model testowej sieci - problem rozpływu mocy.

Opis modelu sieci. Jako podstawę do analizy wykorzystano, dwuwęzłowy model sieci promieniowej, który służy jako tzw. „prawdziwy stan" systemu. Sieć składa się z węzła bilansującego (lewy), do którego przyłączono źródło zewnętrzne o ustalonym napięciu $V_0 = 1.02$ p.u., oraz węzła odbiorczego (prawy), zasilającego obciążenie o rzeczywistej mocy $P = 0.1$ MW oraz $Q = 0.05$ MVAr. Węzły połączone są linią przesyłową o długości 0.5 km i parametrach $R = 0.3\,\Omega/\text{km}$ oraz $X = 0.1\,\Omega/\text{km}$.

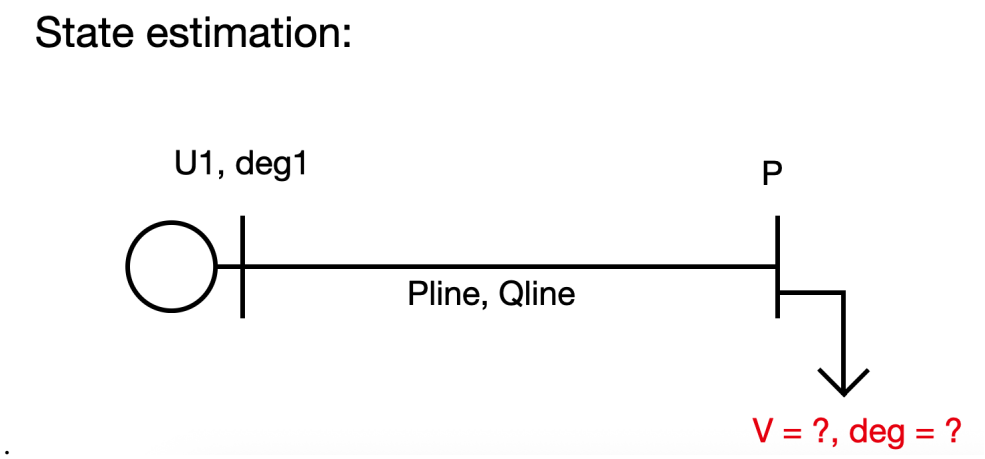
Generowanie danych pomiarowych. Proces generowania danych pomiarowych składał się z dwóch kroków. Najpierw, na opisanym Rysunku 1 modelu, przeprowadzono deterministyczne obliczenia rozpływu mocy w celu uzyskania idealnych, bezbłędnych wartości wszystkich wielkości elektrycznych w sieci. Następnie, na podstawie tych prawdziwych wartości, wygenerowano zestaw czterech syntetycznych pomiarów poprzez dodanie do nich losowego szumu. Szum ten symuluje niedokładności aparatury pomiarowej i pochodzi z rozkładu normalnego (Gaussowskiego) o średniej równej zero i zadanym odchyleniu standardowym.

Generowanie danych pomiarowych. Proces generowania danych pomiarowych składał się z dwóch kroków. Najpierw, na opisanym modelu, przeprowadzono deterministyczne obliczenia rozpływu mocy w celu uzyskania idealnych, bezbłędnych wartości wszystkich wielkości elektrycznych. Następnie, na podstawie tych wartości, wygenerowano zestaw czterech syntetycznych pomiarów poprzez dodanie do nich losowego szumu z rozkładu normalnego o zadanych odchyleniach standardowych.

- Pomiar napięcia w w. odniesienia (V_0): $\sigma = 0.002$ (pomiar o wysokiej wiarygodności).
- Pomiary mocy w w. odniesienia (P_0, Q_0): $\sigma = 0.002$ (pomiary o wysokiej wiarygodności).

- Pomiar mocy w w. obciążenia (P_1): $\sigma = 0.05$ (pomiar o niskiej wiarygodności, symulujący błąd gruby).

Różnicę między problemem rozpływu mocy zaprezentowanym na rysunku 1 a estymacją stanu, która została przyjęta w danym scenariuszu testowym, pokazuje rysunek 2



Rysunek 2: Model testowej sieci - problem estymacji stanu

Tak przygotowany, składający się z czterech pomiarów zestaw danych stanowi nadokreślony system wejściowy, zawierający wyraźną wartość odstającą, co stanowi wymagający test dla obu estymatorów.

4.3 Implementacja i wyniki estymacji

W celu weryfikacji opisanej metodyki, przygotowano skrypt w języku Python. Pełny kod źródłowy implementacji znajduje się w załączniku A. Poniżej przedstawiono chronologicznie kolejne etapy implementacji, które doprowadziły do uzyskania końcowych wyników estymacji.

4.3.1 Definicja sieci referencyjnej i obliczenie stanu rzeczywistego.

Pierwszym krokiem było stworzenie w bibliotece `pandapower` modelu referencyjnego sieci. Zdefiniowano dwuwęzłową sieć z określonymi parametrami linii oraz obciążeniem. Następnie, poprzez wykonanie obliczeń rozpływu mocy, uzyskano dokładne wartości napięć i przepływów mocy. Te bezbłędne dane posłużyły jako punkt odniesienia do dalszych kroków.

Model sieci

```
# Definicja obciążenia
true_p = 0.1
true_q = 0.05

```

```python
# Funkcja tworząca sieć referencyjną
def create_base_network():
    net = pp.create_empty_network()
    b_s = pp.create_bus(net, vn_kv=0.4)
    b_l = pp.create_bus(net, vn_kv=0.4)
    pp.create_line_from_parameters(net, ...)
    pp.create_ext_grid(net, bus=b_s, vm_pu=1.02)
    pp.create_load(net, bus=b_l, p_mw=true_p, q_mvar=true_q)
    return net

# Stworzenie sieci i obliczenie rozpływu mocy
net_true = create_base_network()
pp.runpp(net_true)
```

Wygenerowanie zestawu pomiarów. Kolejnym etapem była symulacja zestawu pomiarów poprzez dodanie do wartości prawdziwych losowego szumu. Zaimplementowano scenariusz „bad data”, w którym trzem pomiarom przypisano małe odchylenie standardowe ($\sigma = 0.002$), a czwartemu, symulującemu dużą wartość ($\sigma = 0.05$). W ten sposób powstał wektor pomiarów z.

pomiary

```python
# Definicja odchyleń standardowych
std_dev_v_slack = 0.002   # Dobry pomiar
std_dev_p_slack = 0.002   # Dobry pomiar
std_dev_q_slack = 0.002   # Dobry pomiar
std_dev_p_load_BAD = 0.05 # Zły pomiar

# Wygenerowanie zaszumionych wartości
v_meas = net_true.res_bus.vm_pu[0] + np.random.normal(0, std_dev_v_slack)
# ... (pozostałe pomiary) ...

z = np.array([v_meas, p_meas_slack, q_meas_slack, p_meas_load_BAD])
```

Sformułowanie modelu matematycznego rozpływu mocy Przed implementacją pętli algorytmu estymacji, konieczne było analityczne sformułowanie rozpływów mocy. Równania opisujące przepływy mocy czynnej i biernej w węzłach wynikają bezpośrednio z praw Kirchhoffa dla obwodów prądu przemiennego i są wyrażone za pomocą macierzy admitancji systemu Y_{bus}. Macierz ta, o wymiarach $n \times n$ (gdzie n to liczba węzłów w sieci), stanowi kompletny matematyczny opis topologii i parametrów elektrycznych sieci. Jej elementy diagonalne Y_{ii} reprezentują sumę admitancji wszystkich elementów przyłączonych do węzła i, natomiast elementy pozadiagonalne Y_{ij} odpowiadają ujemnej wartości admitancji gałęzi łączącej węzły i oraz j. Dla węzła i, ogólna postać równania na moc

czynną to:

$$P_i = V_i \sum_{j=1}^{n} V_j (G_{ij} \cos\theta_{ij} + B_{ij} \sin\theta_{ij}) \tag{13}$$

gdzie V to moduł napięcia, θ to kąt, a G_{ij} i B_{ij} to odpowiednio część rzeczywista i urojona elementu Y_{ij} z macierzy admitancji.

Macierz Jacobiego H, kluczowa dla algorytmu Gaussa-Newtona, jest zdefiniowana jako macierz pochodnych cząstkowych funkcji pomiarowych $h(x)$ (czyli równań mocy 13) względem wektora stanu x. Wszystkie pochodne, np. $\frac{\partial P_i}{\partial V_j}$, zostały wyznaczone w sposób analityczny.

Jako przykład, poniżej przedstawiono wyprowadzenie pochodnej cząstkowej mocy czynnej w węźle odniesienia (P_0) względem modułu napięcia w węźle obciążenia (V_1). Równanie wyjściowe ma postać:

$$P_0(x) = V_0^2 G_{00} + V_0 V_1 (G_{01} \cos\theta_{01} + B_{01} \sin\theta_{01}) \tag{14}$$

Obliczając pochodną cząstkową względem V_1, traktujemy wszystkie pozostałe zmienne (V_0, θ_{01}) jako stałe. Pierwszy składnik równania jest stałą, więc jego pochodna wynosi zero. W drugim składniku jedyną zmienną jest V_1, zatem pochodna jest po prostu współczynnikiem stojącym przy V_1:

$$\frac{\partial P_0}{\partial V_1} = V_0 (G_{01} \cos\theta_{01} + B_{01} \sin\theta_{01}) \tag{15}$$

Wszystkie pozostałe elementy macierzy Jacobiego zostały wyprowadzone w analogiczny sposób. Następnie te analityczne wzory zostały przełożone na kod w języku Python. Przykładowo, implementacja powyższej pochodnej cząstkowej w funkcji obliczającej Jakobian wygląda następująco:

Jacobian

```python
# Implementacja macierzy Jacobiego
def jacobian_full(...):

# Implementacja analitycznie wyprowadzonej pochodnej dP0/dV1
dP0_dV1 = V0_nominal * (G01 * np.cos(delta01) + B01 * np.sin(delta01))
    # ... (obliczenia kolejnych pochodnych cząstkowych) ...
    return np.array(...)
```

Dla metody WLS, zdefiniowano macierz wag kowariancji błędów pomiarowych R. Przy standardowym założeniu, że błędy poszczególnych pomiarów są od siebie niezależne, macierz R jest macierzą diagonalną, której elementy na głównej przekątnej to wariancje

(σ^2) odpowiadających im pomiarów:

$$R = \begin{bmatrix} \sigma_1^2 & 0 & \cdots & 0 \\ 0 & \sigma_2^2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_m^2 \end{bmatrix} \tag{16}$$

Macierz wag R^{-1} jest zatem również macierzą diagonalną, z elementami równymi $1/\sigma_i^2$. Taka konstrukcja w naturalny sposób realizuje ideę ważenia: pomiar o dużej niepewności (duża wariancja σ^2) otrzyma małą wagę w procesie estymacji, podczas gdy pomiar bardzo dokładny (mała wariancja σ^2) będzie miał duży wpływ na ostateczny wynik.

Macierz wag

```
# Obliczenie wariancji (sigma^2)
variances = np.array([std_dev_v_slack**2, std_dev_p_slack**2,
                      std_dev_q_slack**2, std_dev_p_load_BAD**2])

# Stworzenie macierzy wag R^-1
R_inv = np.diag(1 / variances)
```

Ostatnim etapem była implementacja pętli iteracyjnej Gaussa-Newtona. Zgodnie z teorią przedstawioną w rozdziale 3, w każdym kroku obliczany jest wektor poprawek Δx, który następnie aktualizuje wektor stanu. Fundamentalna różnica między metodami LS i WLS leży w sposobie, w jaki traktują one błędy pomiarowe. Metoda najmniejszych kwadratów zakłada, że wszystkie pomiary są jednakowo wiarygodne i minimalizuje prostą sumę kwadratów reszty. W przeciwieństwie do niej, metoda ważonych najmniejszych kwadratów wprowadza kluczowy mechanizm różnicowania pomiarów, przypisując każdemu z nich wagę adekwatną do jego dokładności.

Narzędziem realizującym tę ideę jest macierz wag R^{-1}. Jej rola polega na wprowadzeniu do modelu matematycznego informacji o niepewności (wariancji, σ^2) każdego z pomiarów. Wagi te są odwrotnie proporcjonalne do wariancji błędów, co oznacza, że pomiary o wysokiej precyzji (mała wariancja σ^2) otrzymują dużą wagę, a ich wpływ na ostateczny wynik jest znaczący. Z kolei pomiary obarczone dużą niepewnością (duża wariancja σ^2), takie jak te pochodzące z czujnika punktu obciążenia, otrzymują wagę bliską zeru, przez co są w naturalny sposób marginalizowane w procesie estymacji. Ta kluczowa różnica sprowadza się do pojedynczej operacji matematycznej w kroku aktualizacji wektora stanu. Poniżej przedstawiono implementację obu wariantów tego kluczowego kroku.

Gauss-Newton

```
# Krok aktualizacji dla metody LS
delta_x = np.linalg.pinv(H.T @ H) @ H.T @ r

```

```
# Krok aktualizacji dla metody WLS (z uwzględnieniem wag R_inv)
delta_x = np.linalg.pinv(H.T @ R_inv @ H) @ H.T @ R_inv @ r
```

Po iteracyjnym zastosowaniu odpowiedniego kroku aktualizacji aż do osiągnięcia zbieżności, uzyskano ostateczne estymaty stanu dla obu metod. Wyniki tego porównania przedstawiono w Tabeli 1.

Tabela 1: Porównanie wyników estymacji pełnego wektora stanu dla scenariusza „bad data".

Metoda	**V** [p.u.]	θ [°]	**P** [MW]	**Q** [MVAr]	$\Delta\theta$ [°]	Δ **V** [%]	Δ **P** [%]	Δ **Q** [%]
Prawdziwa Wartość	0.898	0.977	0.100	0.050	0.000	0.000	0.000	0.000
Implementacja LS	0.866	0.098	0.127	0.044	0.879	3.526	27.272	11.914
Implementacja WLS	0.898	1.056	0.100	0.051	0.079	0.023	0.271	2.505

4.4 Analiza i dyskusja wyników

Wyniki estymacji przedstawione w Tabeli 1 jednoznacznie wskazują na znaczącą przewagę metody ważonych najmniejszych kwadratów w analizowanym scenariuszu. Różnica w dokładności obu metod jest bezpośrednim, empirycznym potwierdzeniem ich teoretycznych właściwości, zwłaszcza w kontekście estymacji pełnego wektora stanu, czyli modułu napięcia V_1 i kąta fazowego θ_1.

Metoda najmniejszych kwadratów wygenerowała estymatę obarczoną bardzo dużym błędem. Błąd estymacji modułu napięcia wyniósł 3.5%, natomiast błędy estymacji mocy były znacznie wyższe, osiągając 27.3% dla mocy czynnej oraz 11.9% dla mocy biernej. Ta rozbieżność w dokładności wynika z faktu, że algorytm LS, próbując "pogodzić"jeden rażąco błędny pomiar mocy z trzema poprawnymi, był zmuszony do wyestymowania znacząco błędnego kąta fazowego θ_1 (błąd bliski 0.9°). Ponieważ moc czynna jest silnie zależna od kąta, nawet niewielki błąd w module napięcia, w połączeniu z dużym błędem kąta, skutkuje kaskadowym wzrostem błędu w ostatecznie obliczonej mocy. (nw czy to dobrze sformułowany wniosek, czy to nie wynika bezpośrednio z zadanego błędu)

Zupełnie inaczej zachowała się metoda WLS. Uzyskany przez nią wynik jest niezwykle dokładny dla wszystkich estymowanych wielkości – błąd modułu napięcia wyniósł zaledwie 0.023%, a błąd mocy czynnej 0.27%. Wynik ten ilustruje kluczową zaletę WLS, czyli zdolność do odrzucania błędnych danych. Algorytm, wykorzystując macierz wag R^{-1}, poprawnie zidentyfikował pomiar mocy w węźle obciążenia jako wysoce niepewny i przypisał mu znikomą wagę. Estymata została więc oparta niemal wyłącznie na trzech

pozostałych, wiarygodnych pomiarach, co pozwoliło na precyzyjną rekonstrukcję całego wektora stanu, w tym zarówno modułu napięcia, jak i kąta fazowego.

Przeprowadzony eksperyment stanowi zatem praktyczną walidację teoretycznych właściwości obu metod i pokazuje, że zdolność metody WLS do uwzględniania informacji o jakości danych jest kluczowa dla uzyskania wiarygodnych wyników estymacji w naturalnych warunkach funkcjonowania sieci elektroenergetycznych.

4.5 Wnioski

W niniejszym rozdziale przeprowadzono praktyczne porównanie dwóch fundamentalnych metod estymacji stanu: klasycznej metody najmniejszych kwadratów oraz metody ważonych najmniejszych kwadratów. W tym celu, przy użyciu języka Python oraz bibliotek `numpy` i `pandapower`, zaimplementowano od podstaw iteracyjny algorytm Gaussa-Newtona. Został on przetestowany na dwuwęzłowej sieci elektroenergetycznej w scenariuszu symulującym wystąpienie błędu grubego w jednym z pomiarów. Eksperyment numeryczny jednoznacznie wykazał, że metoda WLS jest znacznie bardziej odporna na obecność błędnych danych w zestawie pomiarowym. Pozwoliło to na precyzyjną rekonstrukcję pełnego wektora stanu – w tym modułu napięcia i kąta fazowego – uzyskując estymatę o rząd wielkości dokładniejszą niż w przypadku metody LS.

Należy podkreślić, że wykorzystany model dwuwęzłowy jest celowym uproszczeniem, mającym stanowić jedynie czytelną ilustrację fundamentalnych zasad i różnic między analizowanymi estymatorami. Mimo swojej prostoty, model ten w sposób pokazał, jak kluczowe jest prawidłowe podejście do niepewności pomiarowej. W rzeczywistych, wielkowymiarowych systemach elektroenergetycznych, składających się z setek lub tysięcy węzłów, problemy te jedynie eskalują. Kwestie takie jak złożoność obliczeniowa, zła warunkowość macierzy Jacobiego oraz prawdopodobieństwo wystąpienia wielu błędów grubych jednocześnie sprawiają, że solidność i odporność algorytmu estymacji nabierają jeszcze większego znaczenia.

Eksperyment ten ustanowił metodę WLS jako sprawdzony punkt odniesienia. Potwierdzenie jej skuteczności, ale również świadomość jej ograniczeń, stanowi motywację do poszukiwania alternatywnych podejść. W kolejnych rozdziałach pracy zbadane zostaną nowoczesne techniki oparte na uczeniu maszynowym, w celu oceny ich potencjału w zadaniu estymacji stanu systemów elektroenergetycznych.

5 Wprowadzenie do sztucznych sieci neuronowych

5.1 Rys historyczny

Koncepcja sztucznych sieci neuronowych, mimo iż jej dynamiczny rozwój przypada na ostatnią dekadę, ma swoje teoretyczne korzenie w połowie XX wieku. Pierwotne idee wywodziły się z dążenia do formalizacji obliczeniowej procesów neurobiologicznych. Już w 1943 roku Warren McCulloch i Walter Pitts zaproponowali pierwszy uproszczony model matematyczny neuronu, zdolny do przetwarzania sygnałów binarnych [15]. Był to jednak model czysto teoretyczny, pozbawiony mechanizmu uczenia się z danych.

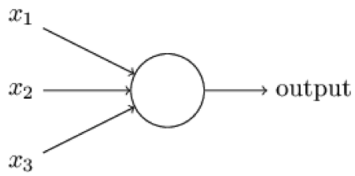
Istotny postęp w tej dziedzinie został osiągnięty w 1958 roku, kiedy to Frank Rosenblatt opracował i zaprezentował perceptron – model neuronowy wyposażony w algorytm uczenia się na podstawie przykładów [16]. Zdolność perceptronu do iteracyjnej adaptacji wag w celu poprawnej klasyfikacji wzorców stanowiła fundament pod współczesne uczenie maszynowe. Okres początkowy, optymistycznego przyjęcia tej koncepcji został zahamowany w 1969 roku przez publikację Marvina Minsky'ego i Seymoura Paperta. W swojej monografii "Perceptrons"przedstawili oni formalny dowód wykazujący fundamentalne ograniczenia tego modelu, w tym jego niezdolność do rozwiązywania problemów nieliniowo separowalnych, czego klasycznym przykładem jest problem XOR [17]. Ta merytoryczna krytyka, w połączeniu z ówczesnymi ograniczeniami mocy obliczeniowej, doprowadziła do okresu stagnacji i zredukowanego finansowania badań, znanego jako "zima sztucznej inteligencji".

Odnowione zainteresowanie sieciami neuronowymi nastąpiło w latach 80. XX wieku, głównie za sprawą spopularyzowania algorytmu "propagacji wstecznej błędu"[18]. Algorytm ten umożliwił efektywny trening wielowarstwowych sieci neuronowych (MLP - Multi-Layer Perceptron), które, w przeciwieństwie do jednowarstwowego perceptronu, były w stanie modelować złożone, nieliniowe zależności. Mimo tego sukcesu, w kolejnych latach sieci neuronowe ustępowały popularnością innym metodom, takim jak maszyny wektorów nośnych (SVM - Support Vector Machine), które oferowały lepsze gwarancje teoretyczne.

Dopiero druga dekada XXI wieku przyniosła gwałtowny wzrost znaczenia sieci neuronowych, napędzany przez jednoczesne wystąpienie trzech kluczowych czynników: dostęp do bezprecedensowo dużych zbiorów danych (Big Data), rozwój wysoce wydajnych, równoległych jednostek obliczeniowych (GPU - Graphics Processing Unit) oraz szereg istotnych innowacji algorytmicznych. To połączenie dało początek erze "głębokiego uczenia"(deep learning), w której bardzo głębokie architektury neuronowe zaczęły osiągać przełomową skuteczność w wielu dziedzinach, w tym w złożonych problemach inżynierskich, takich jak estymacja stanu systemów elektroenergetycznych [19].

5.2 Podstawy matematyczne

Fundamentalnym budulcem każdej sztucznej sieci neuronowej jest pojedynczy sztuczny neuron, którego model matematyczny jest inspirowany jego biologicznym odpowiednikiem. W swojej najprostszej formie, znanej jako perceptron [16], neuron jest prostą jednostką obliczeniową, która przetwarza wiele sygnałów wejściowych w jeden sygnał wyjściowy, co przedstawiono na Rysunku 3. Proces ten składa się z dwóch kluczowych kroków.



Rysunek 3: Schemat pojedynczego sztucznego neuronu z n wejściami.

W pierwszym kroku, neuron oblicza sumę ważoną swoich sygnałów wejściowych. Każdemu wejściu x_j przypisana jest waga w_j, będąca liczbą rzeczywistą, która symbolizuje siłę danego połączenia synaptycznego. Wagi te są kluczowymi, uczącymi się parametrami modelu – to właśnie ich wartości są modyfikowane w procesie treningu. Do sumy ważonej dodawany jest również dodatkowy, uczący się parametr b, nazywany obciążeniem (ang. *bias*). Jego wprowadzenie, będące matematycznym uproszczeniem koncepcji progu aktywacji, pozwala na przesunięcie funkcji aktywacji, co znacząco zwiększa elastyczność i zdolności reprezentacyjne modelu. Operację tę można zapisać w zwięzłej postaci wektorowej jako:

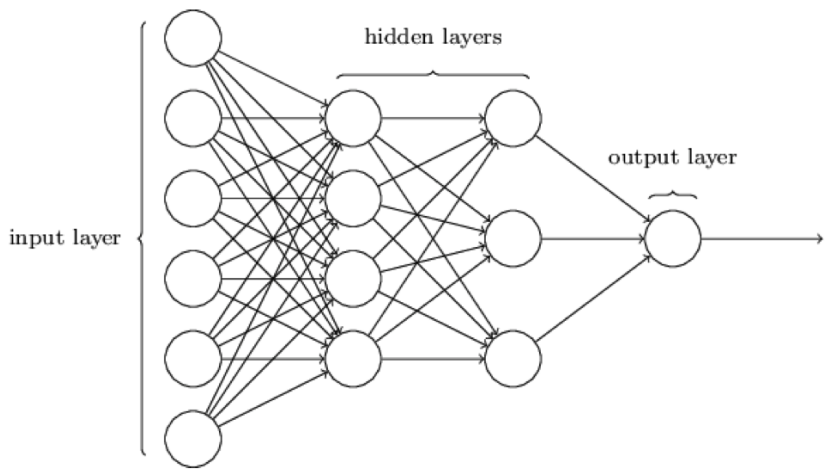
$$z = \mathbf{w}^T\mathbf{x} + b = \sum_{j=1}^{n} w_j x_j + b \tag{17}$$

gdzie $\mathbf{x}$ to wektor wejścia, a $\mathbf{w}$ to wektor wag.

W drugim kroku, uzyskana wartość z jest przepuszczana przez nieliniową funkcję aktywacji ϕ. Funkcja ta decyduje o ostatecznym sygnale wyjściowym neuronu $a = \phi(z)$. Wprowadzenie nieliniowości jest absolutnie kluczowe – to właśnie dzięki niej wielowarstwowe sieci neuronowe są w stanie aproksymować dowolnie złożone funkcje nieliniowe [20]. Bez nieliniowej funkcji aktywacji, cała sieć, niezależnie od jej głębokości, sprowadzałaby się do prostej transformacji liniowej. W historii badań stosowano wiele różnych funkcji aktywacji, od historycznej funkcji skokowej w perceptronie, przez funkcje sigmoidalne (np. sigmoida logistyczna, tangens hiperboliczny), aż po obecnie najpopularniejszą i najefektywniejszą obliczeniowo funkcję ReLU (ang. *Rectified Linear Unit*), zdefiniowaną jako $\phi(z) = \max(0, z)$ [21].

Pojedynczy neuron ma jednak ograniczone możliwości. Prawdziwa moc obliczeniowa sieci neuronowych bierze się z łączenia wielu takich jednostek w zorganizowaną, warstwową strukturę, tworząc tak zwany perceptron wielowarstwowy (MLP), przedstawiony na Rysunku 4. Architektura MLP składa się z:

- **Warstwy wejściowej**, która jedynie przekazuje wektor cech $\mathbf{x}$ do pierwszej warstwy ukrytej.
- **Jednej lub więcej warstw ukrytych**, w których każdy neuron jest połączony z wszystkimi neuronami z warstwy poprzedniej (stąd nazwa: warstwy w pełni połączone, ang. *fully-connected*). Każda warstwa ukryta przetwarza sygnały z warstwy poprzedniej i przekazuje swoje wyjścia do warstwy następnej.
- **Warstwy wyjściowej**, która produkuje ostateczny wynik (predykcję) całej sieci. W zadaniach regresji, neurony w tej warstwie zazwyczaj nie posiadają funkcji aktywacji (lub używają liniowej funkcji tożsamościowej), aby umożliwić przewidywanie dowolnych wartości rzeczywistych.



Rysunek 4: Architektura perceptronu wielowarstwowego (MLP) z dwoma warstwami ukrytymi.

Taka hierarchiczna, wielowarstwowa struktura pozwala sieci na uczenie się cech o rosnącym poziomie abstrakcji. Pierwsze warstwy mogą uczyć się prostych, niskopoziomowych wzorców, a kolejne, bazując na wyjściach poprzednich, mogą składać je w coraz bardziej złożone i abstrakcyjne reprezentacje, co jest podstawą potęgi głębokiego uczenia.

5.3 Proces uczenia: propagacja wsteczna i optymalizacja gradientowa

Posiadając zdefiniowaną architekturę sieci, kolejnym krokiem jest proces uczenia, czyli iteracyjna procedura dopasowywania wag i obciążeń modelu w taki sposób, aby jego

predykcje były jak najbliższe wartościom rzeczywistym. Proces ten można sformułować jako problem optymalizacji, którego celem jest znalezienie takiego zestawu parametrów, który minimalizuje globalny błąd sieci.

Sercem tego procesu jest *funkcja kosztu* (lub funkcja straty, ang. *cost/loss function*), oznaczana jako $C(\mathbf{w}, b)$. Jest to funkcja, która mierzy, jak bardzo "zły"jest aktualny stan sieci. Dla pojedynczego przykładu uczącego $(\mathbf{x}, \mathbf{y})$, oblicza ona rozbieżność między wektorem predykcji sieci $a(\mathbf{x})$ a prawdziwym wektorem docelowym $\mathbf{y}$. Najczęściej stosowaną funkcją kosztu w zadaniach regresji jest *błąd średniokwadratowy (MSE)*, zdefiniowany jako:

$$C(\mathbf{w}, b) = \frac{1}{2n} \sum_{\mathbf{x}} \|\mathbf{y}(\mathbf{x}) - a(\mathbf{x})\|^2 \tag{18}$$

gdzie suma przebiega po wszystkich n przykładach w zbiorze uczącym. Celem algorytmu uczącego jest znalezienie takiego zestawu wag $\mathbf{w}$ i obciążeń b, dla którego wartość funkcji C jest jak najmniejsza.

Do minimalizacji funkcji kosztu wykorzystuje się algorytmy optymalizacji oparte na metodach gradientowych. Najbardziej fundamentalnym z nich jest algorytm *spadku gradientu* (ang. *gradient descent*). Funkcję kosztu można wyobrazić sobie jako wielowymiarową powierzchnię, gdzie każdy punkt na tej powierzchni odpowiada określonej wartości błędu dla danego zestawu wag i obciążeń. Algorytm ma za zadanie znaleźć globalne minimum tej powierzchni. W tym celu obliczany jest *gradient funkcji kosztu*, ∇C. Jest to wektor pochodnych cząstkowych funkcji kosztu względem wszystkich parametrów modelu, $\nabla C = \left(\frac{\partial C}{\partial w_1}, \frac{\partial C}{\partial b_1}, \ldots\right)$, który wskazuje kierunek najszybszego wzrostu wartości funkcji. Aby minimalizować błąd, parametry są iteracyjnie aktualizowane poprzez wykonanie małego kroku w kierunku przeciwnym do gradientu:

$$w_k \to w_k' = w_k - \eta \frac{\partial C}{\partial w_k} \quad \text{oraz} \quad b_l \to b_l' = b_l - \eta \frac{\partial C}{\partial b_l} \tag{19}$$

gdzie η to *tempo uczenia* (ang. *learning rate*), mały, dodatni hiperparametr[3], który kontroluje wielkość kroku. W praktyce, ze względu na wydajność obliczeniową, stosuje się wariant stochastyczny (SGD), gdzie gradient jest obliczany dla małych partii danych (ang. *mini-batches*), a nie dla całego zbioru na raz [22].

Wyzwaniem obliczeniowym pozostaje efektywne wyznaczenie gradientu ∇C dla sieci składającej się z milionów parametrów. Rozwiązaniem tego problemu jest *algorytm propagacji wstecznej błędu* (ang. *backpropagation*) [18]. Jest to niezwykle wydajna metoda,

[3]Hiperparametr to zewnętrzna konfiguracja algorytmu uczenia, której wartość jest ustalana przed rozpoczęciem procesu treningu. W przeciwieństwie do parametrów (np. wag), które są uczone z danych, hiperparametry (np. tempo uczenia, liczba warstw w sieci, liczba neuronów w warstwie) muszą być dobrane przez projektanta modelu, często w procesie eksperymentalnym.

która, wykorzystując regułę łańcuchową różniczkowania[4], oblicza wszystkie pochodne cząstkowe w dwóch krokach. Najpierw sygnał jest propagowany "do przodu"przez sieć, od warstwy wejściowej do wyjściowej, obliczając aktywacje wszystkich neuronów. Następnie błąd jest propagowany "wstecz", od warstwy wyjściowej do wejściowej, a na każdym etapie obliczane są lokalne gradienty. Propagacja wsteczna błędu jest fundamentem, na którym opiera się całe współczesne głębokie uczenie.

W praktycznych zastosowaniach, zamiast podstawowego algorytmu spadku gradientu, stosuje się jego bardziej zaawansowane warianty, tzw. optymalizatory, takie jak Adagrad, RMSprop, czy obecnie najpopularniejszy *Adam* (Adaptive Moment Estimation), które adaptacyjnie dostosowują tempo uczenia dla każdego parametru, co znacznie przyspiesza i stabilizuje proces zbieżności [23].

5.4 Metodyka oceny i walidacji modeli

Opracowanie skutecznego modelu uczenia maszynowego wymaga rygorystycznej metodyki oceny, która pozwala na obiektywne zmierzenie jego zdolności do generalizacji, czyli poprawnego działania na nowych, niewidzianych wcześniej danych. Kluczowym elementem tej metodyki jest strategiczny podział posiadanego zbioru danych oraz wybór odpowiednich metryk oceny.

5.4.1 Podział zbioru danych.

Przygotowany zbiór danych jest standardowo dzielony na trzy niezależne podzbiory [24]:

- **Zbiór uczący (ang. *training set*):** Wykorzystywany bezpośrednio do treningu, czyli iteracyjnej optymalizacji parametrów modelu.
- **Zbiór walidacyjny (ang. *validation set*):** Służy do monitorowania procesu uczenia oraz do strojenia hiperparametrów modelu,
- **Zbiór testowy (ang. *test set*):** Używany tylko raz, na samym końcu, do przeprowadzenia ostatecznej, bezstronnej oceny wytrenowanego modelu. Wynik na tym zbiorze jest raportowany jako finalna miara skuteczności.

Bardziej zaawansowaną i statystycznie bardziej wiarygodną techniką walidacji jest *walidacja krzyżowa* (ang. *cross-validation*). W jej najpopularniejszej formie, k-krotnej (ang. *k-fold*), zbiór danych jest dzielony na k równych części. Proces treningu i walidacji jest

[4]Reguła łańcuchowa to fundamentalna zasada rachunku różniczkowego, która pozwala na obliczenie pochodnej funkcji złożonej. W kontekście sieci neuronowych, umożliwia ona dekompozycję skomplikowanej pochodnej funkcji kosztu na szereg prostszych, lokalnych pochodnych w poszczególnych warstwach sieci.

powtarzany k razy – za każdym razem inna część służy jako zbiór walidacyjny, a pozostałe $k-1$ jako zbiór uczący. Ostateczny wynik jest uśredniany ze wszystkich k przebiegów, co daje znacznie bardziej stabilną i wiarygodną ocenę modelu.

Głównym celem podziału danych na zbiór uczący i walidacyjny jest monitorowanie i przeciwdziałanie zjawisku *przeuczenia* (ang. *overfitting*). Przeuczenie ma miejsce, gdy model, zamiast uczyć się ogólnych wzorców, zaczyna "zapamiętywać"specyficzne, losowe szumy obecne w zbiorze uczącym. Taki model osiąga znakomite wyniki na danych, na których był trenowany, ale traci zdolność do generalizacji. Klasycznym symptomem przeuczenia jest sytuacja, w której błąd na zbiorze uczącym wciąż maleje, podczas gdy błąd na zbiorze walidacyjnym zaczyna rosnąć. Jedną z najskuteczniejszych technik walki z tym zjawiskiem jest *wczesne zatrzymanie* (ang. *early stopping*), które polega na przerwaniu treningu w momencie, gdy błąd na zbiorze walidacyjnym przestaje maleć i zapisaniu modelu z tego punktu.

5.4.2 Metody oceny jakości modelu

Do ilościowej oceny skuteczności modelu w zadaniach regresji, wykorzystuje się szereg metryk statystycznych. Najważniejsze z nich to:

- **Błąd średniokwadratowy (MSE – Mean Squared Error):** Obliczany jako średnia z kwadratów różnic między wartościami przewidywanymi a rzeczywistymi. Ze względu na podnoszenie do kwadratu, metryka ta jest szczególnie wrażliwa na duże błędy (wartości odstające).

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2 \tag{20}$$

- **Pierwiastek błędu średniokwadratowego (RMSE – Root Mean Squared Error):** Jest to pierwiastek kwadratowy z MSE. Jego zaletą jest to, że wynik jest wyrażony w tych samych jednostkach co zmienna docelowa, co ułatwia interpretację. W niniejszej pracy będzie to główna miara dokładności.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2} \tag{21}$$

- **Średni błąd bezwzględny (MAE – Mean Absolute Error):** Obliczany jako średnia z wartości bezwzględnych błędów. W przeciwieństwie do MSE/RMSE, MAE jest mniej wrażliwy na wartości odstające.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i| \tag{22}$$

gdzie n to liczba próbek, $\hat{y}_i$ to wartość przewidziana przez model, a y_i to wartość rzeczywista. Prawidłowe zastosowanie tych metryk na niezależnym zbiorze testowym jest fundamentem wiarygodnej oceny każdego modelu predykcyjnego.

5.5 Zastosowanie sieci neuronowych w zadaniu estymacji stanu

Po przedstawieniu teoretycznych podstaw działania sztucznych sieci neuronowych, możliwe jest precyzyjne sformułowanie, w jaki sposób techniki te mogą być zastosowane do rozwiązania problemu estymacji stanu systemu elektroenergetycznego. Jak wykazano, zadanie to można zdefiniować jako problem nieliniowej regresji, co doskonale wpisuje się w możliwości i architekturę sieci neuronowych.

Zdolność wielowarstwowych perceptronów (MLP) do aproksymacji dowolnie złożonych funkcji nieliniowych [20] czyni je naturalnym kandydatem do modelowania złożonych zależności między pomiarami a stanem sieci, które w metodach analitycznych są opisane za pomocą równań rozpływu mocy. Sieć neuronowa, w przeciwieństwie do estymatora WLS, nie wymaga jawnego sformułowania tych równań ani analitycznego obliczania macierzy Jacobiego. Zamiast tego, uczy się ona tej zależności w sposób end-to-end, bezpośrednio z danych.

W praktycznej implementacji, model sieci neuronowej dla estymacji stanu przyjmuje następującą strukturę:

- **Warstwa wejściowa** sieci ma liczbę neuronów odpowiadającą wymiarowi wektora pomiarów z. Każdy neuron wejściowy przyjmuje wartość jednego, konkretnego pomiaru (np. napięcia w danym węźle, mocy czynnej w danej linii).

- **Warstwy ukryte** (jedna lub więcej) o odpowiednio dobranej liczbie neuronów i funkcji aktywacji przetwarzają te dane, ucząc się wyodrębniać z nich ukryte, nieliniowe wzorce i korelacje.

- **Warstwa wyjściowa** składa się z $2n - k$ neuronów (gdzie n to liczba węzłów, a k to liczba węzłów odniesienia), co odpowiada wymiarowi estymowanego wektora stanu x. Każdy neuron wyjściowy przewiduje wartość jednej zmiennej stanu

Proces uczenia tak zdefiniowanej sieci odbywa się zgodnie z metodyką opisaną w poprzednich podrozdziałach. Na podstawie dużego, syntetycznego zbioru danych, wygenerowanego za pomocą symulacji rozpływu mocy, model jest trenowany z wykorzystaniem algorytmu propagacji wstecznej i optymalizatora w celu minimalizacji funkcji kosztu, takiej jak MSE. Proces ten jest monitorowany na zbiorze walidacyjnym w celu uniknięcia przeuczenia i doboru optymalnych hiperparametrów. Ostateczna ocena jakości estymatora neuronowego jest przeprowadzana na zbiorze testowym przy użyciu metryk regresyjnych,

takich jak RMSE, co pozwala na bezpośrednie porównanie jego dokładności z innymi podejściami.

Główną, potencjalną przewagą estymatora opartego na sieciach neuronowych nad metodami analitycznymi jest szybkość działania w fazie inferencji[5]. Po zakończeniu długotrwałego procesu treningu, wyestymowanie stanu dla nowego zestawu pomiarów sprowadza się do jednego, szybkiego przejścia sygnału "w przód"przez sieć (propagacja w przód), co jest operacją znacznie mniej kosztowną obliczeniowo niż iteracyjne rozwiązywanie układu równań w metodzie WLS. Ta cecha czyni sieci neuronowe obiecującą alternatywą, szczególnie w kontekście aplikacji czasu rzeczywistego w wielkoskalowych systemach elektroenergetycznych.

[5]to proces, w którym wytrenowany model sztucznej inteligencji wykorzystywany jest do predykcji na podstawie nowych danych

6 Przegląd technik ML stosowanych w modelowaniu systemów elektroenergetycznych

Jak wykazano w poprzednich rozdziałach, metody estymacji stanu mimo swojej skuteczności, opierają się na precyzyjnym, analitycznym modelu który odwzorowuje fizykę sieci. W obliczu rosnącej złożoności systemów, dużej liczby pomiarów oraz potencjalnych niedoskonałości w modelach, coraz większe zainteresowanie budzą techniki oparte na uczeniu maszynowym (ML - Machine Learning). Ich fundamentalną zaletą jest zdolność do uczenia się złożonych, nieliniowych zależności bezpośrednio z danych, bez konieczności jawnego programowania reguł fizycznych. W zadaniu estymacji stanu problem ten można sformułować jako zadanie uczenia nadzorowanego, gdzie celem jest stworzenie modelu, który na podstawie wejściowego wektora pomiarowego jest w stanie z dużą dokładnością przewidzieć wektor wyjściowy (stan systemu). W niniejszym rozdziale dokonano przeglądu kluczowych technik ML, które zostały zastosowane w tym obszarze, od modeli regresyjnych po zaawansowane architektury sieci neuronowych.

6.1 Problem estymacji stanu jako zadanie uczenia nadzorowanego

Przejście od analitycznych metod estymacji stanu do technik opartych na uczeniu maszynowym stanowi fundamentalną zmianę paradygmatu modelowania. Zamiast opierać się na jawnym, fizycznym modelu systemu, którego parametry muszą być dokładnie znane, podejście ML traktuje estymator jako system uczący się. Jego celem jest samodzielne odkrycie i zamodelowanie złożonych, nieliniowych zależności, które istnieją pomiędzy zbiorem dostępnych pomiarów a rzeczywistym stanem sieci. W tym ujęciu problem estymacji jest przeformułowany na zadanie uczenia nadzorowanego (supervised learning).

Uczenie nadzorowane to jedna z trzech głównych gałęzi uczenia maszynowego, obok uczenia nienadzorowanego i uczenia przez wzmacnianie [25]. Jego istotą jest tworzenie modelu predykcyjnego na podstawie zbioru danych zawierającego tak zwane "przykłady uczące". Każdy przykład składa się z pary: obiektu wejściowego (wektora cech) oraz przypisanej mu, znanej "poprawnej odpowiedzi"(etykiety lub wartości docelowej). Algorytm, analizując te pary, uczy się generalizować, czyli odkrywać wzorce, które pozwolą mu na przewidywanie poprawnych odpowiedzi dla nowych, niewidzianych wcześniej danych wejściowych. Innymi słowy, celem jest aproksymacja nieznanej funkcji f, która mapuje wektor wejściowy X na wektor wyjściowy y, tak że $y \approx f(X)$. "Nadzór"w tym procesie polega właśnie na dostarczaniu modelowi poprawnych odpowiedzi, które służą jako sygnał zwrotny do korygowania jego błędów podczas treningu.

W kontekście estymacji stanu systemu elektroenergetycznego, ta koncepcja przekłada

się bezpośrednio na następującą strukturę problemu:

- **Wektor wejściowy X:** Jest to wektor wszystkich dostępnych, surowych pomiarów z systemu (np. moduły napięć, przepływy mocy czynnej i biernej).
- **Wektor wyjściowy y:** Jest to wektor docelowy, czyli "poprawna odpowiedź", którą model ma za zadanie odtworzyć. W tym przypadku jest to pełny, bezbłędny wektor stanu systemu, składający się z modułów napięć i kątów fazowych we wszystkich węzłach sieci.

Zadaniem modelu uczenia maszynowego jest zatem nauczenie się nieliniowej funkcji, która potrafi odfiltrować szum i zrekonstruować pełny, spójny stan systemu na podstawie niekompletnych i niedokładnych danych wejściowych. Zasadniczą przewagą takiego podejścia jest jego potencjalna niezależność od precyzyjnego modelu fizycznego – model ML może nauczyć się zachowania systemu nawet wtedy, gdy dokładne parametry linii są nieznane lub zmieniają się w czasie, o ile zjawiska te są odpowiednio reprezentowane w danych uczących.

6.2 Wielowarstwowe perceptrony (MLP)

Historycznie, wielowarstwowe perceptrony (MLP) stanowią fundamentalną i jedną z pierwszych architektur głębokiego uczenia, która umożliwiła modelowanie złożonych, nieliniowych zależności w danych [18]. Architektura ta jest bezpośrednim rozwinięciem idei pojedynczego perceptronu, organizując neurony w sekwencyjną strukturę składającą się z wielu połączonych ze sobą warstw.

Strukturę MLP definiują trzy kluczowe komponenty: warstwa wejściowa, jedna lub więcej warstw ukrytych oraz warstwa wyjściowa. Cechą charakterystyczną tej architektury jest to, że neurony w każdej warstwie są połączone z *wszystkimi* neuronami w warstwie poprzedniej, tworząc gęstą sieć połączeń. Informacja w sieci MLP przepływa w jednym kierunku (ang. *feedforward*), od wejścia do wyjścia.

Siła MLP leży w zdolności do uczenia się hierarchicznej reprezentacji danych. Każda kolejna warstwa ukryta operuje na wyjściach warstwy poprzedniej, ucząc się coraz bardziej przetworzonych danych. Ta zdolność do automatycznej ekstrakcji cech jest jedną z fundamentalnych przewag głębokich sieci neuronowych nad innymi metodami.

Mimo swojej uniwersalności, MLP posiadają również wady. Są one podatne na przeuczenie, zwłaszcza przy dużej liczbie parametrów i ograniczonej ilości danych, co wymaga stosowania technik regularyzacji (np. Dropout, regularyzacja L1/L2). Ponadto, ich trening jest procesem kosztownym obliczeniowo, wymagającym dużych zbiorów danych i często specjalistycznego sprzętu (GPU). Najważniejszym ograniczeniem w kontekście systemów

elektroenergetycznych jest jednak fakt, że architektura MLP jest niezmienna względem permutacji cech wejściowych – traktuje ona wektor pomiarów jako nieuporządkowany zbiór liczb, całkowicie ignorując fizyczną topologię i strukturę grafową sieci, co zostanie szerzej omówione w dalszej części rozdziału.

6.3 Modele oparte na zespołach drzew decyzyjnych

Jedną z najskuteczniejszych i najczęściej stosowanych grup algorytmów w zadaniach regresji na danych tabelarycznych są modele oparte na *zespołach drzew decyzyjnych* (ang. *ensemble methods*). Ich siła nie leży w pojedynczym, skomplikowanym modelu, ale w agregacji predykcji pochodzących z wielu prostszych modeli bazowych, którymi są drzewa decyzyjne. Dwa najważniejsze podejścia w tej klasie to Lasy Losowe oraz modele typu Gradient Boosting.

6.3.1 Lasy Losowe (ang. *Random Forest*).

Algorytm Lasów Losowych, zaproponowany przez Leo Breimana [26], opiera się na koncepcji *bootstrap aggregating*, znanej również jako *bagging*. Proces tworzenia modelu przebiega następująco: ze zbioru uczącego wielokrotnie losowane są (ze zwracaniem) mniejsze podzbiory danych. Dla każdego takiego podzbioru trenowane jest niezależne, w pełni "rozwinięte"drzewo decyzyjne. Dodatkowo, w każdym węźle drzewa, podczas poszukiwania optymalnego podziału, algorytm nie bierze pod uwagę wszystkich dostępnych cech, ale jedynie ich losowy podzbiór. Te dwa mechanizmy losowości (losowanie próbek i losowanie cech) zapewniają, że poszczególne drzewa w "lesie"są od siebie zróżnicowane i nieskorelowane. W fazie predykcji, dla nowego wektora pomiarów, każde drzewo w lesie niezależnie przewiduje wartość wyjściową. Ostateczna predykcja całego modelu jest po prostu średnią arytmetyczną predykcji ze wszystkich drzew. Dzięki temu procesowi uśredniania, Lasy Losowe są niezwykle odporne na przeuczenie i charakteryzują się wysoką stabilnością oraz dokładnością.

6.3.2 Gradient Boosting Machines.

Drugie podejście, znane jako *boosting*, opiera się na innej filozofii. Zamiast budować wiele niezależnych modeli, algorytmy typu *Gradient Boosting* [27] tworzą zespół drzew w sposób sekwencyjny. Pierwsze drzewo jest trenowane w standardowy sposób. Następnie, drugie drzewo jest uczone tak, aby przewidywać i korygować błędy (rezydua) popełnione przez pierwsze drzewo. Trzecie drzewo uczy się korygować błędy popełnione przez zespół dwóch pierwszych drzew, i tak dalej. Każdy kolejny model w zespole skupia się na tych próbkach, z którymi poprzednie modele radziły sobie najgorzej. Ostateczna predykcja

jest sumą predykcji wszystkich drzew w sekwencji. Najpopularniejszą i najskuteczniejszą implementacją tej idei jest algorytm XGBoost (Extreme Gradient Boosting), który dzięki licznym optymalizacjom i technikom regularyzacji stał się jednym z najpotężniejszych narzędzi w uczeniu maszynowym [28].

W kontekście SE, modele te oferują kilka zalet: są wysoce nieliniowe, dobrze radzą sobie z dużą liczbą cech wejściowych i są stosunkowo łatwe w strojeniu. Ich głównym ograniczeniem jest jednak fakt, że, podobnie jak inne modele regresyjne, traktują one wektor pomiarów jako płaską, nieustrukturyzowaną listę liczb, ignorując fizyczne powiązania i topologię sieci elektroenergetycznej.

6.4 Ograniczenia standardowych architektur i potrzeba uwzględnienia topologii

Mimo iż modele takie jak wielowarstwowe perceptrony (MLP) oraz metody oparte na zespołach drzew decyzyjnych oferują potężne mechanizmy do modelowania złożonych zależności, w kontekście systemów elektroenergetycznych posiadają one fundamentalne, wrodzone ograniczenie. Problem ten nie leży w ich zdolnościach do aproksymacji, ale w sposobie, w jaki postrzegają one dane wejściowe.

Zarówno MLP, jak i Lasy Losowe czy XGBoost, operują na danych w formie wektorów cech. Oznacza to, że zbiór wszystkich pomiarów z systemu jest "spłaszczany"do jednowymiarowej, nieustrukturyzowanej listy liczb, która następnie jest podawana na wejście modelu. W takim ujęciu, model nie posiada żadnej inherentnej wiedzy o tym, skąd pochodzi dany pomiar ani jakie są jego fizyczne relacje z innymi. Pomiary z dwóch elektrycznie sąsiadujących ze sobą węzłów są traktowane w dokładnie taki sam sposób, jak pomiary z dwóch węzłów znajdujących się na przeciwległych końcach systemu przesyłowego. Algorytmy te są niezmienne względem permutacji cech wejściowych – zamiana miejscami dwóch kolumn w zbiorze danych (np. pomiaru mocy z węzła A i węzła B) nie wpłynie na proces uczenia, co jest całkowicie sprzeczne z fizyką systemu elektroenergetycznego.

Ta niezdolność do uwzględnienia *topologii sieci* jest kluczową wadą. Stan systemu w danym węźle, zgodnie z prawami Kirchhoffa, jest najsilniej zdeterminowany przez stany i przepływy w jego najbliższym sąsiedztwie. Standardowe architektury ML, ignorując tę fundamentalną zasadę, muszą uczyć się tych zależności w sposób pośredni i wysoce nieefektywny, jedynie na podstawie korelacji statystycznych w danych. Co więcej, model wytrenowany dla jednej, konkretnej topologii sieci, staje się w dużej mierze bezużyteczny w przypadku jej zmiany (np. po wyłączeniu linii). Musiałby on zostać potencjalnie przetrenowany od nowa, ponieważ zmieniłaby się struktura i znaczenie wektora wejściowego [29].

Te ograniczenia wskazują na potrzebę zastosowania nowej klasy modeli, które są w stanie w naturalny sposób przetwarzać dane o strukturze grafowej. System elektroenergetyczny jest w swojej istocie grafem, gdzie węzły (szyny zbiorcze) są połączone krawędziami (liniami i transformatorami). Architektury zdolne do bezpośredniego operowania na takiej reprezentacji, takie jak *grafowe sieci neuronowe (GNN)*, oferują obiecującą alternatywę. Posiadają one potencjał do uczenia się reguł, które są bardziej fundamentalne, ponieważ uwzględniają lokalne, fizyczne interakcje zdefiniowane przez topologię sieci.

7 Wprowadzenie do grafowych sieci neuronowych i ich potencjału w analizie topologii sieci

Fundamentalnym krokiem umożliwiającym zastosowanie architektur neuronowych zdolnych do przetwarzania danych strukturalnych jest formalne przedstawienie systemu elektroenergetycznego w postaci grafu. Podejście to pozwala na przezwyciężenie ograniczeń modeli operujących na danych wektorowych, poprzez jawne uwzględnienie fizycznych relacji oraz topologii połączeń, które są kluczowe dla poprawnego odwzorowania praw fizyki rządzących pracą systemu.

System elektroenergetyczny może być w sposób naturalny zamodelowany jako *graf ważony* (*attributed graph*), zdefiniowany jako struktura $G = (V, E)$, której towarzyszą macierze atrybutów węzłów i krawędzi. Poszczególne zbiory i ich desygnaty w kontekście elektroenergetycznym definiuje się następująco:

- $V = \{v_1, v_2, \ldots, v_N\}$ jest zbiorem *wierzchołków (węzłów)*, gdzie N to całkowita liczba węzłów w analizowanym systemie. Każdy wierzchołek $v_i \in V$ odpowiada fizycznemu elementowi sieci, takiemu jak szyna zbiorcza w stacji elektroenergetycznej, punkt przyłączenia jednostki wytwórczej lub węzeł obciążeniowy. Zbiór ten stanowi dyskretną reprezentację kluczowych punktów systemu.

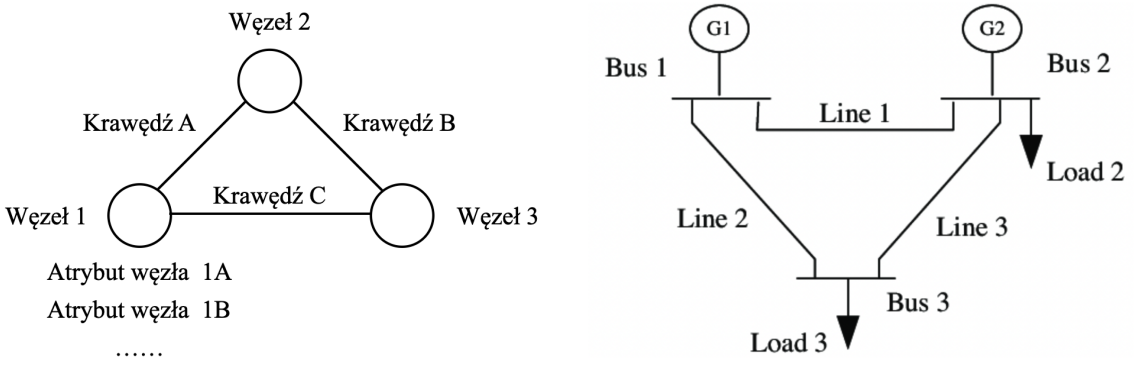
- $E \subseteq V \times V$ jest zbiorem **krawędzi**, które reprezentują fizyczne połączenia pomiędzy wierzchołkami. Krawędź $(v_i, v_j) \in E$ istnieje wtedy i tylko wtedy, gdy węzły i oraz j są bezpośrednio połączone elementem przesyłowym, takim jak linia napowietrzna, linia kablowa lub transformator. Struktura połączeń w grafie jest izomorficzna z topologią analizowanej sieci elektroenergetycznej.

Kluczowe dla zastosowań w uczeniu maszynowym jest przypisanie do elementów grafu cech ilościowych:

- **Atrybuty wierzchołków**: Każdemu wierzchołkowi v_i przypisany jest wektor cech $x_i \in R^{D_v}$, gdzie D_v to wymiar przestrzeni cech. Wektor ten zawiera dostępne w danym węźle informacje pomiarowe (np. moduł napięcia V_i, moce czynne i bierne P_i, Q_i) lub znane parametry (np. typ węzła – PQ, PV, slack).

- **Atrybuty krawędzi**: Analogicznie, każdej krawędzi (v_i, v_j) można przypisać wektor cech $e_{ij} \in R^{D_e}$, który zazwyczaj zawiera parametry fizyczne danego elementu, takie jak jego admitancja (przewodność G_{ij} i susceptancja B_{ij}), determinujące przepływ mocy między węzłami.

Przykład 3-węzłowego grafu oraz odpowiadający mu model sieci wykorzystany w literaturze [30] zaprezentowano na rysunku 5



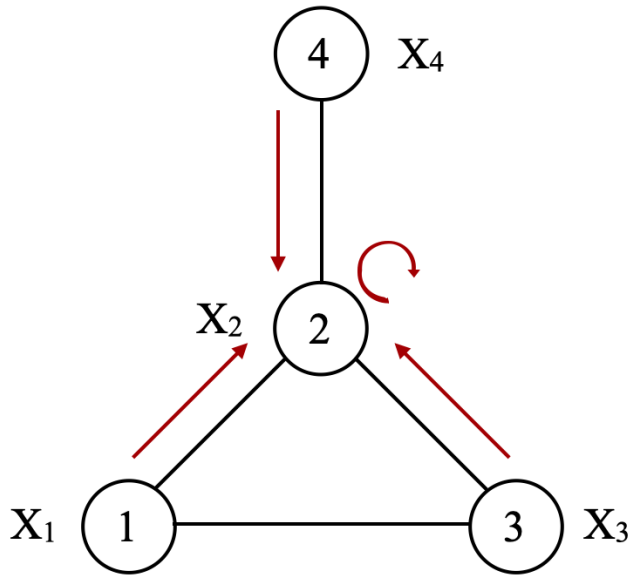
Rysunek 5: Przykładowy graf wraz z opisem oraz model sieci dostępny w literaturze

W ramach tak zdefiniowanej struktury problem estymacji stanu zostaje przeformułowany. Zamiast zadania regresji, w którym model usiłuje zmapować płaski wektor pomiarów na wektor stanu, staje się on znacznie bardziej złożonym i bogatszym informacyjnie zadaniem *predykcji atrybutów na grafie*. Celem modelu jest estymacja nieznanych atrybutów węzłów (tworzących wektor stanu $x = [\theta^T, V^T]^T$) na podstawie dostępnych pomiarów (atrybuty węzłów) oraz znanej struktury połączeń i parametrów fizycznych (atrybuty krawędzi).

Paradygmat modelowania systemów w postaci grafów jest podejściem o ugruntowanej pozycji w wielu dziedzinach nauki. W chemii obliczeniowej i bioinformatyce struktury molekularne są analizowane jako grafy, co pozwala na predykcję właściwości związków chemicznych i badanie interakcji białek. W analizie sieci społecznych grafy modelują dynamikę propagacji informacji. W badaniach operacyjnych i logistyce teoria grafów stanowi fundament dla algorytmów optymalizacji przepływów w sieciach transportowych i telekomunikacyjnych. Szerokie i skuteczne zastosowanie tej metodyki w różnorodnych domenach potwierdza jej uniwersalność i potencjał jako formalizmu do opisu złożonych systemów, w tym systemów elektroenergetycznych.

7.1 Mechanizm przekazywania wiadomości (Message Passing)

Przekazywanie wiadomości to fundamentalna operacja leżąca u podstaw działania większości architektur grafowych sieci neuronowych (GNN). Jest to proces, w którym węzły grafu iteracyjnie aktualizują swoje stany (reprezentacje wektorowe) poprzez wymianę informacji z bezpośrednimi sąsiadami. Znaczenie tego mechanizmu polega na tym, że pozwala on sieci neuronowej na uwzględnienie topologii grafu – model uczy się nie tylko na podstawie cech poszczególnych węzłów, ale również na podstawie struktury połączeń między nimi. Dzięki temu GNN jest w stanie tworzyć bogate, kontekstowe reprezentacje, które odzwierciedlają zarówno lokalne, jak i (po kilku warstwach) globalne właściwości sieci.



Rysunek 6: mechanizm przekazywania wiadomości dla węzła docelowego nr 2.

$$H_{2,L+1} = \sigma\left(\left(X_{1,L} + X_{3,L} + X_{4,L} + X_{2,L}\right) W_L\right) \tag{23}$$

Czerwone strzałki obrazują przepływ informacji (reprezentacji wektorowych X_i) od sąsiednich węzłów (1, 3, 4) oraz z pętli własnej (uwzględnienie informacji o samym węźle 2). Równanie 23 opisuje ten proces: reprezentacje z warstwy L są sumowane (agregowane), następnie transformowane przez uczącą się macierz wag W_L i przepuszczane przez nieliniową funkcję aktywacji σ, aby obliczyć nową, zaktualizowaną reprezentację $H_{2,L+1}$ dla węzła 2 w kolejnej warstwie.

W celu efektywnej implementacji obliczeniowej operacji w grafowych sieciach neuronowych stosuje się zwięzłą notację algebry liniowej, czyli macierzową reprezentację modelu. Kluczowe jest przedstawienie zarówno atrybutów węzłów, jak i topologii grafu w ustandaryzowanej formie macierzowej. Podstawą tej reprezentacji są dwie macierze: macierz cech oraz macierz sąsiedztwa.

Macierz cech (ang. Feature Matrix), oznaczana jako X, jest macierzą o wymiarach $N \times D$, gdzie N to liczba węzłów w grafie, a D to wymiar przestrzeni cech. Każdy i-ty wiersz tej macierzy stanowi wektor cech X_i dla i-tego węzła, agregując w ten sposób wszystkie dane wejściowe w jedną strukturę. Topologia grafu jest natomiast kodowana za pomocą *macierzy sąsiedztwa (ang. Adjacency Matrix)* A o wymiarach $N \times N$. Jest to macierz binarna, w której element A_{ij} przyjmuje wartość 1, jeśli między węzłami i oraz j istnieje krawędź, a 0 w przeciwnym razie. Macierz ta stanowi kompletny, matematyczny opis struktury połączeń w sieci.

Dla analizowanego na rysunku 6 przypadku sieci 4-węzłowej, macierz sąsiedztwa A przyjmuje następującą postać:

$$A = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}$$

gdzie poszczególne wiersze i kolumny odpowiadają kolejnym węzłom grafu. Przykładowo, drugi wiersz macierzy jednoznacznie wskazuje, że węzeł nr 2 jest połączony z węzłami 1, 3 oraz 4, natomiast wartość binarna 1 na przekątnej nie świadczy o połączeniu węzła z samym sobą, a przekazaniu informacji .

Mając zdefiniowaną macierz cech X oraz macierz sąsiedztwa A, możemy w elegancki sposób zrealizować krok agregacji wiadomości za pomocą pojedynczej operacji mnożenia macierzy. Taka operacja jest fundamentalnym blokiem budulcowym dla wielu architektur GNN.

Podstawowa forma agregacji, w której każdy węzeł sumuje wektory cech swoich sąsiadów, jest równoważna operacji:

$$Z = A \cdot X \tag{24}$$

gdzie A to macierz sąsiedztwa, a X to macierz cech z poprzedniej warstwy. Wynikowa macierz Z będzie miała wymiary $N \times D$, a każdy jej wiersz Z_i będzie sumą wektorów cech sąsiadów węzła i.

Aby uwzględnić w procesie uczenia transformację tych zagregowanych cech, wprowadzamy uczącą się *macierz wag* W. Pełna operacja dla jednej, prostej warstwy GNN, realizującej sumowanie wiadomości, przyjmuje postać:

$$\underset{\text{Cechy wyjściowe}}{H_{L+1}} = \sigma \left(\underset{\substack{\text{Macierz} \\ \text{sąsiedztwa}}}{A} \cdot \underset{\substack{\text{Cechy} \\ \text{wejściowe}}}{H_L} \cdot \underset{\substack{\text{Parametry} \\ \text{uczące się}}}{W_L} \right) \tag{25}$$

Równanie to stanowi symboliczną reprezentację operacji macierzowej dla warstwy GNN. Opisy pod symbolami ilustrują rolę każdej z macierzy w procesie transformacji: macierz sąsiedztwa A koduje topologię, macierz H_L dostarcza cechy wejściowe, a ucząca się macierz wag W_L transformuje zagregowane informacje (dla pierwszej warstwy $H_0 = X$), tworząc macierz wyjściową H_{L+1}.

Chociaż przedstawione podejście, uwzględniające już pętle własne w macierzy sąsiedztwa, jest proste i wydajne obliczeniowo, wciąż posiada jedno fundamentalne ograniczenie. Stopień węzła (liczba jego sąsiadów) nie jest brany pod uwagę w procesie agregacji. Prowadzi to do problemu niestabilności numerycznej: wektory cech węzłów o

dużej liczbie połączeń będą miały nieproporcjonalnie większą skalę niż wektory węzłów o małej liczbie połączeń, co może zdominować i zaburzyć proces uczenia. Problem ten jest rozwiązany w bardziej zaawansowanych architekturach, takich jak Graph Convolutional Network (GCN), poprzez wprowadzenie etapu normalizacji, który skaluje wkład każdego sąsiada w zależności od stopnia danego węzła.

Problem niestabilności numerycznej, wynikający z sumowania cech w procesie agregacji, jest rozwiązywany poprzez normalizację. Najprostsza forma normalizacji polega na uśrednieniu wektorów cech sąsiadów, co w języku algebry liniowej jest równoznaczne z przemnożeniem macierzy sąsiedztwa A (uwzględniającej pętle własne) przez odwrotność jej macierzy stopni D.

Macierz stopni D jest macierzą diagonalną, w której element na przekątnej D_{ii} jest równy stopniowi węzła i (liczbie jego połączeń, wliczając pętlę własną). Operacja mnożenia $D^{-1}A$ skutkuje powstaniem nowej, znormalizowanej wierszowo macierzy, w której każdy element j-ty w wierszu i jest dzielony przez stopień węzła i. W efekcie, agregacja przestaje być sumą, a staje się operacją uśredniania, co stabilizuje proces uczenia poprzez uniezależnienie skali wektorów wyjściowych od stopnia węzłów.

Dla analizowanego grafu 4-węzłowego, którego macierz sąsiedztwa A (z pętlami własnymi) i macierz stopni D mają postać:

$$A = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix} \quad , \quad D = \begin{pmatrix} 3 & 0 & 0 & 0 \\ 0 & 4 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}$$

Wynikowa macierz znormalizowana, $A_{norm} = D^{-1}A$, która uwzględnia "wagi"wynikające z normalizacji, wygląda następująco:

$$A_{norm} = \begin{pmatrix} 1/3 & 1/3 & 1/3 & 0 \\ 1/4 & 1/4 & 1/4 & 1/4 \\ 1/3 & 1/3 & 1/3 & 0 \\ 0 & 1/2 & 0 & 1/2 \end{pmatrix}$$

Każdy wiersz tej macierzy sumuje się do jedności, co potwierdza, że operacja agregacji przy użyciu A_{norm} jest formą uśredniania.

Chociaż przedstawiona normalizacja asymetryczna ($D^{-1}A$) skutecznie rozwiązuje problem skalowania, w architekturze GCN wprowadzono bardziej zaawansowaną *normalizację symetryczną*[31]. Zapewnia ona lepszą stabilność i wydajność w praktyce. Finalna formuła dla warstwy GCN, która jest standardem w literaturze, przyjmuje postać:

$$H_{L+1} = \sigma\left(D^{-1/2}AD^{-1/2}H_L W_L\right) \tag{26}$$

gdzie $A = A + I$ to macierz sąsiedztwa z pętlami własnymi, a D to jej macierz stopni.

W niniejszym rozdziale przedstawiono teoretyczne podstawy zastosowania grafowych sieci neuronowych w problemie estymacji stanu. Rozpoczynając od reprezentacji systemu elektroenergetycznego w postaci grafu, przez szczegółowe omówienie mechanizmu przekazywania wiadomości i jego macierzowej implementacji, dochodząc do zaawansowanej architektury Graph Convolutional Network. Wykazano, w jaki sposób GCN, poprzez wprowadzenie pętli własnych i mechanizmu normalizacji, rozwiązuje fundamentalne problemy prostej agregacji, tworząc stabilne i efektywne narzędzie do uczenia na danych o strukturze sieciowej.

8 Modelowanie systemu elektroenergetycznego w środowisku Power Factory

8.1 Charakterystyka i struktura danych uczących

Skuteczność modelu uczenia maszynowego jest nierozerwalnie związana z jakością i reprezentatywnością danych, na których jest on trenowany. Duża ilość danych uczących, nawet w połączeniu ze słabym, źle stworzonym algorytmem sieci neuronowej, będzie dawała lepsze rezultaty niż wysublimowane struktury obliczeniowe, które nie będą posiadać wystarczającej ilości danych do nauki, lub dane te będą niereprezentatywne. W przeciwieństwie do wielu innych dziedzin, w systemach elektroenergetycznych pozyskanie dużego, zróżnicowanego zbioru historycznych pomiarów z przypisanym im, znanym, prawdziwym stanem systemu jest w praktyce niemożliwe. Prawdziwy, bezbłędny wektor stanu nigdy nie jest w pełni znany w warunkach operacyjnych – jest on właśnie wielkością, którą próbujemy oszacować. Z tego powodu standardową i powszechnie akceptowaną praktyką w literaturze jest generowanie syntetycznego zbioru danych [32].

Proces ten polega na przeprowadzeniu setek tysięcy, a nawet milionów symulacji obliczeń rozpływu mocy dla szerokiego spektrum możliwych stanów pracy sieci. Każda pojedyncza symulacja tworzy jedną, kompletną parę uczącą (X, y). Kluczowym wyzwaniem jest zapewnienie, aby wygenerowany zbiór był wystarczająco zróżnicowany i pozwalał modelowi na naukę generalizacji, a nie tylko zapamiętanie kilku typowych scenariuszy. W tym celu, podczas generowania danych, wprowadza się losowość w kluczowych parametrach operacyjnych, takich jak:

- **Zmienność obciążeń:** Losowe zmiany mocy czynnej i biernej w węzłach odbiorczych, często modelowane na podstawie profili dobowych lub sezonowych, aby odzwierciedlić naturalne wahania zapotrzebowania na energię.
- **Zmienność generacji:** Wahania mocy generowanej, szczególnie istotne w przypadku odnawialnych źródeł energii (OZE), których praca jest z natury stochastyczna.
- **Zmiany topologiczne:** Symulacje awarii, czy wyłączeń elementów sieci (linii lub transformatorów), które zmieniają strukturę i parametry elektryczne systemu, ucząc model odporności na zmiany w topologii

Dla każdej tak wygenerowanej, unikalnej symulacji, wektor cech X jest tworzony poprzez dodanie do idealnych wyników rozpływu mocy losowego szumu, który modeluje niedokładność aparatury pomiarowej. Poziom tego szumu jest zazwyczaj oparty na znanych klasach dokładności rzeczywistych urządzeń. W ten sposób powstaje realistyczny

zbiór danych, który stanowi fundament, na którym model ML buduje swoją "wiedzę"o fizycznych prawach rządzących systemem elektroenergetycznym.

W niniejszej pracy, do generowania syntetycznego zbioru danych wykorzystane zostanie zaawansowane środowisko symulacyjne DIgSILENT PowerFactory, które pozwala na precyzyjne modelowanie i automatyzację złożonych scenariuszy pracy sieci.

8.2 Środowisko PowerFactory

DIgSILENT PowerFactory jest zintegrowanym, komercyjnym oprogramowaniem do analizy systemów elektroenergetycznych, które stanowi uznany standard przemysłowy oraz narzędzie badawcze w wiodących instytucjach na świecie. Jego wybór jako platformy do generowania danych podyktowany był szeregiem kluczowych cech, gwarantujących wysoką wierność i wiarygodność symulacji. Środowisko to operuje na centralnej bazie danych z obiektowym modelem, co zapewnia spójność i integralność parametrów dla wszystkich elementów sieci w ramach złożonych projektów badawczych. Taka struktura umożliwia efektywne zarządzanie wariantami i scenariuszami obliczeniowymi (tzw. study cases), co jest nieocenione przy prowadzeniu analiz parametrycznych na dużą skalę.

Jedną z fundamentalnych zalet oprogramowania (w porównaniu chociażby z przytoczonym wcześniej środowiskiem PandaPower) jest graficzny interfejs użytkownika, który umożliwia intuicyjne, wizualne modelowanie systemów za pomocą edytora schematów jednokreskowych. Metoda ta, w odróżnieniu od podejść czysto programistycznych, nie tylko znacząco przyspiesza proces tworzenia i modyfikacji złożonych topologii sieciowych, ale także minimalizuje ryzyko błędu ludzkiego, szczególnie w przypadku bardziej rozbudowanych modeli. Interfejs ten służy również jako potężne narzędzie do wizualizacji wyników, pozwalając na graficzną prezentację rozpływów mocy czy poziomów napięć bezpośrednio na schemacie, co ułatwia walidację poszczególnych symulacji. Sercem oprogramowania są numeryczne moduły obliczeniowe, pozwalające na realizację szerokiego spektrum analiz, w tym statycznych rozpływów mocy, analiz zwarciowych, analiz stabilności oraz symulacji dynamicznych.

Kluczową z perspektywy tej pracy funkcjonalnością jest jednak wbudowany interfejs programistyczny dla języka Python. Umożliwia on pełną automatyzację procesu symulacyjnego, który jest podstawą generowania zbioru uczącego. Skrypt sterujący, operując na modelu sieci, iteracyjnie modyfikuje jej parametry – takie jak wartości mocy czynnej i biernej w węzłach obciążeniowych, punkty pracy jednostek wytwórczych czy stany łączeniowe wybranych linii w celu symulacji zmian topologicznych. Dla każdej tak wygenerowanej, unikalnej konfiguracji, skrypt jest w stanie automatycznie uruchamiać obliczenia rozpływu mocy, a następnie eksportować uzyskane wyniki – pełny wektor stanu systemu (moduły i kąty napięć) oraz odpowiadające mu ‘pomiary‘ – do zewnętrznych plików w ustruktu-

ryzowanym formacie. Ta zdolność do prowadzenia w pełni zautomatyzowanych studiów parametrycznych była niezbędna do wygenerowania obszernego i zróżnicowanego zbioru uczącego, który jest warunkiem koniecznym do skutecznego treningu głębokich sieci neuronowych. Zastosowanie środowiska PowerFactory gwarantuje zatem, że dane treningowe stanowią wynik symulacji przeprowadzonych na modelach o wysokiej wierności fizycznej, zgodnych ze standardami stosowanymi w przemyśle elektroenergetycznym.

8.3 Metodyka budowy modelu sieci

W poprzednich rozdziałach przedstawiono metodykę budowy i analizy modelu sieci elektroenergetycznej z wykorzystaniem programistycznej biblioteki `pandapower`. W niniejszym rozdziale, dla zachowania spójności i pokazania alternatywnego podejścia, ten sam dwuwęzłowy problem zostanie zamodelowany i przeanalizowany w środowisku inżynierskim DIgSILENT PowerFactory. Celem jest zilustrowanie, w jaki sposób narzędzia z graficznym interfejsem użytkownika upraszczają proces modelowania i analizy w porównaniu do podejść czysto programistycznych. Zademonstrowane zostaną kolejne kroki tworzenia modelu, jego parametryzacji oraz wykonania obliczeń rozpływu mocy, które stanowią podstawę do generowania danych na potrzeby systemów uczenia maszynowego.

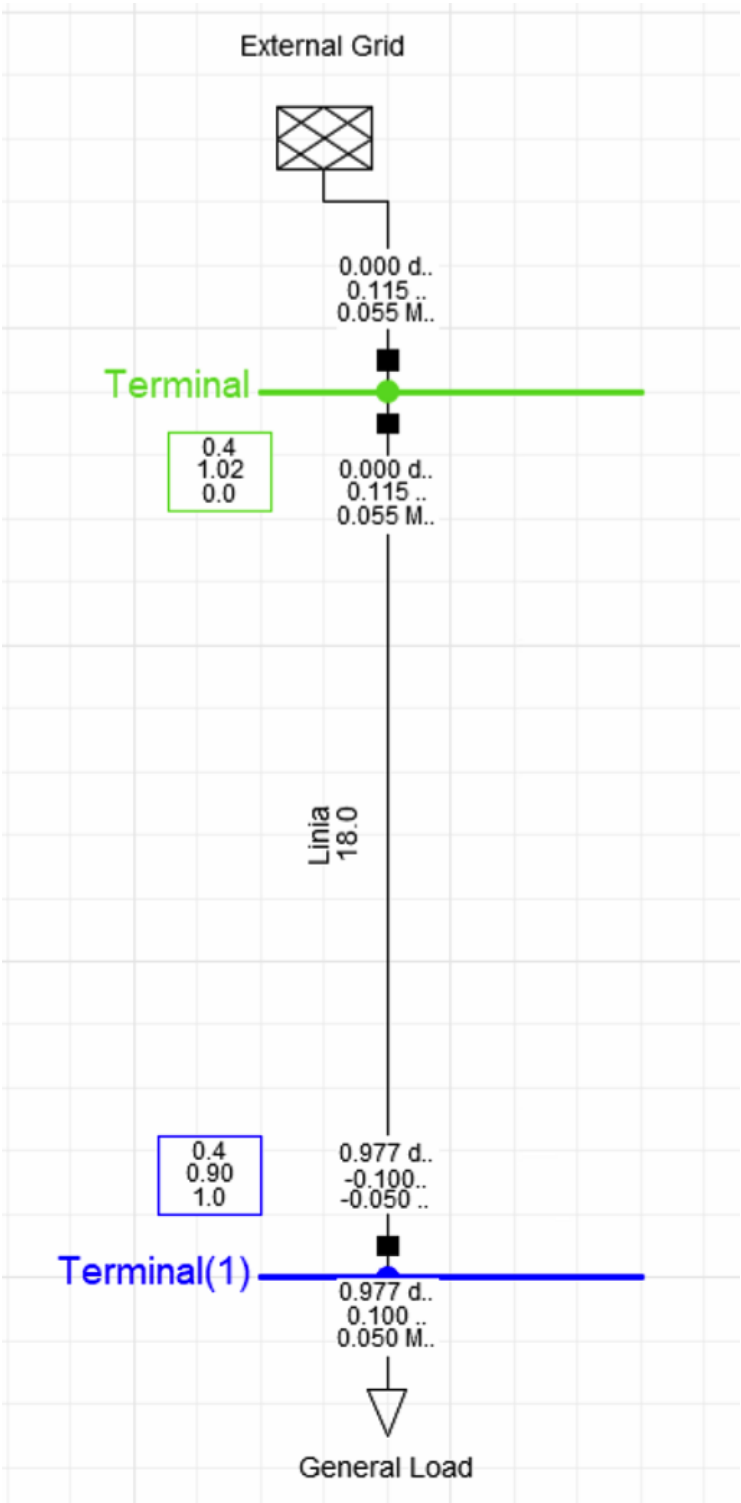
Proces tworzenia modelu referencyjnego w środowisku PowerFactory ma charakter wizualny i interaktywny. Kolejne etapy, analogiczne do tych zaimplementowanych w skrypcie Python, obejmowały definicję topologii sieci, parametryzację jej elementów oraz wykonanie obliczeń w celu wyznaczenia „prawdziwego stanu” systemu.

Opis modelu sieci. Podobnie jak w poprzednim eksperymencie, podstawę analizy stanowi prosty, dwuwęzłowy model sieci promieniowej, którego schemat w środowisku PowerFactory przedstawiono na Rysunku 7. Sieć składa się z:

- Węzła bilansującego (Slack Bus), reprezentowanego przez element Sieci Zewnętrznej (External Grid),
- Węzła odbiorczego (PQ Load Bus), do którego przyłączono Obciążenie (General Load),
- Linii przesyłowej (Line) łączącej oba węzły.

Parametry elementów są tożsame z tymi użytymi w analizie w bibliotece `pandapower`.

Weryfikacja modelu w PowerFactory polega na jego graficznej budowie i parametryzacji, co stanowi alternatywę dla definicji programistycznej. Poniżej przedstawiono kluczowe etapy tego procesu.



Rysunek 7: Schemat jednokreskowy analizowanej sieci dwuwęzłowej w środowisku DIgSILENT PowerFactory.

Definicja sieci referencyjnej. Proces modelowania w PowerFactory rozpoczyna się od umieszczenia na schemacie graficznym odpowiednich komponentów z biblioteki, a następnie ich wzajemnego połączenia. Zamiast wywoływania funkcji `pp.create_bus()` czy `pp.create_load()`, użytkownik przeciąga i upuszcza ikony reprezentujące szyny

zbiorcze, obciążenia, linie i sieć zewnętrzną. Taka wizualna reprezentacja topologii jest intuicyjna i mniej podatna na błędy.

Parametryzacja elementów. Kolejnym krokiem jest wprowadzenie parametrów elektrycznych dla każdego elementu. Odbywa się to poprzez okna dialogowe, które w sposób ustrukturyzowany prezentują wszystkie niezbędne pola do wypełnienia.

- Dla Sieci Zewnętrznej w zakładce *Load Flow* ustawiono typ węzła na SL (Slack) oraz zdefiniowano napięcie zadane $V_m = 1.02$ p.u.
- Dla Obciążenia w analogicznej zakładce wprowadzono wartości mocy $P = 0.1$ MW i $Q = 0.05$ MVAr.
- Dla Linii zdefiniowano nowy typ, w którym podano jej parametry jednostkowe ($R = 0.3\,\Omega$/km, $X = 0.1\,\Omega$/km), a następnie przypisano do niej długość 0.5 km.

Proces ten eliminuje też konieczność analitycznego budowania macierzy admitancji Y_{bus}, gdyż jest ona automatycznie tworzona przez wewnętrzny silnik obliczeniowy programu.

Wykonanie obliczeń rozpływu mocy. Po zdefiniowaniu modelu, uruchomienie analizy sprowadza się do wywołania wbudowanej funkcji obliczeń rozpływu mocy. PowerFactory wykorzystuje zaawansowane, iteracyjne solwery (typowo metodę Newtona-Raphsona), aby rozwiązać nieliniowy układ równań sieci. Wyniki, takie jak moduły i kąty napięć w węzłach oraz przepływy mocy, są natychmiastowo dostępne i mogą być zwizualizowane bezpośrednio na schemacie jednokreskowym. Uzyskane w ten sposób wartości „prawdziwego stanu” są identyczne z tymi obliczonymi w `pandapower`.

W celu walidacji poprawności zamodelowania systemu, porównano uzyskane wyniki obliczeń rozpływu mocy z wartościami referencyjnymi, obliczonymi w poprzednim rozdziale. Zestawienie kluczowych parametrów w węźle odbiorczym przedstawiono w Tabeli 2.

Tabela 2: Porównanie wyników obliczeń rozpływu mocy dla węzła odbiorczego, uzyskanych w `pandapower` oraz DIgSILENT PowerFactory.

Źródło Obliczeń	**V** [p.u.]	θ [°]	**P** [MW]	**Q** [MVAr]
`pandapower`	0.898	0.977	0.100	0.050
DIgSILENT PowerFactory	0.900	0.977	0.100	0.050

Jak widać w powyższym zestawieniu, wyniki uzyskane w obu środowiskach są praktycznie tożsame, a drobne różnice wynikają prawdopodobnie z odmiennych ustawień

precyzji numerycznej i kryteriów zbieżności algorytmu. Wysoka zgodność wyników potwierdza, że model zaimplementowany w środowisku PowerFactory został zdefiniowany poprawnie i jest wiernym odwzorowaniem systemu analizowanego w podejściu programistycznym.

Należy również podkreślić, że środowisko DIgSILENT PowerFactory jest kompleksową, zintegrowaną platformą analityczną, której możliwości wykraczają daleko poza statyczne obliczenia rozpływu mocy. Jego modułowa budowa pozwala na prowadzenie pełnego spektrum symulacji i analiz, które są niezbędne w nowoczesnej inżynierii systemów elektroenergetycznych. Zintegrowane w programie, zaawansowane silniki obliczeniowe umożliwiają między innymi analizę zwarciową zgodną ze standardami IEC i ANSI, ocenę stabilności kątowej i napięciowej systemu w odpowiedzi na duże zaburzenia, a także symulacje quasi-dynamiczne i w dziedzinie czasu do badania zjawisk wolnozmiennych.

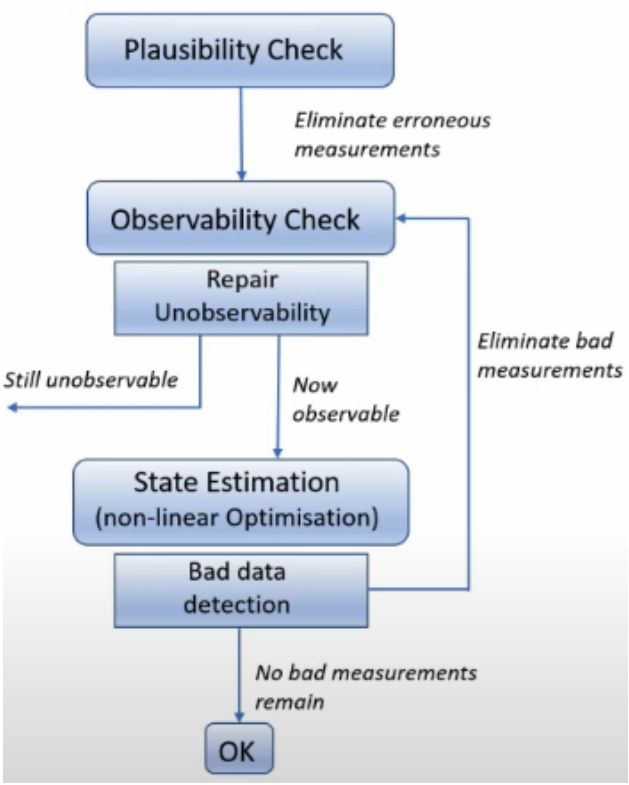
Co więcej, PowerFactory oferuje wyspecjalizowane moduły do analizy harmonicznych, koordynacji zabezpieczeń oraz optymalnego rozpływu mocy, co pozwala na wieloaspektową ocenę i optymalizację pracy sieci. Dostępne są również narzędzia do analizy niezawodnościowej oraz symulacji zjawisk elektromagnetycznych przejściowych, które są kluczowe w badaniach nad pracą systemów z dużą penetracją energoelektroniki. Ta wszechstronność czyni PowerFactory kompletnym środowiskiem, w którym można zamodelować i przeanalizować praktycznie każdy aspekt pracy i dynamiki współczesnego systemu elektroenergetycznego – od planowania jego rozwoju, przez analizę stanów awaryjnych, aż po projektowanie systemów sterowania i zabezpieczeń.

Poza funkcjami do deterministycznych analiz, najważniejszym modułem tego środowiska z punktu widzenia pracy jest ocena stanu sieci w warunkach niepewności pomiarowej, a więc estymacja stanu.

Moduł ten pozwala na zdefiniowanie zestawu pomiarów, a następnie uruchomienie procesu obliczeniowego w celu wyznaczenia najbardziej prawdopodobnego wektora stanu systemu. Dostępność tej funkcji jest niezwykle cenna, gdyż stanowić będzie wiarygodny punkt odniesienia (benchmark), umożliwiający bezpośrednie porównanie skuteczności opracowanego w tej pracy estymatora opartego na GNN z wynikami uzyskanymi za pomocą sprawdzonej, standardowej metody przemysłowej.

Należy przy tym zaznaczyć, że funkcja estymacji stanu w profesjonalnych narzędziach, takich jak DIgSILENT PowerFactory, nie sprowadza się jedynie do jednorazowego matematycznego rozwiązania problemu . Jest to zaawansowany, wieloetapowy algorytm przetwarzania danych, którego celem jest zapewnienie maksymalnej wiarygodności i dokładności wyniku. Schemat tego procesu zilustrowano na Rysunku 8.

Jak widać na diagramie, proces rozpoczyna się od *kontroli wiarygodności* (*Plausibility Check*), gdzie odrzucane są pomiary w sposób oczywisty błędne, np. o niemożliwych



Rysunek 8: Schemat blokowy zaawansowanego algorytmu estymacji stanu zaimplementowanego w środowisku DIgSILENT PowerFactory.

fizycznie wartościach. Następnie, kluczowym krokiem jest *analiza obserwowalności* (*Observability Check*), która weryfikuje, czy dostępny zestaw pomiarów jest wystarczający do jednoznacznego określenia stanu całej sieci. Dopiero po pomyślnym przejściu tych kroków uruchamiany jest właściwy algorytm *estymacji stanu*.

Jednak nawet po uzyskaniu rozwiązania, proces się nie kończy. Uruchamiana jest procedura *detekcji błędów grubych* (*Bad Data Detection*), która na podstawie analizy statystycznej residuów pomiarowych identyfikuje wartości odstające. Jeśli takie pomiary zostaną znalezione, są one eliminowane, a cały cykl – począwszy od analizy obserwowalności – jest powtarzany. Proces kończy się dopiero wtedy, gdy uzyskany wektor stanu jest oparty na spójnym i pozbawionym błędów grubych zestawie danych.

Ten kompletny, iteracyjny proces, łączący walidację danych, analizę topologiczną i optymalizację statystyczną, sprawia, że estymator w PowerFactory jest potężnym narzędziem referencyjnym. Wynik uzyskany z tego algorytmu nie jest zwykłym rozwiązaniem, lecz w pełni sprawdzonym i oczyszczonym z błędów obrazem stanu sieci. (*Stanowi to solidny i wiarygodny benchmark, z którym w dalszej części pracy porównane zostaną wyniki estymacji uzyskane za pomocą modelu opartego na GNN.*

Dodatkową, istotną zaletą pracy w środowisku PowerFactory jest wbudowany terminal (Output Window), który dostarcza szczegółowych informacji na temat przebiegu procesu

obliczeniowego. Jak zilustrowano na Rysunku 9, po uruchomieniu analizy log obliczeniowy w czasie rzeczywistym informuje o rozpoczęciu algorytmu, kolejnych iteracjach metody Newtona-Raphsona oraz ostatecznym statusie zbieżności.

```
Element 'External Grid' is local reference in separated area of '— Terminal'
Calculating load flow...
--------------------------------------------------------------------------------
Start Newton-Raphson Algorithm...
Load flow iteration: 0
Load flow iteration: 1
Load flow iteration: 2
Load flow iteration: 3
Newton-Raphson converged with 3 iterations.
Load flow calculation successful.
--------------------------------------------------------------------------------
      Report of Control Condition for Relevant Controllers
--------------------------------------------------------------------------------
Control conditions for all controllers of interest are fulfilled.
```

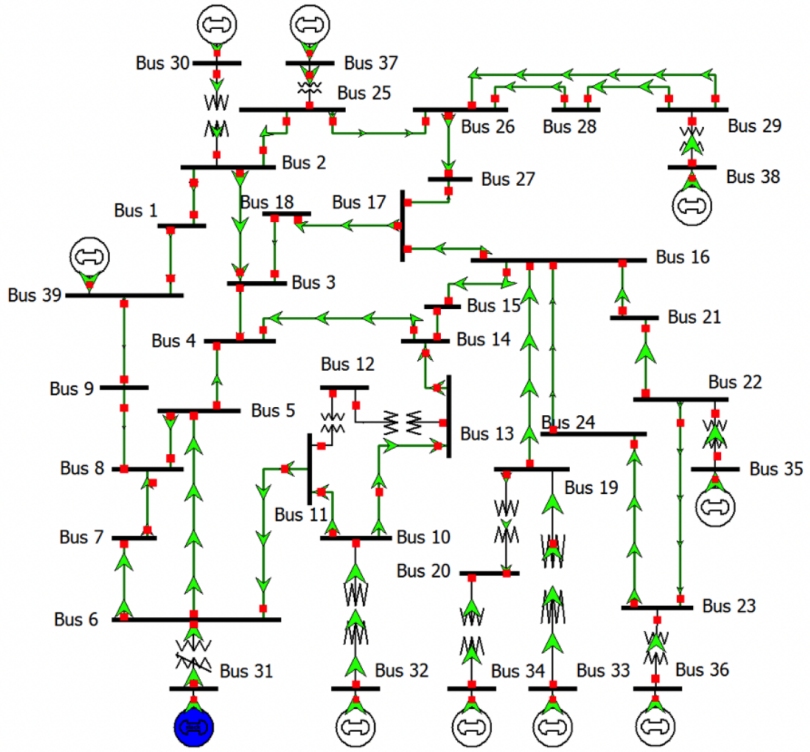
Rysunek 9: Log programu PowerFactory

Funkcjonalność ta ma kluczowe znaczenie diagnostyczne. W przypadku problemów z modelem, takich jak błędy w parametryzacji, niespójności topologiczne czy rozbieżność obliczeń, okno wyjściowe natychmiastowo zwraca precyzyjne komunikaty o błędach i ostrzeżenia, często wskazując element sieci będący źródłem problemu. Stanowi to znaczące usprawnienie w porównaniu do środowisk programistycznych, takich jak ‘pandapower‘, gdzie debugowanie problemów z rozbieżnością rozpływu mocy wymaga od użytkownika samodzielnej implementacji dodatkowego kodu diagnostycznego i analizy surowych wyników numerycznych. Wbudowany w PowerFactory mechanizm informacji zwrotnej znacząco przyspiesza zatem proces budowy i walidacji poprawnych modeli sieciowych.

9 Generowanie zbioru uczącego dla modelu GNN na przykładzie sieci IEEE 39-Bus

Po zademonstrowaniu zasad modelowania i analizy na uproszczonym przykładzie dwuwęzłowym, kolejnym krokiem jest przygotowanie zbioru danych, który posłuży do treningu docelowego estymatora stanu opartego na grafowej sieci neuronowej. Jak wskazano w podrozdziale 8.1, skuteczność modeli uczenia maszynowego wymaga danych, które odzwierciedlają szerokie spektrum możliwych stanów pracy systemu. W tym celu, jako model referencyjny, wybrano znacznie bardziej złożoną i realistyczną sieć testową IEEE 39-Bus New England System.

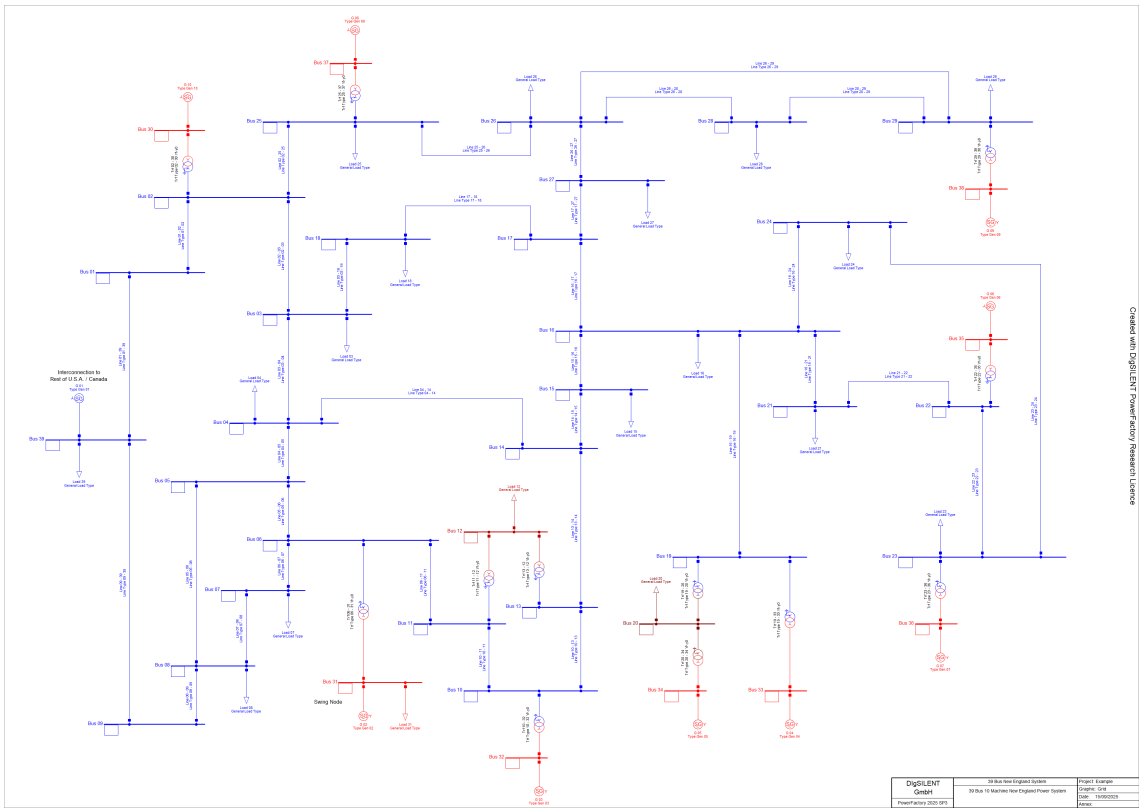
Wybór sieci 39-węzłowej jako podstawy do generowania danych był podyktowany kilkoma kluczowymi czynnikami. W przeciwieństwie do modelu dwuwęzłowego, który służył jedynie celom ilustracyjnym, system New England stanowi wierne odzwierciedlenie regionalnego systemu przesyłowego. Składa się on z 39 szyn, 10 jednostek wytwórczych, 19 obciążeń oraz 46 gałęzi (34 linii i 12 transformatorów). Jego złożona, pętlowa topologia wprowadza nieliniowe zależności i interakcje między odległymi węzłami, które są niemożliwe do zaobserwowania w prostym układzie promieniowym. Ta złożoność stanowi adekwatne wyzwanie dla grafowej sieci neuronowej, której zadaniem jest właśnie modelowanie takich skomplikowanych relacji.



Rysunek 10: Model sieci 39 Bus New England zaprezentowany w [33]

System ten, ze względu na swoje dobrze udokumentowane parametry i znane właściwości dynamiczne, od lat służy jako standardowy model odniesienia do walidacji i porównywania nowych algorytmów. Pierwotnie został opracowany do badań nad stabilnością dynamiczną, w szczególności do analizy zjawisk przejściowych i małosygnałowych [33]. W literaturze naukowej jest on powszechnie wykorzystywany do testowania zaawansowanych systemów sterowania, takich jak stabilizatory systemowe (PSS), weryfikacji algorytmów optymalnego rozpływu mocy (OPF), a w ostatnich latach stał się również popularną platformą do demonstracji i oceny metod opartych na uczeniu maszynowym, w tym właśnie do zadań estymacji stanu. Jego ugruntowana pozycja sprawia, że stanowi on solidny i wiarygodny punkt odniesienia, pozwalający na obiektywną ocenę nowo proponowanych rozwiązań na tle istniejącego stanu wiedzy. Schemat jednokreskowy modelu

zaimplementowanego w środowisku PowerFactory przedstawiono na Rysunku 11.



Rysunek 11: Model sieci 39 BUS New England w środowisku PowerFactory.

9.1 Metodyka generowania danych.

Proces generowania zbioru uczącego został w pełni zautomatyzowany za pomocą zewnętrznego skryptu w języku Python, który sterował symulacjami w środowisku `PowerFactory`. Głównym celem było stworzenie dużej liczby unikalnych, ale fizycznie spójnych stanów pracy systemu. Każdy taki stan, określany jako „test case”, stanowi jeden przykład (próbkę) w zbiorze uczącym. Procedura generowania danych przebiegała według następujących kroków:

1. **Inicjalizacja i wczytanie modelu:** Skrypt rozpoczyna pracę od nawiązania połączenia z aktywną sesją `PowerFactory` i wczytania bazowego stanu pracy sieci 39-Bus.

2. **Pętla symulacyjna:** Uruchamiana jest pętla, w której generowana jest zadana liczba (np. 1000) unikalnych stanów pracy systemu.

3. **Randomizacja obciążeń:** W każdej iteracji pętli, wartość mocy czynnej każdego z 19 obciążeń w sieci jest losowo modyfikowana. Nowa wartość jest losowana

z rozkładu jednostajnego w zadanym przedziale, co symuluje naturalne wahania zapotrzebowania na moc.

4. **Obliczenia rozpływu mocy:** Dla nowo wylosowanego stanu obciążeń, skrypt uruchamia pełne obliczenia rozpływu mocy metodą Newtona-Raphsona. Stany, dla których rozpływ jest niezbieżny, są odrzucane.

5. **Akwizycja danych:** Po pomyślnym zakończeniu obliczeń, skrypt zbiera kompletny wektor cech opisujących zarówno węzły (szyny), jak i gałęzie (linie i transformatory).

   - **Cechy węzłów:** Dla każdej z 39 szyn zbierane są cztery kluczowe wielkości: moc czynna netto (P_{net}), moc bierna netto (Q_{net})[6], moduł napięcia (V) oraz kąt fazowy napięcia (θ).
   - **Cechy gałęzi:** Dla każdej z 46 gałęzi (linii i transformatorów) zbierane są trzy wielkości wynikowe: przepływ mocy czynnej (P_{flow}), przepływ mocy biernej (Q_{flow}) oraz obciążenie procentowe (*loading*).

6. **Eksport danych:** Zebrane dane ze wszystkich szyn i gałęzi są agregowane w jeden wiersz i zapisywane do zewnętrznego pliku w formacie CSV.

Po zakończeniu pętli, oryginalne, znamionowe wartości obciążeń są przywracane w modelu PowerFactory. Wynikowy plik CSV stanowi kompletny, gotowy do dalszego przetwarzania zbiór danych.

9.2 Implementacja skryptu automatyzującego

Opisana metodyka została zaimplementowana w postaci jednego skryptu w języku Python. Poniżej przedstawiono i omówiono kluczowe fragmenty kodu, które realizują poszczególne etapy procesu.

Połączenie ze środowiskiem PowerFactory i inicjalizacja. Pierwszym krokiem jest nawiązanie połączenia z aplikacją `PowerFactory`. Następnie, skrypt dynamicznie identyfikuje wszystkie kluczowe grupy obiektów w projekcie: szyny, linie, transformatory oraz obciążenia. Elementy są sortowane alfabetycznie w celu zapewnienia stałej i przewidywalnej kolejności kolumn w pliku wynikowym.

[6] zarówno moc P_{net} jak i Q_{net} obliczane są jako różnica odpowiadających im mocy wpływających i wypływających z danego węzła

Inicjalizacja i przygotowanie skryptu

```python
import sys, powerfactory, random, os, csv

# ... (definicje ustawień globalnych) ...

# Nawiązanie połączenia z aplikacją PowerFactory
try:
    app = powerfactory.GetApplicationExt()
    if app is None:
        raise Exception("Nie udało się połączyć z PowerFactory.")
except Exception as e:
    sys.exit(f"BŁĄD POŁĄCZENIA: {e}")

# Pobranie obiektów w celu ustalenia stałej kolejności i przywrócenia wartości
initial_loads = app.GetCalcRelevantObjects('*.ElmLod')
ldf = app.GetFromStudyCase("ComLdf")
all_buses = sorted(app.GetCalcRelevantObjects('*.ElmTerm'), key=lambda x: x.loc_name)
all_lines = sorted(app.GetCalcRelevantObjects('*.ElmLne'), key=lambda x: x.loc_name)
all_trafos = sorted(app.GetCalcRelevantObjects('*.ElmTr2'), key=lambda x: x.loc_name)
```

Pętla symulacyjna i akwizycja danych. Rdzeń skryptu stanowi pętla symulacyjna. W każdej iteracji losowo modyfikowane są wartości obciążeń, a następnie uruchamiany jest rozpływ mocy. Kluczowym elementem, zapewniającym poprawność odczytu wyników, jest ponowne pobieranie referencji do wszystkich obiektów (`GetCalcRelevantObjects`) po każdej udanej symulacji. Gwarantuje to, że odczytywane atrybuty pochodzą z obiektów, które zostały zaktualizowane przez silnik obliczeniowy. Skrypt rozróżnia obiekty linii i transformatorów, stosując dla nich odpowiednie nazwy atrybutów w celu odczytania przepływów mocy.

Główna pętla symulacyjna skryptu

```python
for i in range(LICZBA_SYMULACJI):
    # ... (randomizacja obciążeń) ...

    # Wykonaj rozpływ mocy; pomiń krok w przypadku braku zbieżności
    if ldf.Execute() != 0:
        continue

    # Przygotuj wiersz z danymi i pobierz obiekty z wynikami
    row_data = [i]
    buses_with_results = {b.loc_name: b for b in app.GetCalcRelevantObjects('*.ElmTerm')}
    lines_with_results = {l.loc_name: l for l in app.GetCalcRelevantObjects('*.ElmLne')}
    trafos_with_results = {t.loc_name: t for t in app.GetCalcRelevantObjects('*.ElmTr2')}
```

```
    # Zbieranie danych o węzłach (szynach)
    for bus in all_buses:
        bus_res = buses_with_results.get(bus.loc_name)
                    p_net = bus_res.GetAttribute('m:Pgen') -
                    ↪  bus_res.GetAttribute('m:Pload')
        # ... (odczyt, Qnet, V, Angle) ...

    # Zbieranie danych o liniach
    for line in all_lines:
        line_res = lines_with_results.get(line.loc_name)
        p_flow = line_res.GetAttribute('m:P:bus1')
        # ... (odczyt Qflow, Loading) ...

    # Zbieranie danych o transformatorach
    for trafo in all_trafos:
        trafo_res = trafos_with_results.get(trafo.loc_name)
        p_flow = trafo_res.GetAttribute('m:P:bushv')
        # ... (odczyt Qflow, Loading) ...

    writer.writerow(row_data)

# ... (przywrócenie oryginalnych wartości obciążeń) ...
```

W wyniku działania skryptu, każda z 1000 symulacji generuje kompletny wektor cech (pomiarów), opisujących stan pracy systemu. Na wektor ten składa się:

$$\underbrace{(39 \times 4)}_{\text{cechy węzłów}} + \underbrace{(46 \times 3)}_{\text{cechy krawędzi}} = 294 \text{ pomiary.} \tag{27}$$

Tak ustrukturyzowany zbiór danych stanowi kompletną i spójną bazę, gotową do wykorzystania w procesie trenowania grafowej sieci neuronowej.

10 Implementacja grafowej sieci neuronowej

W niniejszym rozdziale przedstawiono praktyczną implementację modelu grafowej sieci neuronowej, która stanowi centralny punkt badawczy niniejszej pracy. Po omówieniu teoretycznych podstaw GNN i metodyki generowania danych, celem jest połączenie tych dwóch elementów i zademonstrowanie, jak dane z symulacji PowerFactory są przekształcane w format grafowy, a następnie wykorzystywane do treningu i walidacji modelu.

10.1 Przygotowanie środowiska i bibliotek

Przed przystąpieniem do implementacji modelu GNN, kluczowe było odpowiednie przygotowanie środowiska programistycznego oraz weryfikacja kompatybilności niezbędnych bibliotek. Wybór narzędzi podyktowany był ich wydajnością, szerokim wsparciem społeczności naukowej oraz specjalizacją w przetwarzaniu danych grafowych, co stanowi fundament niniejszego podejścia badawczego.

Implementacja modelu GNN została oparta na ekosystemie *PyTorch*, który jest wiodącym frameworkiem[7] do głębokiego uczenia. Jego kluczową zaletą jest elastyczność w budowaniu niestandardowych architektur sieci neuronowych, co umożliwiło autorską implementację warstwy Message Passing. Ponadto, PyTorch zapewnia efektywne wykorzystanie akceleracji sprzętowej, w szczególności GPU, co jest niezbędne do prowadzenia symulacji na dużą skalę.

W kontekście analizy danych grafowych, fundamentem pracy jest biblioteka *PyTorch Geometric (PyG)*, która stanowi rozszerzenie PyTorch'a, dostarczając zoptymalizowanych implementacji warstw GNN, narzędzi do przetwarzania danych o strukturze grafowej oraz modułów do tworzenia grafowych zbiorów danych. Użycie tej biblioteki pozwoliło na sprawne przejście od surowych danych tabelarycznych do formatu grafowego, co było warunkiem koniecznym do treningu modelu.

Do wstępnej obróbki i normalizacji danych, a także do oceny skuteczności modelu, wykorzystano standardowe biblioteki naukowe w ekosystemie Pythona: *Pandas* do manipulacji danymi, *NumPy* do obliczeń numerycznych, oraz *Scikit-learn* do podziału danych i obliczania metryk jakości. Wizualizacje wyników, niezbędne do interpretacji rezultatów, zostały zrealizowane przy użyciu biblioteki *Matplotlib*.

[7] Gotowa, ustrukturyzowana platforma, która zapewnia podstawowe komponenty i biblioteki do tworzenia aplikacji, ułatwiając i standaryzując proces programowania, w kontekście uczenia maszynowego, framework'i takie jak PyTorch udostępniają zbiór predefiniowanych klas i funkcji do efektywnej implementacji i treningu sieci neuronowych.

10.2 Przetwarzanie i normalizacja danych

Surowe dane, wygenerowane w środowisku PowerFactory i zapisane w formacie CSV, wymagają specyficznego przetworzenia, aby mogły zostać wykorzystane przez model GNN. Kluczowym krokiem jest konwersja danych tabelarycznych w ustrukturyzowany format grafowy, na którym może operować biblioteka `PyTorch Geometric`.

10.2.1 Formułowanie danych do formatu grafowego

Dane wejściowe, takie jak pomiary mocy czynnej i biernej w węzłach oraz parametry linii, muszą zostać przekształcone w obiekt danych grafowych, który będzie zawierał:

- **Cechy węzłów (node features)**: Wektory atrybutów dla każdego węzła (P_{net} i Q_{net}).
- **Cechy krawędzi (edge features)**: Wektory atrybutów dla każdej linii (P_{flow}, Q_{flow}, i obciążenie lini).
- **Topologia grafu (edge index)**: Macierz połączeń opisująca, które węzły są ze sobą połączone.

Jak wspomniano w rozdziale 9, surowe dane z symulacji zostały zapisane w pliku CSV. Poniższy fragment kodu demonstruje proces ich wczytania, podziału oraz transformacji na obiekty `Data` z biblioteki `PyTorch Geometric`.

```python
# Klasa Dataset do tworzenia obiektów grafowych
class PowerSystemDataset(Dataset):
    def __init__(self, node_features, edge_features, targets, edge_index):
        self.node_features = torch.tensor(node_features, dtype=torch.float32)
        self.edge_features = torch.tensor(edge_features, dtype=torch.float32)
        self.targets = torch.tensor(targets, dtype=torch.float32)
        self.edge_index = edge_index

    def __len__(self):
        return len(self.node_features)

    def __getitem__(self, idx):
        edge_features_single_direction = self.edge_features[idx]
        edge_attr_bidirectional = torch.cat([
            edge_features_single_direction,
            edge_features_single_direction
        ], dim=0)

        data = Data(x=self.node_features[idx],
                    edge_index=self.edge_index,
                    edge_attr=edge_attr_bidirectional,
                    y=self.targets[idx])
        return data
```

Powyższy kod definiuje klasę `PowerSystemDataset`, która rozszerza standardową klasę `Dataset` z biblioteki `PyTorch`. Jej głównym zadaniem jest opakowanie przetworzonych danych numerycznych w format grafowy. W metodzie `__getitem__`, dla każdego indeksu tworzony jest obiekt `Data`, który jest podstawowym elementem do pracy z grafami w `PyTorch Geometric`. Obiekt ten łączy cechy węzłów `x`, cechy krawędzi `edge_attr` oraz macierz połączeń `edge_index` w jedną, spójną strukturę danych.

Przed treningiem, dane są poddawane normalizacji za pomocą skalera `StandardScaler` z biblioteki `scikit-learn`. Proces ten ma bezpośredni wpływ na stabilność i efektywność algorytmu optymalizacyjnego. Wartości atrybutów w zbiorze danych, takie jak moce (rzędu MW/MVAr) i kąty fazowe (rzędu stopni), mają bardzo różne skale i jednostki. Bez odpowiedniego przetworzenia, te różnice prowadzą do problemów, które mogą znacząco utrudnić proces uczenia. Dlatego normalizacja sprowadza wartości zmiennych do rozkładu ze średnią zero i odchyleniem standardowym równym jeden.

Normalizacja danych

```python
# Inicjalizacja i uczenie skalerów na danych treningowych
x_node_scaler = StandardScaler().fit(X_nodes_train.reshape(-1,
↪  X_nodes_train.shape[-1]))
#...(odpowiednio dla X_edges oraz Y_nodes)...

# Funkcja pomocnicza do skalowania
def scale_data(data, scaler):
    shape = data.shape
    return scaler.transform(data.reshape(-1, shape[-1])).reshape(shape)

# Skalowanie danych treningowych i testowych
X_nodes_train_scaled = scale_data(X_nodes_train, x_node_scaler)
# ... (pozostałe skalowanie) ...
```

10.2.2 Podział zbioru danych i skalowanie

W celu rzetelnej oceny zdolności modelu do generalizacji na nieznanych danych, kluczowe jest prawidłowe i bezstronne przygotowanie zbioru danych. Metodologia ta zakłada strategiczny podział na trzy niezależne podzbiory:

- **Zbiór uczący (ang. *training set*)**: Stanowi trzon danych, na których model optymalizuje swoje parametry wewnętrzne (wagi i obciążenia). Uczy się on na nich rozpoznawania zależności między cechami wejściowymi (pomiarami) a pożądanym stanem wyjściowym (napięciami i kątami).

- **Zbiór walidacyjny (ang. *validation set*)**: Wykorzystywany do monitorowania postępów treningu i strojenia hiperparametrów. Model nie uczy się bezpośrednio na tych danych.

- **Zbiór testowy (ang. *test set*)**: Jest to zbiór, na którym model jest testowany jednorazowo, po zakończeniu całego procesu uczenia. Służy do ostatecznej, bezstronnej oceny wydajności i stanowi podstawę do obliczenia finalnych metryk jakościowych, takich jak MAE czy RMSE.

Poniższy fragment kodu, zaimplementowany w języku Python, ilustruje proces podziału zbioru danych wejściowych (`X_nodes`, `X_edges`) i wyjściowych (`Y_nodes`) na trzy niezależne podzbiory za pomocą funkcji `train_test_split` z biblioteki `scikit-learn`.

Podział danych

```
# Podział całego datasetu na zbiór tymczasowy oraz zbiór testowy (1% danych)
    train_test_split(X_nodes, X_edges, Y_nodes, test_size=0.2, random_state=42)

# Podział zbioru tymczasowego na treningowy (75%) i walidacyjny (25%)
# 0.25 z 0.8 daje 0.2 (20% pierwotnego zbioru)
    train_test_split(X_nodes_temp, X_edges_temp, Y_nodes_temp, test_size=0.25,
    ↪  random_state=42)
```

Poprawne, dwuetapowe zastosowanie funkcji `train_test_split` zapewnia, że zbiór testowy pozostaje całkowicie niewidoczny dla modelu podczas uczenia i walidacji. Taka rygorystyczna metodyka jest fundamentalna dla wiarygodności przeprowadzonych eksperymentów.

10.3 Definicja architektury modelu GNN

Bazując na teorii mechanizmu Przekazywania Wiadomości, zaimplementowano własną architekturę GNN, dostosowaną do specyfiki problemu estymacji stanu. Model uwzględnia zarówno cechy węzłów, jak i krawędzi, co pozwala mu na jednoczesne przetwarzanie pomiarów z szyn i linii.

W sercu modelu znajduje się warstwa, która rozszerza klasę `MessagePassing` z biblioteki `PyTorch Geometric`. Zgodnie z teorią przedstawioną w rozdziale 7, warstwa ta agreguje informacje od sąsiadujących węzłów i krawędzi, a następnie przetwarza je za pomocą wielowarstwowego perceptronu (MLP), aby wygenerować nową, zaktualizowaną reprezentację każdego węzła. Poniższy fragment kodu przedstawia implementację tej warstwy.

```
class GNNLayer(MessagePassing):
    def __init__(self, node_channels, edge_channels, output_channels):
        super().__init__(aggr='mean')
        self.mlp = nn.Sequential(
            nn.Linear(node_channels * 2 + edge_channels, output_channels),
            nn.ReLU(),
```

```
            nn.Linear(output_channels, output_channels)
        )

    def forward(self, x, edge_index, edge_attr):
        return self.propagate(edge_index, x=x, edge_attr=edge_attr)

    def message(self, x_i, x_j, edge_attr):
        tmp = torch.cat([x_i, x_j, edge_attr], dim=1)
        return self.mlp(tmp)
```

Funkcja `forward` inicjuje proces przekazywania wiadomości (`propagate`), a następnie wywołuje funkcję `message`, która jest kluczowa dla procesu uczenia. W funkcji `message` cechy węzła docelowego (`x_i`), cechy węzła źródłowego (`x_j`) oraz cechy krawędzi (`edge_attr`) są łączone i podawane na wejście MLP. Ta struktura pozwala modelowi na uwzględnienie informacji o sąsiednich węzłach i łączących je elementach.

10.3.1 Pełna architektura estymatora stanu

Kompletny estymator stanu, nazwany `GNNStateEstimator`, składa się z dwóch kaskadowo połączonych warstw GNN, połączonych warstwami normalizacji wsadowej (`BatchNorm`). Warstwy te stabilizują proces treningu, normalizując aktywacje neuronów. Architektura kończy się warstwą liniową, która przekształca wyjście z ostatniej warstwy GNN w estymowany wektor stanu (`V`, `kąt fazowy`).

```
class GNNStateEstimator(torch.nn.Module):
    def __init__(self, num_node_features, num_edge_features,
                 num_output_features, hidden_channels):
        super().__init__()
        self.conv1 = GNNLayer(num_node_features, num_edge_features,
        ↪ hidden_channels)
        self.bn1 = BatchNorm(hidden_channels)
        self.conv2 = GNNLayer(hidden_channels, num_edge_features,
        ↪ hidden_channels)
        self.bn2 = BatchNorm(hidden_channels)
        self.output_layer = nn.Linear(hidden_channels, num_output_features)

    def forward(self, x, edge_index, edge_attr):
        x = self.conv1(x, edge_index, edge_attr)
        x = self.bn1(x)
        x = x.relu()
        x = self.conv2(x, edge_index, edge_attr)
        x = self.bn2(x)
        x = x.relu()
        out = self.output_layer(x)
        return out
```

Taka architektura pozwala na iteracyjne agregowanie informacji z lokalnego sąsiedztwa, a po dwóch warstwach każdy węzeł ma dostęp do informacji o sąsiadach swoich sąsiadów, co pozwala na uchwycenie zależności na szerszą skalę.

10.4 Trening i walidacja modelu

Proces uczenia modelu został przeprowadzony w oparciu o wygenerowany zbiór danych. Do ilościowej oceny jakości modelu w zadaniu wybrano funkcję kosztu błędu średniokwadratowego (MSE). Natomiast jako optymalizator parametrów modelu wybrano algorytm ADAM. Monitorowanie postępów odbywało się z wykorzystaniem niezależnego zbioru walidacyjnego, co umożliwiło detekcję przeuczenia i wybór optymalnego momentu na zakończenie treningu.

Pętla treningowa

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = torch.nn.MSELoss()

def train():
    model.train()
    total_loss = 0
    for data in train_loader:
        data = data.to(device)
        optimizer.zero_grad()
        out = model(data.x, data.edge_index, data.edge_attr)
        loss = criterion(out, data.y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * data.num_graphs
    return total_loss / len(train_loader.dataset)

def evaluate(loader):
    model.eval()
    total_loss = 0
    all_preds_scaled, all_labels_scaled = [], []
    with torch.no_grad():
        for data in loader:
            data = data.to(device)
            out = model(data.x, data.edge_index, data.edge_attr)
            loss = criterion(out, data.y)
            total_loss += loss.item() * data.num_graphs
            all_preds_scaled.append(out.cpu().numpy())
            all_labels_scaled.append(data.y.cpu().numpy())

    avg_loss = total_loss / len(loader.dataset)
    # ... dalsze obliczenia metryk (MAE, RMSE, R2) ...
    return avg_loss, ...
```

Powyższe funkcje są wywoływane w głównej pętli, która monitoruje postępy. W

każdej epoce obliczana jest **strata treningowa** (`train loss`) oraz **strata walidacyjna** (`val loss`). Porównanie tych dwóch wartości pozwala na ocenę, czy model uczy się w sposób efektywny i generalizuje, czy też zaczyna się przeuczać.

10.4.1 Wczesne zatrzymanie (Early Stopping)

Zjawisko przeuczenia (ang. overfitting) ma miejsce, gdy model zamiast uczyć się ogólnych wzorców, zaczyna „zapamiętywać" specyficzne, losowe szumy obecne w zbiorze treningowym. Skutkuje to spadkiem jego wydajności na danych, których wcześniej nie widział (zbiorze walidacyjnym i testowym).

W niniejszej pracy zastosowano modyfikację tej techniki. Zamiast automatycznie przerywać proces treningu, zdecydowano się na przeprowadzenie go przez pełną liczbę zadeklarowanych epok. Ten celowy zabieg miał na celu zebranie kompletnych danych na temat trajektorii uczenia i wizualne udokumentowanie potencjalnego przeuczenia, które objawia się rozbieżnością między stratą treningową a walidacyjną.

Kluczowym elementem tej metodyki było jednak równoległe monitorowanie wydajności modelu na zbiorze walidacyjnym. W każdej epoce, w której model osiągał nową, najniższą stratę walidacyjną, jego parametry były zapisywane do pliku. Dzięki temu po zakończeniu pełnego cyklu treningowego możliwe było odtworzenie i użycie „najlepszego" modelu - tego, który wykazał największą zdolność do generalizacji. Taka strategia pozwala na:

Ta rygorystyczna metodologia gwarantuje, że ostateczne metryki jakości są oparte na najbardziej optymalnej wersji modelu, a jednocześnie pozwala na głębszą analizę jego zachowania w trakcie uczenia.

11 Ocena jakości zaimplementowanej seci neuronowej

Zaimplementowany model GNN poddano gruntownej ocenie, analizując jego wydajność na podstawie kluczowych metryk błędów. Analiza ta miała na celu weryfikację zdolności modelu do precyzyjnego uogólniania zależności dla niewidzianych wcześniej danych. Końcowe metryki, wyznaczone na zbiorze testowym, świadczą o wyjątkowo wysokiej jakości estymacji. Średni błąd bezwzględny (MAE) wyniósł 1.064, co wskazuje na minimalne odchylenia przewidywań od rzeczywistych wartości. Jeszcze bardziej precyzyjną miarą, pierwiastkiem błędu średniokwadratowego (RMSE), uzyskano wartość 1.612. Fakt, że RMSE jest tylko nieznacznie wyższe od MAE, sugeruje, iż model nie generuje dużych, pojedynczych błędów, które mogłyby mieć krytyczne znaczenie dla stabilności systemu. Fundamentalnym wskaźnikiem potwierdzającym jakość modelu jest współczynnik determinacji (R^2), który osiągnął imponującą wartość 0.976. Oznacza to, że model wyjaśnia niemal 98% wariancji w danych wyjściowych, co dowodzi jego zdolności do efektywnego i dokładnego odwzorowywania zależności w systemie elektroenergetycznym.

Analiza przebiegu procesu treningowego i walidacyjnego dostarcza istotnych informacji na temat zdolności modelu do uogólniania. Początkowo, zarówno strata treningowa (MSE), jak i walidacyjna, systematycznie maleją, co wskazuje, że model skutecznie uczy się na dostarczonych danych i poprawia swoje uogólnianie. W miarę postępu treningu, po pewnym punkcie, choć strata treningowa kontynuuje spadek (osiągając 0.019241 w ostatniej epoce), strata walidacyjna zaczyna wykazywać wahania i nie notuje już znaczącej poprawy (ostatecznie wynosząc 0.029). Taka rozbieżność jest klasycznym sygnałem przeuczenia. Oznacza to, że model zaczął "zapamiętywać"zbiór treningowy, co prowadzi do pogorszenia jego wydajności na danych, których wcześniej nie widział.

W celu przeciwdziałania temu zjawisku, najlepszy model został zapisany w epoce, w której strata walidacyjna osiągnęła swoje minimum. Z danych wynika, że miało to miejsce w epoce 645, gdzie najlepszy model charakteryzował się stratą walidacyjną na poziomie 0.026. Ta wartość jest niższa od straty w ostatniej epoce, co podkreśla znaczenie metody wczesnego zatrzymania treningu. Wybór modelu z epoki 645 gwarantuje, że używamy najbardziej uogólniającej wersji sieci, która najlepiej radzi sobie z nowymi danymi.

11.1 Analiza wydajności obliczeniowej

Analiza wydajności obliczeniowej ma kluczowe znaczenie w kontekście praktycznej użyteczności. Cały proces badawczy, od generacji danych w PowerFactory po ewaluację modelu, był prowadzony na niskiej jakości, przestarzałym sprzęcie konsumenckim. Pomimo tych ograniczeń, model wykazał imponującą efektywność w fazie wnioskowania. Średni czas treningu na próbkę wynosił 0.281 s, co przy tysiącu próbek dało czas 280

s. Natomiast czas estymacji dla 50 próbek testowych wyniósł 0.08 s, co przekłada się na średnio 1.5932 milisekundy na pojedynczą estymację. Taka prędkość działania spełnia rygorystyczne wymogi systemów monitorowania sieci w czasie rzeczywistym, znacznie przewyższając wydajność tradycyjnych, iteracyjnych algorytmów estymacji stanu.

Badanie to należy traktować jako studium wykonalności (ang. proof of concept), którego głównym celem było udowodnienie potencjału sieci GNN w danych warunkach. Architektura sieci oraz zbiór danych były zoptymalizowane pod kątem uzyskania satysfakcjonujących rezultatów na dostępnym sprzęcie. W warunkach produkcyjnych, gdzie priorytetem jest najwyższa niezawodność, możliwe jest znaczne rozbudowanie modelu oraz użycie większych i bardziej zróżnicowanych zbiorów danych. Należy przy tym podkreślić, że koszt treningu jest kosztem jednorazowym. Po wytrenowaniu model mógłby być wielokrotnie wykorzystywany do estymacji w czasie rzeczywistym, generując wyniki w ułamku sekundy, co czyni go wysoce efektywnym narzędziem, niezależnie od skali wdrożenia.

W celu szczegółowej oceny kosztów obliczeniowych fazy treningu, przeprowadzono eksperyment. Porównano czas uczenia modelu na tym samym zbiorze danych, z identycznymi hiperparametrami oraz ustalonym ziarnem losowości, na dwóch różnych jednostkach obliczeniowych dostępnych w ramach komputera Apple Mac M1: jednostce centralnej (CPU) oraz zintegrowanej jednostce graficznej (GPU). Wyniki tego porównania przedstawiono w tabeli 3.

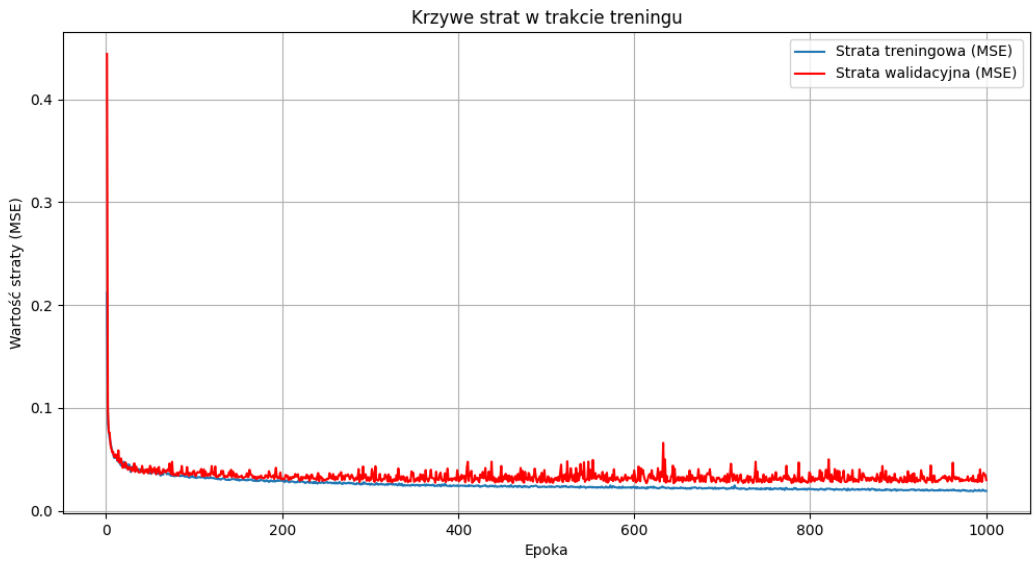
Tabela 3: Porównanie czasu treningu modelu na CPU i GPU (1000 epok).

Urządzenie	**Całkowity czas [s]**	**Średni czas / epoka [s]**	**Przyspieszenie**
CPU	192,30	0,1923	1.00x (referencja)
GPU	202,45	0,2025	**0.95x**

Uzyskane wyniki, wbrew ogólnym oczekiwaniom, wskazują na brak przyspieszenia przy wykorzystaniu akceleracji GPU – proces ten był nieznacznie dłuższy. To nieintuicyjne zjawisko ma swoje źródło w specyfice zastosowanej architektury sprzętowej i programowej. Po pierwsze, rdzenie CPU w procesorach Apple Silicon charakteryzują się bardzo wysoką wydajnością i optymalizacją, stanowiąc silną bazę porównawczą. Po drugie, przy stosunkowo niewielkim modelu oraz paczkach danych (*batch size*), narzut czasowy (ang. *overhead*) związany z przygotowaniem i wysłaniem zadań na GPU może zniwelować zyski z ich równoległego przetwarzania. Kluczowym czynnikiem jest jednak dojrzałość wsparcia programowego – biblioteka PyTorch do akceleracji na sprzęcie Apple wykorzystuje backend Metal Performance Shaders (MPS), który jest znacznie nowszy niż wieloletni standard NVIDIA CUDA. Prawdopodobne jest, że specyficzne dla GNN

operacje, takie jak agregacja wiadomości od nieregularnej liczby sąsiadów, nie są jeszcze w pełni zoptymalizowane dla MPS, co w tym konkretnym przypadku dało przewagę wyspecjalizowanym bibliotekom działającym na CPU.

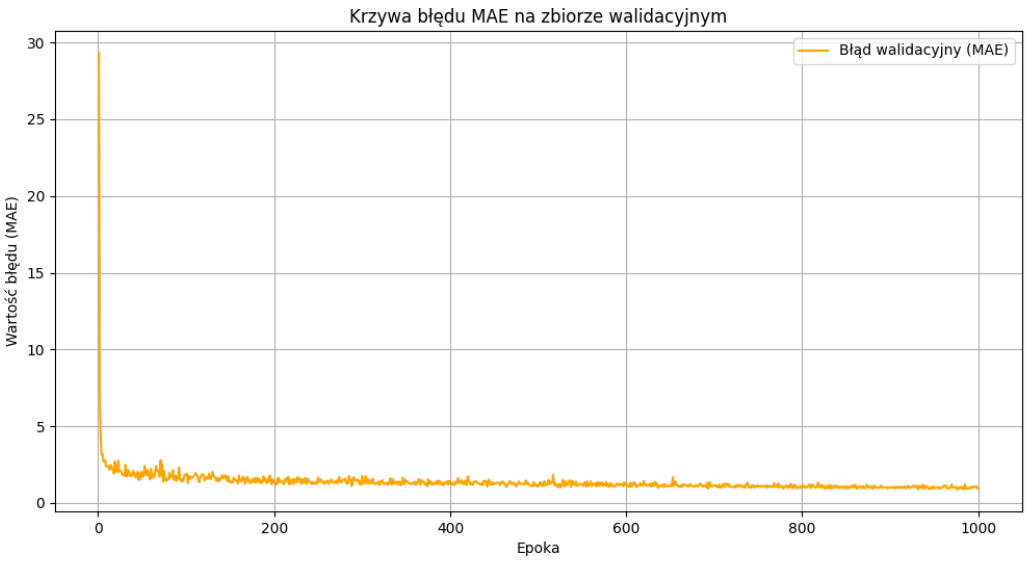
Oprócz wartości numerycznych, istotną wartością dodaną w kontekście analizy wyników jakościowych jest graficzne przedstawienie interesujących nas danych. Pozwala to na poznanie kluczowych informacji na temat charakteru błędów generowanych przez model. Na Rysunku 12 zaprezentowano krzywe strat w trakcie treningu.



Rysunek 12: Zmiana wartości funkcji straty przy kolejnych epokach

Wykres "Krzywe strat w trakcie treningu"przedstawia przebieg wartości straty MSE dla zbioru treningowego i walidacyjnego w funkcji liczby epok. Początkowo, obie krzywe gwałtownie spadają, co świadczy o efektywnym procesie uczenia się modelu. Po około 200 epokach, spadek straty walidacyjnej zaczyna spowalniać, a po osiągnięciu minimum w okolicach 645 epoki, jej wartość zaczyna fluktuować, a nawet nieznacznie rosnąć. Jednocześnie, strata treningowa nadal maleje, co jest klasycznym objawem przeuczenia. Zjawisko to wskazuje, że model zaczął zapamiętywać dane treningowe zamiast uogólniać wzorce, co prowadzi do pogorszenia jego zdolności do pracy. Należy jednak zaznaczyć, że było to działanie celowe mające prowadzić do uwypuklenia tego zjawiska. W rzeczywistości, tak jak wspomniano w punkcie 10.4.1 w trakcie całego procesu iteracyjnego wybrany zostaje najlepszy model sieci z epoki, w której strata walidacyjna była najmniejsza.

Wykres "Krzywa błędu MAE na zbiorze walidacyjnym"uzupełnia analizę procesu uczenia, prezentując przebieg średniego błędu bezwzględnego dla zbioru walidacyjnego. Podobnie jak w przypadku straty MSE, wartość błędu MAE gwałtownie maleje w początkowych epokach, co potwierdza szybką konwergencję modelu. Po osiągnięciu przez



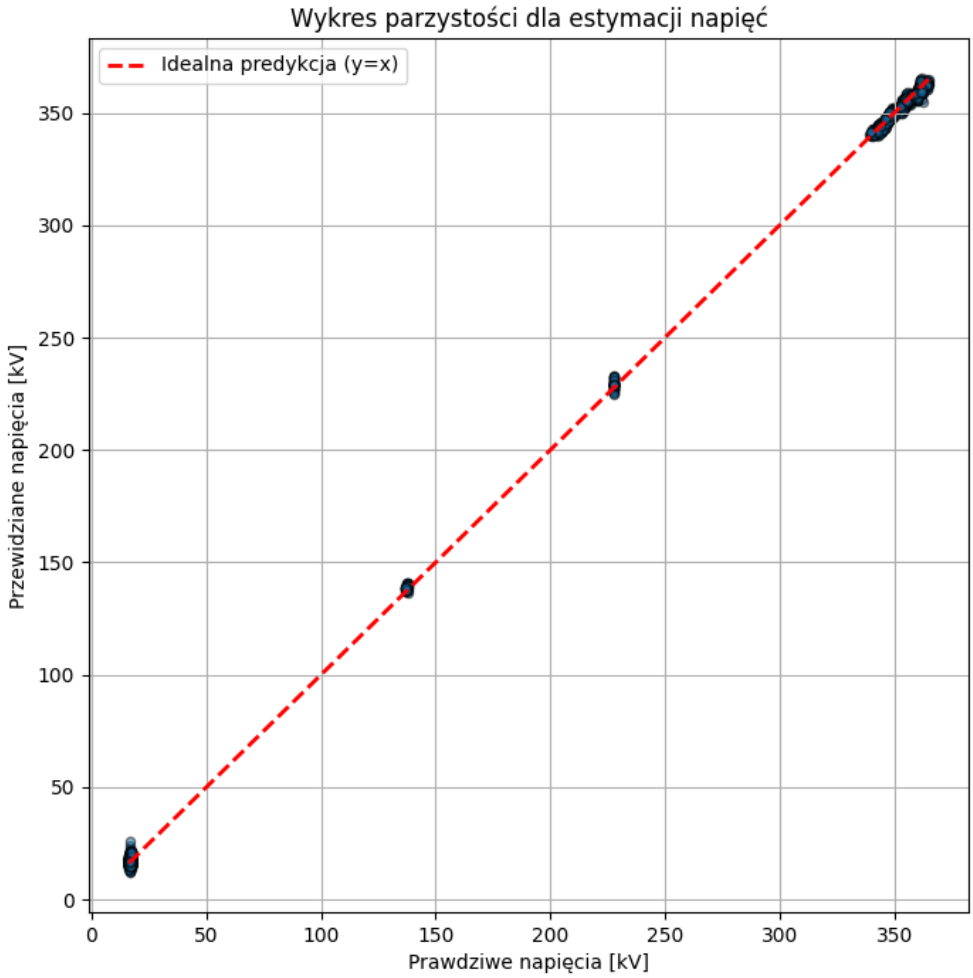
Rysunek 13: Zmiana wartości błędu MAE przy kolejnych epokach

model wysokiej precyzji, krzywa MAE stabilizuje się na niskim poziomie, wykazując jedynie minimalne fluktuacje. Ten stabilny, niski poziom błędu MAE świadczy o tym, że model efektywnie uogólnia zależności i utrzymuje wysoką dokładność estymacji na danych walidacyjnych. Połączenie analizy obu krzywych strat (MSE i MAE) dostarcza kompletnego obrazu procesu treningowego. Pokazuje, że model nie tylko minimalizuje błąd kwadratowy, który penalizuje duże odchylenia, ale także utrzymuje średni błąd na bardzo niskim i stabilnym poziomie, co jest kluczowe dla jego niezawodności w zastosowaniach praktycznych.

Analiza wykresu parzystości dla estymacji napięć, będącego wizualną reprezentacją zależności między wartościami przewidywanymi a rzeczywistymi, dostarcza fundamentalnych informacji o dokładności i charakterze błędów modelu. Na wykresie, gdzie oś odciętych x reprezentuje rzeczywiste wartości napięć, a oś rzędnych y ich przewidywane odpowiedniki, idealna predykcja jest zdefiniowana przez funkcję tożsamościową $y = x$, symbolizowaną przez czerwoną, przerywaną linię.

Analizując rozmieszczenie punktów danych, można zaobserwować ich wyjątkowo silną kohezję i zbieżność wzdłuż tej osi. Fakt, że punkty te tworzą wyraźne, oddzielne skupiska, nie jest błędem modelu ani zjawiskiem losowym, lecz bezpośrednią konsekwencją topologii badanego systemu 39 Bus i różnych poziomów napięć znamionowych w jego węzłach. Sieć ta, podobnie jak rzeczywiste systemy elektroenergetyczne, składa się z szyn o zróżnicowanych poziomach napięcia, co powoduje, że dane wejściowe dla modelu grupowane są w naturalne klastry.

Dodatkowo przykład ten uwypukla istotę normalizacji danych pomiarowych przed ich

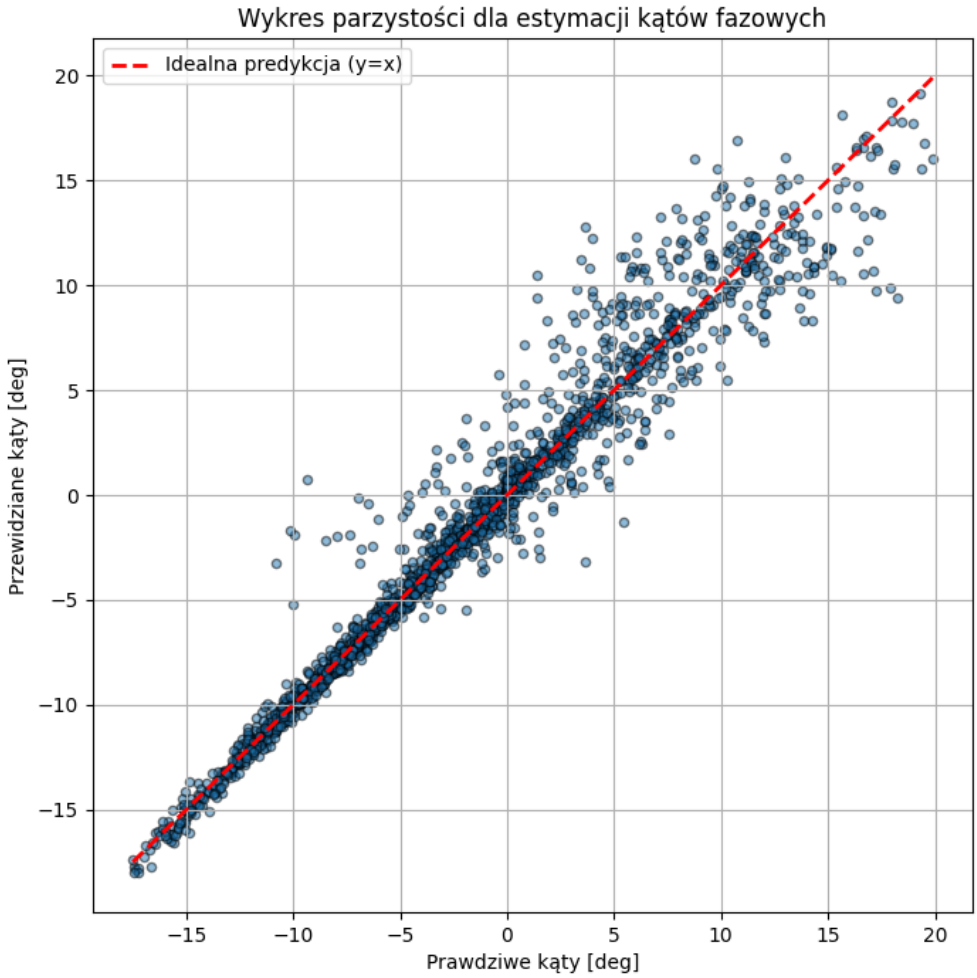


Rysunek 14: Napięcia estymowane w funkcji napięć rzeczywistych

wykorzystaniem, błąd bezwzględny w miejscu pomiaru, gdzie nominalna wartość napięcia była na poziomie 350 kV, może stanowić bardzo duży procent wartości pomiaru miejsca, gdzie napięcie znamionowe było najmniejsze, wprowadzając błąd gruby do modelu. Wykorzystanie techniki normalizacji danych w etapie przygotowania danych jest kluczowe dla prawidłowego działania sieci neuronowej. Ujednolicenie wartości pomiarowych ma na celu wyeliminowanie wpływu różnic w skali.

Natomiast ograniczone rozproszenie punktów w obrębie każdego z klastrów świadczy o niskim poziomie błędu i małej wariancji estymacji w całym zakresie wartości napięć. Należy również zaznaczyć, że w procesie generacji danych, zmienność obciążenia nie zawsze doprowadzała do maksymalnego obciążenia poszczególnych generatorów, co mogło mieć wpływ na stosunkowo niewielkie fluktuacje napięć.

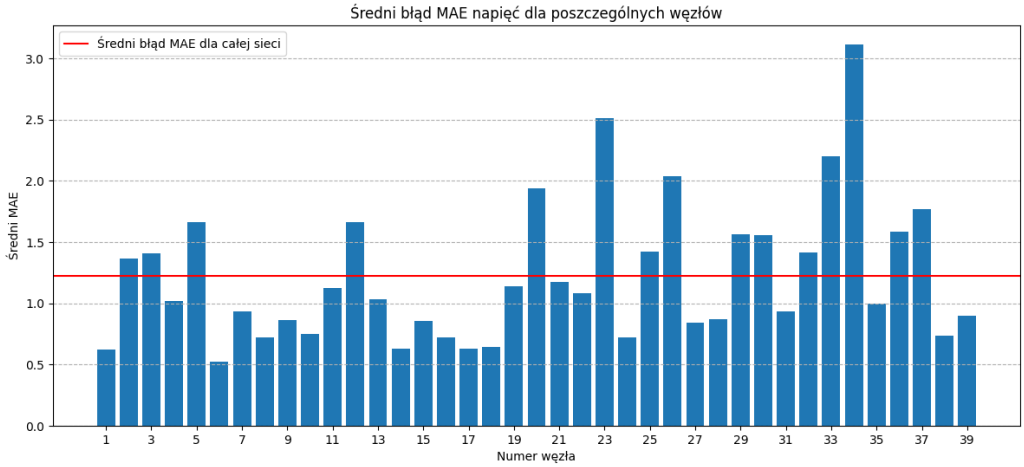
Podobnie jak w przypadku estymacji napięć, wykres parzystości dla kątów fazowych



Rysunek 15: kąty estymowane w funkcji kątów rzeczywistych

potwierdza wysoką korelację między wartościami przewidywanymi a rzeczywistymi. Jednakże, w odróżnieniu od wykresu dla napięć, obserwuje się tutaj wyraźnie większe rozproszenie punktów danych wzdłuż linii idealnej predykcji. Jest to zjawisko, które ma swoje uzasadnienie fizyczne i nie stanowi błędu w samej metodzie.

W systemach elektroenergetycznych, różnice kątów fazowych między węzłami są bezpośrednio powiązane z przepływami mocy. Wzrost przepływu mocy powoduje wzrost różnicy kątów, co może prowadzić do większych wahań i, w konsekwencji, do większych odchyleń estymacji. Widać to szczególnie dla większych wartości kątów, gdzie rozproszenie punktów jest wyraźniejsze. Ta zależność ma kluczowe znaczenie: wraz ze wzrostem obciążenia linii, rosną nie tylko wartości kątów, ale także ich wariancja, co naturalnie prowadzi do zwiększenia błędu estymacji. Zatem, choć model utrzymuje wysoką precyzję, to jego dokładność jest bardziej wrażliwa na dynamiczne warunki pracy systemu,



Rysunek 16: Średnia wartość MAE z uwzględnieniem dla poszczególnych węzłów

co potwierdza zwiększone rozproszenie w miarę oddalania się od zerowej wartości kąta. Mimo tego, model GNN wykazuje wyjątkową zdolność do rozumienia i przetwarzania tych zależności.

Analiza średniego błędu MAE na poziomie poszczególnych węzłów dostarcza kluczowych informacji diagnostycznych, wykraczając poza ogólną ocenę wydajności modelu. Wykres słupkowy prezentuje średni błąd MAE dla estymacji napięć w każdym z 39 węzłów sieci. Czerwona linia na wykresie służy jako punkt odniesienia, reprezentując średnią wartość MAE dla całej sieci. Analizując wykres, zaobserwować można, że błąd MAE w większości węzłów utrzymuje się na poziomie poniżej średniej sieci, co świadczy o ogólnej precyzji modelu. Jednakże, istnieją punkty, takie jak węzeł 34, gdzie błąd MAE jest znacząco wyższy. Taka sytuacja niekoniecznie wskazuje na błąd w działaniu samego modelu, lecz może stanowić sygnał o anomalii lub nietypowych warunkach panujących w tym punkcie sieci, które sprawiają, że estymacja staje się bardziej złożona.

nagły wzrost wskaźnika w jednym punkcie o dużą wartość ponad średnią może stanowić o wpływie czynników zewnętrznych, takich jak:

- Błąd pomiarowy, np. uszkodzenie sensora, które wprowadza szum do danych wejściowych.
- Nieprzewidzianą zmianę topologii sieci, która nie została uwzględniona w modelu.
- Nietypowy stan pracy, który nie był obecny w danych treningowych.

Lokalna analiza błędu MAE ma fundamentalne znaczenie w kontekście wykrywania nieprawidłowości. Znaczący wzrost błędu w konkretnym węźle może być wskaźnikiem problemu, umożliwiając szybkie zdiagnozowanie i reakcję.

12 Wnioski

Niniejsza praca stanowi kompleksową analizę i praktyczną weryfikację potencjału grafowych sieci neuronowych w zadaniu estymacji stanu systemu elektroenergetycznego. Badania rozpoczęto od wnikliwej analizy teoretycznej algorytmów estymacji stanu, w tym metody ważonych najmniejszych kwadratów, której ograniczenia, takie jak brak natywnej zdolności do uwzględniania topologii sieci oraz skalowania w złożonych, wielkoskalowych systemach, posłużyły za główne uzasadnienie dla poszukiwania innowacyjnych, alternatywnych rozwiązań. Centralnym elementem pracy była hipoteza, że GNN, jako architektura naturalnie przystosowana do przetwarzania danych o strukturze grafowej, może przełamać te bariery.

Cały proces badawczy, od generacji danych do finalnej ewaluacji, stanowił studium wykonalności przeprowadzone w warunkach symulacyjnych. Wykorzystując oprogramowanie DIgSILENT PowerFactory, wygenerowano obszerny, syntetyczny zbiór danych na podstawie realistycznej sieci testowej "39 Bus New England System", co pozwoliło na stworzenie kompleksowej bazy danych do treningu. Kluczowym krokiem było precyzyjne przygotowanie danych, w tym ich normalizacja w celu ujednolicenia skal pomiarów, co miało fundamentalne znaczenie dla stabilności procesu uczenia. Zaimplementowany model GNN, oparty na mechanizmie Message Passing, pomimo przyjętych uproszczeń oraz ograniczeń sprzętowych zademonstrował użyteczność w zadaniu estymacji. Współczynnik (R^2) na poziomie 0.976 oraz średni błąd bezwzględny (MAE) na poziomie 1.064 potwierdzają zdolność modelu do dokładnego odwzorowania złożonych zależności w sieci. Analiza przebiegu treningu ujawniła niepożądane zjawisko przeuczenia, któremu skutecznie można by przeciwdziałać za pomocą techniki wczesnego zatrzymania. Działania te pozwoliły na wybranie najbardziej uogólniającej wersji modelu, która osiągnęła najniższy błąd na zbiorze walidacyjnym.

Analiza wydajności obliczeniowej ujawniła jedną z największych zalet zaimplementowanej metody – jej szybkość w fazie wnioskowania. Średni czas estymacji dla pojedynczej próbki wyniósł zaledwie 1.5932 milisekundy, co czyni GNN potężnym narzędziem dla aplikacji czasu rzeczywistego, znacząco przewyższającym tradycyjne, iteracyjne algorytmy. Fakt, że taką wydajność uzyskano na standardowym sprzęcie konsumenckim, demonstruje potencjał metody do skalowania i wdrożenia bez konieczności dostępu do drogiej, specjalistycznej infrastruktury obliczeniowej.

Przeprowadzone badanie stanowi solidny dowód na to, że grafowe sieci neuronowe są obiecującym, nowym paradygmatem w estymacji stanu systemów elektroenergetycznych. Do najważniejszych korzyści tego podejścia należą wysoka precyzja i efektywność obliczeniowa, które pozwalają na dokładne i szybkie monitorowanie sieci. Ponadto, szcze-

gółowa analiza błędu MAE per-węzeł otworzyła drogę do zastosowań w automatycznym wykrywaniu anomalii i diagnostyce, co stanowi znaczącą wartość dodaną w porównaniu do klasycznych metod.

Jednocześnie, należy zaznaczyć ograniczenia, które stanowią pole do dalszych badań. Skuteczność GNN, jak każdej innej formy uczenia maszynowego, jest silnie uzależniona od dostępności obszernego zbioru danych, co w praktyce może być wyzwaniem. Ponadto, proces treningu szczególnie dopracowanych algorytmów na dużych zbiorach danych pozostaje wysoce kosztowny obliczeniowo. Mimo tych ograniczeń, praca ta dostarcza solidnych podstaw do twierdzenia, że koszty treningu modelu, stanowiące jednorazowy nakład, zanikają w obliczu długoterminowych korzyści operacyjnych i szybkości estymacji w czasie rzeczywistym, a korzyści te znacznie przewyższają początkowe nakłady.

Udowadniając potencjał sieci GNN w estymacji stanu systemu elektroenergetycznego, spełnione zostały założenia projektu. Jednak mógłby on zostać rozbudowany o kolejne etapy, które powinny koncentrować się na rozbudowie modelu i jego weryfikacji w bardziej złożonych warunkach, co jest zgodne z aktualnymi trendami w literaturze naukowej. Obecny model, będący solidnym studium, może zostać rozszerzony w celu zwiększenia jego precyzji, odporności i skalowalności. W świetle najnowszych badań można wyróżnić kilka obiecujących kierunków dalszego rozwoju. Jednym z kluczowych obszarów jest pogłębienie architektury GNN. Modelowanie złożonych zależności w dużej skali wymaga warstw, które są w stanie efektywnie agregować informacje nie tylko z najbliższego sąsiedztwa, ale także z bardziej odległych węzłów w grafie. Implementacja głębszych sieci, aby sprawdzić, czy dalszy wzrost liczby warstw konwolucyjnych GNN prowadzi do lepszych wyników. Wartym eksploracji wydają się też zaawansowane mechanizmy uwagi *(ang. attention mechanisms)*, które pozwalają modelowi na dynamiczne przypisywanie większej wagi informacjom z krytycznych węzłów, mających większy wpływ na stan systemu. Takie podejście, badane już w kontekście estymacji stanu, może znacząco poprawić precyzję, zwłaszcza w najbardziej wrażliwych punktach sieci.

Kolejnym obszarem rozwoju jest udoskonalenie zbioru danych. Obecny zestaw, choć wystarczający do celów dowodowych, może zostać wzbogacony, aby odzwierciedlać bardziej realistyczne i zróżnicowane scenariusze pracy. Potencjalne symulacje mogłyby obejmować bardziej ekstremalne stany pracy, takie jak kaskadowe awarie, duże zaburzenia czy nietypowe obciążenia, które rzadko występują w normalnych warunkach. Uczenie na takich danych zwiększyłoby odporność modelu na stany awaryjne. Ponadto, badania nad odpornością na ataki typu bad data injection stanowią istotny kierunek, który pozwoli na weryfikację potencjału GNN w zabezpieczaniu systemów elektroenergetycznych.

Wreszcie, GNN mogą zostać zintegrowane z tradycyjnymi modelami w celu stworzenia modeli hybrydowych. Trend ten, obserwowany w literaturze naukowej, zakłada

połączenie zalet uczenia maszynowego (szybkość i zdolność do uczenia się z danych) z precyzją i fizyczną wiernością klasycznych, analitycznych algorytmów rozpływu mocy. Tak zbudowany hybrydowy estymator mógłby w przyszłości być bardziej niezawodny, szczególnie w warunkach o dużej niepewności.

13 Załączniki

13.1 Załącznik A

W załączniku A przedstawiono pełny kod źródłowy w języku Python, wykorzystany do przeprowadzenia symulacji porównawczej estymatorów stanu, opisanej w rozdziale 4.

```python
# -*- coding: utf-8 -*-
"""
Załącznik A: Skrypt do symulacji i porównania estymatorów stanu LS i WLS

Ten skrypt implementuje i porównuje działanie dwóch klasycznych metod estymacji
stanu systemu elektroenergetycznego: Metody Najmniejszych Kwadratów (LS) oraz
Ważonych Najmniejszych Kwadratów (WLS).

Symulacja jest przeprowadzona na prostej sieci dwuwęzłowej w scenariuszu,
w którym jeden z pomiarów jest obarczony błędem grubym ('bad data').
Celem jest demonstracja przewagi metody WLS w takich warunkach.

Autor: Aleks Piszczek
Data: 17.09.2025
"""

# ==============================================================================
# KROK 1: INSTALACJA I IMPORT BIBLIOTEK
# ==============================================================================
try:
    import pandapower as pp
except ImportError:
    # Upewniamy się, że pandapower jest zainstalowane w środowisku
    print("Instalowanie biblioteki pandapower...")
    import subprocess
    import sys

    subprocess.check_call([sys.executable, "-m", "pip", "install", "pandapower"])
    import pandapower as pp

import pandas as pd
import numpy as np

# ==============================================================================
# KROK 2: USTAWIENIA I TWORZENIE SIECI REFERENCYJNEJ
# ==============================================================================

# Ustawienie "ziarna losowości" dla zapewnienia powtarzalności wyników
np.random.seed(42)

# Definicja prawdziwych wartości mocy w węźle obciążenia
TRUE_P_LOAD_MW = 0.1
TRUE_Q_LOAD_MVAR = 0.05
```

```python


def create_base_network():
    """
    Tworzy i zwraca obiekt sieci pandapower dla dwuwęzłowego systemu testowego.
    """
    net = pp.create_empty_network(name="Sieć 2-węzłowa")

    # Definicja węzłów
    bus_slack = pp.create_bus(net, vn_kv=0.4, name="Węzeł odniesienia (Slack)")
    bus_load = pp.create_bus(net, vn_kv=0.4, name="Węzeł obciążenia (PQ)")

    # Definicja linii przesyłowej
    pp.create_line_from_parameters(
        net, from_bus=bus_slack, to_bus=bus_load, length_km=0.5,
        r_ohm_per_km=0.3, x_ohm_per_km=0.1, c_nf_per_km=50, max_i_ka=0.25,
        name="Linia L1"
    )

    # Definicja sieci zewnętrznej (źródła) i obciążenia
    pp.create_ext_grid(net, bus=bus_slack, vm_pu=1.02, name="Sieć zasilająca")
    pp.create_load(net, bus=bus_load, p_mw=TRUE_P_LOAD_MW,
    ↪  q_mvar=TRUE_Q_LOAD_MVAR, name="Obciążenie")

    return net


# Utworzenie sieci i obliczenie "prawdziwego" rozpływu mocy
net_true = create_base_network()
pp.runpp(net_true)

# Pobranie prawdziwych wartości w węźle obciążenia do późniejszego porównania
true_v_load = net_true.res_bus.vm_pu.iloc[1]
true_theta_load = net_true.res_bus.va_degree.iloc[1]

# ==============================================================================
# KROK 3: PRZYGOTOWANIE MODELU MATEMATYCZNEGO
# ==============================================================================

# Wyciągnięcie parametrów admitancyjnych Ybus z modelu pandapower
Ybus = net_true._ppc["internal"]["Ybus"].toarray()
Y00, Y01, Y10, Y11 = Ybus[0, 0], Ybus[0, 1], Ybus[1, 0], Ybus[1, 1]
G00, B00, G01, B01 = Y00.real, Y00.imag, Y01.real, Y01.imag
G10, B10, G11, B11 = Y10.real, Y10.imag, Y11.real, Y11.imag
V0_NOMINAL = 1.02  # Napięcie w węźle odniesienia jest znane i stałe


def h_full(x_k):
    """
    Oblicza wektor przewidywanych pomiarów h(x) na podstawie wektora stanu x_k.

    Args:
```

```
        x_k (np.array): Wektor stanu [V1, theta1_rad].

    Returns:
        np.array: Wektor przewidywanych pomiarów [V0, P0, Q0, P_load].
    """
    V1, theta1_rad = x_k
    theta0_rad = 0.0
    delta01 = theta0_rad - theta1_rad

    # Moc płynąca z węzła 0 do 1 (na początku linii)
    P0_calc = V0_NOMINAL ** 2 * G00 + V0_NOMINAL * V1 * (G01 * np.cos(delta01) + B01 * np.sin(delta01))
    Q0_calc = -V0_NOMINAL ** 2 * B00 + V0_NOMINAL * V1 * (G01 * np.sin(delta01) - B01 * np.cos(delta01))

    # Moc w węźle 1 (obciążenie)
    delta10 = -delta01
    P1_inj = V1 ** 2 * G11 + V1 * V0_NOMINAL * (G10 * np.cos(delta10) + B10 * np.sin(delta10))
    P_load_calc = -P1_inj  # Moc obciążenia to ujemna moc wprowadzana (injekcja)

    return np.array([V0_NOMINAL, P0_calc, Q0_calc, P_load_calc])


def jacobian_full(x_k):
    """
    Oblicza macierz Jacobiego H (pochodne h(x) względem x) dla danego stanu x_k.

    Args:
        x_k (np.array): Wektor stanu [V1, theta1_rad].

    Returns:
        np.array: Macierz Jacobiego (4x2).
    """
    V1, theta1_rad = x_k
    theta0_rad = 0.0
    delta01 = theta0_rad - theta1_rad
    delta10 = -delta01

    # Pochodne cząstkowe dla mocy wypływającej z węzła 0
    dP0_dV1 = V0_NOMINAL * (G01 * np.cos(delta01) + B01 * np.sin(delta01))
    dP0_dtheta1 = V0_NOMINAL * V1 * (G01 * np.sin(delta01) - B01 * np.cos(delta01))
    dQ0_dV1 = V0_NOMINAL * (G01 * np.sin(delta01) - B01 * np.cos(delta01))
    dQ0_dtheta1 = V0_NOMINAL * V1 * (-G01 * np.cos(delta01) - B01 * np.sin(delta01))

    # Pochodne cząstkowe dla mocy w węźle 1
    dP1_dV1 = 2 * V1 * G11 + V0_NOMINAL * (G10 * np.cos(delta10) + B10 * np.sin(delta10))
    dP1_dtheta1 = V1 * V0_NOMINAL * (-G10 * np.sin(delta10) + B10 * np.cos(delta10))
```

```python

    # V0 jest stałe, więc jego pochodne względem zmiennych stanu są zerowe
    return np.array([
        [0, 0],
        [dP0_dV1, dP0_dtheta1],
        [dQ0_dV1, dQ0_dtheta1],
        [-dP1_dV1, -dP1_dtheta1]  # Znak ujemny, bo P_load = -P1_inj
    ])


def calculate_load_from_state(x_k):
    """Oblicza pełne obciążenie (P, Q) na podstawie wyestymowanego wektora stanu."""
    V1_est, theta1_est_rad = x_k
    theta0_rad = 0.0

    delta10 = theta1_est_rad - theta0_rad
    P1_inj = V1_est ** 2 * G11 + V1_est * V0_NOMINAL * (G10 * np.cos(delta10) + B10 * np.sin(delta10))
    Q1_inj = -V1_est ** 2 * B11 + V1_est * V0_NOMINAL * (G10 * np.sin(delta10) - B10 * np.cos(delta10))

    return -P1_inj, -Q1_inj


# ==============================================================================
# KROK 4: GENEROWANIE POMIARÓW (SCENARIUSZ "BAD DATA")
# ==============================================================================
# Definicja odchyleń standardowych (sigma) dla błędów pomiarowych
std_dev_v_slack = 0.002  # Pomiar napięcia - wysoka dokładność
std_dev_p_slack = 0.002  # Pomiar mocy P na początku linii - wysoka dokładność
std_dev_q_slack = 0.002  # Pomiar mocy Q na początku linii - wysoka dokładność
std_dev_p_load_BAD = 0.05  # Pomiar mocy P obciążenia - NISKA DOKŁADNOŚĆ (BŁĄD GRUBY)

# Generowanie zaszumionych pomiarów na podstawie "prawdziwych" wartości
v_meas = net_true.res_bus.vm_pu[0] + np.random.normal(0, std_dev_v_slack)
p_meas_slack = net_true.res_line.p_from_mw[0] + np.random.normal(0, std_dev_p_slack)
q_meas_slack = net_true.res_line.q_from_mvar[0] + np.random.normal(0, std_dev_q_slack)
p_meas_load_BAD = TRUE_P_LOAD_MW + np.random.normal(0, std_dev_p_load_BAD)

# Stworzenie wektora pomiarów `z` i macierzy wag `R_inv` (odwrotność macierzy kowariancji)
z = np.array([v_meas, p_meas_slack, q_meas_slack, p_meas_load_BAD])
variances = np.array([std_dev_v_slack ** 2, std_dev_p_slack ** 2, std_dev_q_slack ** 2, std_dev_p_load_BAD ** 2])
R_inv = np.diag(1 / variances)


# ==============================================================================
```

```python
# KROK 5: ALGORYTM ESTYMACJI STANU
# ==============================================================================

def estimate_state(z, method='WLS', R_inv=None, max_iter=10, tolerance=1e-6):
    """
    Przeprowadza iteracyjną estymację stanu metodą Gaussa-Newtona dla LS lub WLS.

    Args:
        z (np.array): Wektor pomiarów.
        method (str): Metoda estymacji ('LS' lub 'WLS').
        R_inv (np.array): Odwrotność macierzy kowariancji (tylko dla WLS).
        max_iter (int): Maksymalna liczba iteracji.
        tolerance (float): Próg zbieżności algorytmu.

    Returns:
        np.array: Wyestymowany wektor stanu [V1, theta1_rad].
    """
    # Inicjalizacja "płaskim startem" (V=1.0 p.u., theta=0 rad)
    x_k = np.array([1.0, 0.0])

    for i in range(max_iter):
        H = jacobian_full(x_k)
        r = z - h_full(x_k)  # Wektor residuów

        # Wybór formuły aktualizacji w zależności od metody
        if method == 'WLS':
            # Krok aktualizacji dla Ważonych Najmniejszych Kwadratów
            # Używamy pseudoodwrotności (pinv) dla stabilności numerycznej
            delta_x = np.linalg.pinv(H.T @ R_inv @ H) @ H.T @ R_inv @ r
        else:  # Domyślnie LS
            # Krok aktualizacji dla Najmniejszych Kwadratów
            delta_x = np.linalg.pinv(H.T @ H) @ H.T @ r

        x_k += delta_x

        # Sprawdzenie warunku zbieżności
        if np.linalg.norm(delta_x) < tolerance:
            print(f"Estymacja ({method}) zbieżna po {i + 1} iteracjach.")
            break
    else:
        print(f"Estymacja ({method}) nie osiągnęła zbieżności po {max_iter} iteracjach.")

    return x_k


# Przeprowadzenie estymacji dla obu metod
x_ls = estimate_state(z, method='LS')
x_wls = estimate_state(z, method='WLS', R_inv=R_inv)

# Obliczenie wyników końcowych na podstawie wyestymowanych wektorów stanu
```

```
v_ls, theta_ls_deg = x_ls[0], np.rad2deg(x_ls[1])
p_ls, q_ls = calculate_load_from_state(x_ls)

v_wls, theta_wls_deg = x_wls[0], np.rad2deg(x_wls[1])
p_wls, q_wls = calculate_load_from_state(x_wls)

# ==============================================================================
# KROK 6: PREZENTACJA WYNIKÓW
# ==============================================================================

# Tworzenie tabeli wyników za pomocą biblioteki pandas
results = pd.DataFrame({
    "Metoda": ["Prawdziwa Wartość", "Implementacja (LS)", "Implementacja
    ↪  (WLS)"],
    "V [p.u.]": [true_v_load, v_ls, v_wls],
    "ϑ [°]": [true_theta_load, theta_ls_deg, theta_wls_deg],
    "P [MW]": [TRUE_P_LOAD_MW, p_ls, p_wls],
    "Q [MVAr]": [TRUE_Q_LOAD_MVAR, q_ls, q_wls]
})

# Obliczanie błędów (bezwzględny dla kąta, procentowe dla pozostałych)
results['Błąd ϑ [°]'] = abs(results['ϑ [°]'] - true_theta_load)
results['Błąd V [%]'] = 100 * abs(results['V [p.u.]'] - true_v_load) /
↪  true_v_load
results['Błąd P [%]'] = 100 * abs(results['P [MW]'] - TRUE_P_LOAD_MW) /
↪  TRUE_P_LOAD_MW
results['Błąd Q [%]'] = 100 * abs(results['Q [MVAr]'] - TRUE_Q_LOAD_MVAR) /
↪  TRUE_Q_LOAD_MVAR

# Wyzerowanie błędów dla wiersza z prawdziwymi wartościami dla czytelności
results.loc[0, ['Błąd ϑ [°]', 'Błąd V [%]', 'Błąd P [%]', 'Błąd Q [%]']] = 0.0

# Ustawienia wyświetlania dla ładniejszej tabeli w konsoli
pd.set_option('display.width', 120)
pd.set_option('display.float_format', '{:.5f}'.format)

# Wyświetlenie finalnego porównania
print("\n\n" + "=" * 85)
print("       PORÓWNANIE WYNIKÓW (SCENARIUSZ 'BAD DATA' - PEŁNY WEKTOR STANU)")
print("=" * 85)
print(results.to_string(index=False))
```

W załączniku B zaprezentowano skrypt stanowiący kluczowy element metodologii badawczej, odpowiedzialny za wygenerowanie zbioru danych dla modelu GNN. Implementacja w Pythonie automatyzuje proces tworzenia tysięcy unikalnych stanów pracy sieci w DIgSILENT PowerFactory, co było warunkiem koniecznym do przeprowadzenia treningu i ewaluacji estymatora stanu.

13.2 Załącznik B

```python
# -*- coding: utf-8 -*-
"""
Załącznik: Skrypt do generowania zbioru uczącego w DIgSILENT PowerFactory

Ten skrypt w języku Python służy do automatycznego generowania syntetycznego
zbioru danych na podstawie modelu sieci zaimplementowanego w PowerFactory.
Skrypt iteracyjnie modyfikuje obciążenia w sieci, uruchamia obliczenia
rozpływu mocy i eksportuje wyniki do pliku CSV.

Autor: Aleks Piszczek
"""
import sys
import powerfactory
import random
import os
import csv
import time

# ==============================================================================
# GŁÓWNE USTAWIENIA SKRYPTU
# ==============================================================================
# --- Parametry symulacji ---
LICZBA_SYMULACJI = 1000
ZMIENNOSC_OBCIAZENIA_MIN = 0.80  # 80% obciążenia nominalnego
ZMIENNOSC_OBCIAZENIA_MAX = 1.20  # 120% obciążenia nominalnego

# --- Ustawienia eksportu ---
DOCUMENTS_PATH = os.path.join(os.path.expanduser("~"), "Documents")
FOLDER_ZAPISU = os.path.join(DOCUMENTS_PATH, "PF_Dataset_Output")
NAZWA_PLIKU = 'power_system_dataset.csv'

# --- Nazwy atrybutów PowerFactory ---
ATTR_BUS = ['m:Pgen', 'm:Pload', 'm:Qgen', 'm:Qload', 'm:Ul', 'm:phiu']
ATTR_LINE = ['m:P:bus1', 'm:Q:bus1', 'c:loading']
ATTR_TRAFO = ['m:P:bushv', 'm:Q:bushv', 'c:loading']


# ==============================================================================
# FUNKCJE POMOCNICZE
# ==============================================================================

def get_attributes(obj, attributes, default_val=0.0):
    """Bezpiecznie pobiera listę atrybutów z obiektu PowerFactory."""
    if obj is None:
        return [default_val] * len(attributes)
    try:
        return [obj.GetAttribute(attr) for attr in attributes]
    except AttributeError:
        return [default_val] * len(attributes)
```

```python


# ==============================================================================
# GŁÓWNA LOGIKA SKRYPTU
# ==============================================================================

def main(app):
    """Główna funkcja sterująca procesem generowania danych."""
    start_time = time.time()
    app.ClearOutputWindow()
    app.PrintPlain(f"--- Rozpoczynam generowanie zbioru danych ({LICZBA_SYMULACJI} symulacji) ---")

    # --- Inicjalizacja obiektów PowerFactory ---
    ldf = app.GetFromStudyCase("ComLdf")
    if ldf is None:
        raise Exception("Nie znaleziono obiektu 'ComLdf' (Load Flow Calculation).")

    # Pobranie obiektów w celu ustalenia stałej kolejności i struktury
    all_loads = app.GetCalcRelevantObjects('*.ElmLod')
    all_buses = sorted(app.GetCalcRelevantObjects('*.ElmTerm'), key=lambda x: x.loc_name)
    all_lines = sorted(app.GetCalcRelevantObjects('*.ElmLne'), key=lambda x: x.loc_name)
    all_trafos = sorted(app.GetCalcRelevantObjects('*.ElmTr2'), key=lambda x: x.loc_name)

    if not all_loads or not all_buses:
        raise Exception("Nie znaleziono obciążeń (*.ElmLod) lub szyn (*.ElmTerm) w projekcie.")

    original_loads = {load: load.GetAttribute('plini') for load in all_loads}

    # --- Przygotowanie pliku CSV ---
    os.makedirs(FOLDER_ZAPISU, exist_ok=True)
    output_path = os.path.join(FOLDER_ZAPISU, NAZWA_PLIKU)

    with open(output_path, 'w', newline='', encoding='utf-8') as f:
        writer = csv.writer(f)

        # --- Tworzenie nagłówka CSV ---
        header = ['simulation_id']
        for bus in all_buses:
            name = bus.loc_name.replace(' ', '_')
            header.extend([f"Pnet_{name}", f"Qnet_{name}", f"V_{name}", f"Angle_{name}"])
        for line in all_lines:
            name = line.loc_name.replace(' ', '_')
            header.extend([f"Pflow_{name}", f"Qflow_{name}", f"Loading_{name}"])
        for trafo in all_trafos:
            name = trafo.loc_name.replace(' ', '_')
```

```
95              header.extend([f"Pflow_{name}", f"Qflow_{name}", f"Loading_{name}"])
96          writer.writerow(header)
97
98          # --- Pętla symulacyjna ---
99          valid_sim_count = 0
100         for i in range(LICZBA_SYMULACJI):
101             # 1. Randomizacja obciążeń
102             for load, original_p in original_loads.items():
103                 scaling_factor = random.uniform(ZMIENNOSC_OBCIAZENIA_MIN,
                    ↪  ZMIENNOSC_OBCIAZENIA_MAX)
104                 load.SetAttribute('plini', original_p * scaling_factor)
105
106             # 2. Wykonanie rozpływu mocy
107             if ldf.Execute() != 0:
108                 if i % 20 == 0:  # Drukuj ostrzeżenie co 20-tą odrzuconą próbkę,
                    ↪  aby nie zaspamować konsoli
109                     app.PrintWarn(f"Symulacja {i + 1}: ODRZUCONO (rozpływ mocy
                        ↪  niezbieżny).")
110                 continue
111
112             # 3. Zbieranie wyników
113             row_data = [valid_sim_count]
114
115             # Odświeżenie referencji do obiektów z wynikami
116             buses_res = {bus.loc_name: bus for bus in
                ↪  app.GetCalcRelevantObjects('*.ElmTerm')}
117             lines_res = {line.loc_name: line for line in
                ↪  app.GetCalcRelevantObjects('*.ElmLne')}
118             trafos_res = {trafo.loc_name: trafo for trafo in
                ↪  app.GetCalcRelevantObjects('*.ElmTr2')}
119
120             for bus in all_buses:
121                 p_gen, p_load, q_gen, q_load, voltage, angle =
                    ↪  get_attributes(buses_res.get(bus.loc_name), ATTR_BUS)
122                 row_data.extend([p_gen - p_load, q_gen - q_load, voltage,
                    ↪  angle])
123
124             for line in all_lines:
125                 row_data.extend(get_attributes(lines_res.get(line.loc_name),
                    ↪  ATTR_LINE))
126
127             for trafo in all_trafos:
128                 row_data.extend(get_attributes(trafos_res.get(trafo.loc_name),
                    ↪  ATTR_TRAFO))
129
130             writer.writerow(row_data)
131             valid_sim_count += 1
132
133             if valid_sim_count % 100 == 0:
134                 app.PrintInfo(f"Wygenerowano {valid_sim_count} poprawnych
                    ↪  próbek...")
135
```

```
    # --- Przywrócenie oryginalnych wartości i podsumowanie ---
    for load, original_p in original_loads.items():
        load.SetAttribute('plini', original_p)

    end_time = time.time()
    total_time = end_time - start_time
    discarded_count = LICZBA_SYMULACJI - valid_sim_count

    app.PrintPlain("\n" + "=" * 50)
    app.PrintPlain("--- Skrypt zakończył działanie ---")
    app.PrintInfo(f"Zapisano {valid_sim_count} poprawnych symulacji.")
    app.PrintWarn(f"Odrzucono {discarded_count} symulacji (niezbieżny
    ↪  rozpływ).")
    app.PrintPlain(f"Całkowity czas wykonania: {total_time:.2f} sekund.")
    app.PrintPlain(f"Plik wynikowy zapisano w: {output_path}")
    app.PrintPlain("=" * 50)


# ==============================================================================
# PUNKT URUCHOMIENIA SKRYPTU
# ==============================================================================
if __name__ == "__main__":
    try:
        pf_app = powerfactory.GetApplicationExt()
        if pf_app is None:
            raise RuntimeError("Nie udało się połączyć z aplikacją
            ↪  PowerFactory.")
        main(pf_app)
    except Exception as e:
        # Ten blok przechwyci błędy i wyświetli je w oknie PowerFactory
        # zanim skrypt zostanie zamknięty.
        if 'pf_app' in locals() and pf_app is not None:
            pf_app.PrintError(f"\nKRYTYCZNY BŁĄD PODCZAS WYKONYWANIA
            ↪  SKRYPTU:\n{e}")
        else:
            print(f"KRYTYCZNY BŁĄD: {e}")
```

13.3 Załącznik C

W załączniku C zaprezentowano kod źródłowy stanowiący praktyczną realizację głównego wkładu badawczego tej pracy. Implementacja w języku Python przedstawia kompletny proces budowy, treningu i walidacji estymatora stanu opartego na Grafowej Sieci Neuronowej (GNN), którego wyniki i analiza zostały szczegółowo omówione w rozdziałach 10 i 11.

```
# -*- coding: utf-8 -*-
"""
Załącznik: Implementacja Grafowej Sieci Neuronowej do Estymacji Stanu
```

```python

Skrypt do treningu i ewaluacji modelu GNN na potrzeby estymacji stanu
systemu elektroenergetycznego. Narzędzie stworzone w ramach pracy magisterskiej
w celu weryfikacji potencjału uczenia maszynowego w monitorowaniu pracy sieci.

Autor: Aleks Piszczek
"""

# ==============================================================================
# Krok 1: Wczytanie niezbędnych modułów
# ==============================================================================
print("--- Krok 1: Wczytanie modułów ---")

import pandas as pd
import numpy as np
import torch
from torch.utils.data import Dataset
import torch.nn as nn
from torch_geometric.data import Data
from torch_geometric.loader import DataLoader as GraphDataLoader
from torch_geometric.nn import MessagePassing, BatchNorm
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt
import os
import time

print(f"PyTorch wersja: {torch.__version__}")
print("Wymagane biblioteki wczytano.")

# ==============================================================================
# Krok 2: Określenie lokalizacji danych
# ==============================================================================
print("\n--- Krok 2: Wskazanie ścieżki do pliku pomiarowego ---")

# UWAGA: Poniższą ścieżkę należy dostosować do lokalizacji pliku na komputerze.
FILE_PATH = '/Users/alekspiszczek/Downloads/complete_dataset.csv'

if not os.path.exists(FILE_PATH):
    raise FileNotFoundError(f"Plik danych nie został znaleziony: {FILE_PATH}.
    ↪  Sprawdź ścieżkę.")

print(f'Plik danych "{os.path.basename(FILE_PATH)}" zostanie użyty.')

# ==============================================================================
# Krok 3: Definicja topologii sieci (struktura grafu)
# ==============================================================================
print("\n--- Krok 3: Definicja topologii sieci ---")

# Lista połączeń (krawędzi) w sieci testowej IEEE 39-Bus
edge_list_1_based = [
```

```
    [1, 2], [1, 39], [2, 3], [2, 25], [3, 4], [3, 18], [4, 5], [4, 14],
    [5, 6], [5, 8], [6, 7], [6, 11], [7, 8], [8, 9], [9, 39], [10, 11],
    [10, 13], [11, 12], [13, 14], [14, 15], [15, 16], [16, 17], [16, 19],
    [16, 24], [17, 18], [17, 27], [19, 20], [19, 33], [20, 21], [20, 34],
    [21, 22], [22, 23], [22, 35], [23, 24], [23, 36], [25, 26], [25, 37],
    [26, 27], [26, 28], [26, 29], [28, 29], [29, 38], [30, 38], [31, 32],
    [33, 34], [35, 36]
]
edge_list_1_based.sort()

# Konwersja do formatu wymaganego przez PyTorch Geometric (indeksowanie od zera)
edge_list_0_based = [[u - 1, v - 1] for u, v in edge_list_1_based]
edge_list_bidirectional = edge_list_0_based + [[v, u] for u, v in edge_list_0_based]
edge_index = torch.tensor(edge_list_bidirectional, dtype=torch.long).t().contiguous()

num_nodes = 39
num_unique_edges = len(edge_list_1_based)

print(f"Struktura grafu gotowa. Liczba węzłów: {num_nodes}, Liczba krawędzi: {num_unique_edges}.")

# ==============================================================================
# Krok 4: Przygotowanie i skalowanie danych
# ==============================================================================
print("\n--- Krok 4: Przygotowanie i skalowanie danych ---")

df = pd.read_csv(FILE_PATH)
num_simulations = len(df)

# Ekstrakcja danych węzłów i krawędzi z płaskiej struktury CSV
node_cols = num_nodes * 4
node_data_flat = df.iloc[:, 1:(1 + node_cols)].values
edge_data_flat = df.iloc[:, (1 + node_cols):].values
node_data = node_data_flat.reshape(num_simulations, num_nodes, 4)
edge_data = edge_data_flat.reshape(num_simulations, num_unique_edges, 3)

# Podział na cechy wejściowe (X) i docelowe (Y)
X_nodes = node_data[:, :, :2]  # Pomiary P i Q w węzłach
X_edges = edge_data  # Pomiary P, Q i obciążenia w gałęziach
Y_nodes = node_data[:, :, 2:]  # Prawdziwe wartości V i Theta w węzłach

# Podział na zbiory treningowy, walidacyjny i testowy
X_nodes_temp, X_nodes_test, X_edges_temp, X_edges_test, Y_nodes_temp, Y_nodes_test = \
    train_test_split(X_nodes, X_edges, Y_nodes, test_size=0.2, random_state=42)
X_nodes_train, X_nodes_val, X_edges_train, X_edges_val, Y_nodes_train, Y_nodes_val = \
    train_test_split(X_nodes_temp, X_edges_temp, Y_nodes_temp, test_size=0.25, random_state=42)  # 0.25 * 0.8 = 0.2

```

```python
# Inicjalizacja i dopasowanie skalerów (tylko na danych treningowych)
x_node_scaler = StandardScaler().fit(X_nodes_train.reshape(-1, X_nodes_train.shape[-1]))
x_edge_scaler = StandardScaler().fit(X_edges_train.reshape(-1, X_edges_train.shape[-1]))
y_node_scaler = StandardScaler().fit(Y_nodes_train.reshape(-1, Y_nodes_train.shape[-1]))


def scale_data(data, scaler):
    """Pomocnicza funkcja do skalowania danych."""
    shape = data.shape
    return scaler.transform(data.reshape(-1, shape[-1])).reshape(shape)


# Skalowanie wszystkich zbiorów
X_nodes_train_scaled = scale_data(X_nodes_train, x_node_scaler)
X_nodes_val_scaled = scale_data(X_nodes_val, x_node_scaler)
X_nodes_test_scaled = scale_data(X_nodes_test, x_node_scaler)
X_edges_train_scaled = scale_data(X_edges_train, x_edge_scaler)
X_edges_val_scaled = scale_data(X_edges_val, x_edge_scaler)
X_edges_test_scaled = scale_data(X_edges_test, x_edge_scaler)
Y_nodes_train_scaled = scale_data(Y_nodes_train, y_node_scaler)
Y_nodes_val_scaled = scale_data(Y_nodes_val, y_node_scaler)
Y_nodes_test_scaled = scale_data(Y_nodes_test, y_node_scaler)

print("Dane podzielono i przeskalowano.")
print(f"Rozmiary zbiorów: Treningowy={len(X_nodes_train)}, Walidacyjny={len(X_nodes_val)}, Testowy={len(X_nodes_test)}")


class PowerSystemDataset(Dataset):
    """Niestandardowa klasa Dataset do tworzenia obiektów grafowych."""

    def __init__(self, node_features, edge_features, targets, edge_index):
        self.node_features = torch.tensor(node_features, dtype=torch.float32)
        self.edge_features = torch.tensor(edge_features, dtype=torch.float32)
        self.targets = torch.tensor(targets, dtype=torch.float32)
        self.edge_index = edge_index

    def __len__(self):
        return len(self.node_features)

    def __getitem__(self, idx):
        # Tworzy dwukierunkowe atrybuty krawędzi dla każdego grafu
        edge_features_single = self.edge_features[idx]
        edge_attr_bidirectional = torch.cat([edge_features_single, edge_features_single], dim=0)

        data = Data(x=self.node_features[idx],
                    edge_index=self.edge_index,
                    edge_attr=edge_attr_bidirectional,
```

```
                    y=self.targets[idx])
        return data


train_dataset = PowerSystemDataset(X_nodes_train_scaled, X_edges_train_scaled, Y_nodes_train_scaled, edge_index)
val_dataset = PowerSystemDataset(X_nodes_val_scaled, X_edges_val_scaled, Y_nodes_val_scaled, edge_index)
test_dataset = PowerSystemDataset(X_nodes_test_scaled, X_edges_test_scaled, Y_nodes_test_scaled, edge_index)

# ==============================================================================
# Krok 5: Przygotowanie obiektów do ładowania danych (DataLoader)
# ==============================================================================
print("\n--- Krok 5: Przygotowanie DataLoaderów ---")
BATCH_SIZE = 32
train_loader = GraphDataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = GraphDataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = GraphDataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)
print(f"DataLoadery gotowe. Rozmiar paczki (batch size): {BATCH_SIZE}")

# ==============================================================================
# Krok 6: Definicja architektury sieci GNN
# ==============================================================================
print("\n--- Krok 6: Definicja architektury GNN ---")


class GNNLayer(MessagePassing):
    """Niestandardowa warstwa GNN implementująca mechanizm przekazywania wiadomości."""

    def __init__(self, node_channels, edge_channels, output_channels):
        super().__init__(aggr='mean')
        self.mlp = nn.Sequential(
            nn.Linear(node_channels * 2 + edge_channels, output_channels),
            nn.ReLU(),
            nn.Linear(output_channels, output_channels)
        )

    def forward(self, x, edge_index, edge_attr):
        return self.propagate(edge_index, x=x, edge_attr=edge_attr)

    def message(self, x_i, x_j, edge_attr):
        # Wiadomość jest funkcją cech węzła docelowego (i), źródłowego (j) i krawędzi
        tmp = torch.cat([x_i, x_j, edge_attr], dim=1)
        return self.mlp(tmp)


class GNNStateEstimator(torch.nn.Module):
```

```
    """Kompletny model GNN do estymacji stanu."""

    def __init__(self, num_node_features, num_edge_features, num_output_features, hidden_channels):
        super().__init__()
        self.conv1 = GNNLayer(num_node_features, num_edge_features, hidden_channels)
        self.bn1 = BatchNorm(hidden_channels)
        self.conv2 = GNNLayer(hidden_channels, num_edge_features, hidden_channels)
        self.bn2 = BatchNorm(hidden_channels)
        self.output_layer = nn.Linear(hidden_channels, num_output_features)

    def forward(self, x, edge_index, edge_attr):
        x = self.conv1(x, edge_index, edge_attr)
        x = self.bn1(x)
        x = x.relu()
        x = self.conv2(x, edge_index, edge_attr)
        x = self.bn2(x)
        x = x.relu()
        out = self.output_layer(x)
        return out


# Inicjalizacja modelu
model = GNNStateEstimator(num_node_features=2, num_edge_features=3, num_output_features=2, hidden_channels=128)
print("Zdefiniowano architekturę modelu GNN.")
print(model)

# ==============================================================================
# Krok 7: Trening modelu
# ==============================================================================
print("\n--- Krok 7: Trening modelu ---")

# Wybór urządzenia (GPU lub CPU)
if torch.backends.mps.is_available():
    device = torch.device("mps")
    print("Korzystam z akceleracji GPU (Apple Metal Performance Shaders)")
elif torch.cuda.is_available():
    device = torch.device("cuda")
    print("Korzystam z akceleracji GPU (NVIDIA CUDA)")
else:
    device = torch.device("cpu")
    print("Korzystam z CPU")

model = model.to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
criterion = torch.nn.MSELoss()


def train_epoch():
```

```
    """Wykonuje jedną epokę treningu."""
    model.train()
    total_loss = 0
    start_time = time.time()
    for data in train_loader:
        data = data.to(device)
        optimizer.zero_grad()
        out = model(data.x, data.edge_index, data.edge_attr)
        loss = criterion(out, data.y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * data.num_graphs
    end_time = time.time()
    return total_loss / len(train_loader.dataset), end_time - start_time


def evaluate_model(loader):
    """Ocenia model na danym zbiorze danych (walidacyjnym lub testowym)."""
    model.eval()
    all_preds_scaled, all_labels_scaled = [], []
    with torch.no_grad():
        for data in loader:
            data = data.to(device)
            out = model(data.x, data.edge_index, data.edge_attr)
            all_preds_scaled.append(out.cpu().numpy())
            all_labels_scaled.append(data.y.cpu().numpy())

    all_preds_scaled = np.vstack(all_preds_scaled)
    all_labels_scaled = np.vstack(all_labels_scaled)

    # Przeskalowanie wyników do oryginalnych jednostek
    all_preds_unscaled = y_node_scaler.inverse_transform(all_preds_scaled.reshape(-1, 2)).reshape(
        all_preds_scaled.shape)
    all_labels_unscaled = y_node_scaler.inverse_transform(all_labels_scaled.reshape(-1, 2)).reshape(
        all_labels_scaled.shape)

    val_loss = mean_squared_error(all_labels_scaled, all_preds_scaled)
    mae = mean_absolute_error(all_labels_unscaled, all_preds_unscaled)
    rmse = np.sqrt(mean_squared_error(all_labels_unscaled, all_preds_unscaled))
    r2 = r2_score(all_labels_unscaled, all_preds_unscaled)

    return mae, rmse, r2, val_loss, all_labels_unscaled, all_preds_unscaled


# Pętla treningowa
NUM_EPOCHS = 100
train_loss_history, val_loss_history, val_mae_history = [], [], []
best_val_loss = float('inf')
```

```python
total_train_time = 0

print(f"Rozpoczynam trening na {NUM_EPOCHS} epokach...")
for epoch in range(1, NUM_EPOCHS + 1):
    train_loss, train_time = train_epoch()
    val_mae, _, _, val_loss, _, _ = evaluate_model(val_loader)

    total_train_time += train_time
    train_loss_history.append(train_loss)
    val_mae_history.append(val_mae)
    val_loss_history.append(val_loss)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(model.state_dict(), 'best_model.pth')

    if epoch % 10 == 0 or epoch == 1:
        print(f'Epoka: {epoch:04d}, Strata tren. (MSE): {train_loss:.6f}, Strata wal. (MSE): {val_loss:.6f}')

print("Trening zakończony.")
model.load_state_dict(torch.load('best_model.pth'))  # Wczytanie najlepszego modelu
# ==============================================================================
# Krok 8: Ostateczna ocena i wizualizacja wyników
# ==============================================================================
print("\n--- Krok 8: Finalna ocena i wizualizacje ---")

start_test_time = time.time()
final_mae, final_rmse, final_r2, _, final_labels, final_preds = evaluate_model(test_loader)
end_test_time = time.time()
total_test_time = end_test_time - start_test_time

print("\n--- Finalne metryki na zbiorze testowym ---")
print(f"Średni błąd bezwzględny (MAE):  {final_mae:.6f}")
print(f"Pierw. błędu śr.-kwadr. (RMSE): {final_rmse:.6f}")
print(f"Współczynnik determinacji (R^2): {final_r2:.6f}")

print("\n--- Analiza czasu obliczeń ---")
print(f"Całkowity czas treningu: {total_train_time:.2f} s")
print(f"Średni czas na epokę: {total_train_time / NUM_EPOCHS:.4f} s")
print(f"Czas estymacji na zbiorze testowym: {total_test_time:.2f} s")
print(f"Średni czas estymacji dla pojedynczej próbki: {total_test_time / len(test_dataset) * 1000:.4f} ms")

# Wykresy historii strat
plt.figure(figsize=(10, 5))
plt.plot(train_loss_history, label='Strata treningowa (MSE)')
plt.plot(val_loss_history, label='Strata walidacyjna (MSE)')
plt.title('Krzywe strat w trakcie treningu')
plt.xlabel('Epoka')
```

```
plt.ylabel('Wartość straty (MSE)')
plt.legend()
plt.grid(True)
plt.show()

# Wykresy parzystości (predicted vs. true) - POPRAWIONA SEKCJA
true_voltages = final_labels[:, 0]
pred_voltages = final_preds[:, 0]
true_angles = final_labels[:, 1]
pred_angles = final_preds[:, 1]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
ax1.scatter(true_voltages, pred_voltages, alpha=0.5, s=10)
ax1.plot([true_voltages.min(), true_voltages.max()], [true_voltages.min(),
↪  true_voltages.max()], 'r--', label='Idealna predykcja')
ax1.set_title('Wykres parzystości dla napięć')
ax1.set_xlabel('Prawdziwe wartości [p.u.]')
ax1.set_ylabel('Przewidziane wartości [p.u.]')
ax1.grid(True)
ax1.legend()

ax2.scatter(true_angles, pred_angles, alpha=0.5, s=10)
ax2.plot([true_angles.min(), true_angles.max()], [true_angles.min(),
↪  true_angles.max()], 'r--', label='Idealna predykcja')
ax2.set_title('Wykres parzystości dla kątów fazowych')
ax2.set_xlabel('Prawdziwe wartości [stopnie]')
ax2.set_ylabel('Przewidziane wartości [stopnie]')
ax2.grid(True)
ax2.legend()
plt.tight_layout()
plt.show()

# Wykres błędów per-węzeł
# Aby to policzyć, musimy przywrócić strukturę 3D
num_test_samples = len(test_dataset)
final_labels_reshaped = final_labels.reshape(num_test_samples, num_nodes, 2)
final_preds_reshaped = final_preds.reshape(num_test_samples, num_nodes, 2)

errors_per_node = np.abs(final_labels_reshaped - final_preds_reshaped)
mean_mae_per_node_V = np.mean(errors_per_node[:, :, 0], axis=0)
mean_mae_per_node_theta = np.mean(errors_per_node[:, :, 1], axis=0)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(15, 10), sharex=True)
ax1.bar(range(1, num_nodes + 1), mean_mae_per_node_V)
ax1.set_title('Średni błąd MAE napięć dla poszczególnych węzłów')
ax1.set_ylabel('Średni MAE [p.u.]')
ax1.set_xticks(range(1, num_nodes + 1))
ax1.grid(axis='y', linestyle='--')
ax1.axhline(y=np.mean(mean_mae_per_node_V), color='r', linestyle='-',
↪  label=f'Średni błąd MAE dla całej sieci')
ax1.legend()

```

```
ax2.bar(range(1, num_nodes + 1), mean_mae_per_node_theta, color='orange')
ax2.set_title('Średni błąd MAE kątów dla poszczególnych węzłów')
ax2.set_xlabel('Numer węzła')
ax2.set_ylabel('Średni MAE [stopnie]')
ax2.grid(axis='y', linestyle='--')
ax2.axhline(y=np.mean(mean_mae_per_node_theta), color='r', linestyle='-',
↪  label=f'Średni błąd MAE dla całej sieci')
ax2.legend()

plt.tight_layout()
plt.show()
```

Literatura

[1] Massoud Amin and John Stringer. The electric power grid: Today and tomorrow. *MRS bulletin*, 33(4):399–407, 2008.

[2] Frede Blaabjerg, Remus Teodorescu, Marco Liserre, and Adrian V Timbus. Overview of control and grid synchronization for distributed power generation systems. *IEEE Transactions on industrial electronics*, 53(5):1398–1409, 2006.

[3] Fred C Schweppe and J Wildes. Power system static-state estimation, part i: Exact model. *IEEE Transactions on Power Apparatus and systems*, (1):120–125, 2007.

[4] Leila Kamyabi, Tek Tjing Lie, Samaneh Madanian, and Sarah Marshall. A comprehensive review of hybrid state estimation in power systems: Challenges, opportunities and prospects. *Energies*, 17(19):4806, 2024.

[5] Yi Wang, Qixin Chen, Tao Hong, and Chongqing Kang. Review of smart meter data analytics: Applications, methodologies, and challenges. *IEEE Transactions on smart Grid*, 10(3):3125–3148, 2018.

[6] Solon Falas, Markos Asprou, Charalambos Konstantinou, and Maria K Michael. Physics-informed neural networks for accelerating power system state estimation. In *2023 IEEE PES Innovative Smart Grid Technologies Europe (ISGT EUROPE)*, pages 1–5. IEEE, 2023.

[7] Alcir Monticelli. *State estimation in electric power systems: a generalized approach*. Springer Science & Business Media, 2012.

[8] Arun G Phadke and John Samuel Thorp. *Synchronized phasor measurements and their applications*, volume 1. Springer, 2008.

[9] Dai Wang, Xiaohong Guan, Ting Liu, Yun Gu, Yanan Sun, and Yang Liu. A survey on bad data injection attack in smart grid. In *2013 IEEE PES Asia-Pacific Power and Energy Engineering Conference (APPEEC)*, pages 1–6, 2013.

[10] Pandapower Developers. State estimation in pandapower, 2025. [Online; accessed 28-August-2025].

[11] Breno Carvalho and Newton Bretas. Analysis of the largest normalized residual test robustness for measurements gross errors processing in the wls state estimator. *Journal of Systemics, Cybernetics and Informatics*, 11(7):1–6, 2013.

[12] Soummya Kar, Gabriela Hug, Javad Mohammadi, and Jose MF Moura. Distributed state estimation and energy management in smart grids: A consensus {+} innovations approach. *IEEE Journal of selected topics in signal processing*, 8(6):1022–1038, 2014.

[13] Joddumahanthi Vijaychandra, Bugatha Ram Vara Prasad, Vijaya Kumar Darapureddi, Bathina Venkateswara Rao, and Łukasz Knypiński. A review of distribution system state estimation methods and their applications in power systems. *Electronics*, 12(3):603, 2023.

[14] Leon Thurner, Alexander Scheidler, Florian Schäfer, Jan-Hendrik Menke, Julian Dollichon, Friederike Meier, Steffen Meinecke, and Martin Braun. pandapower - an Open-Source Python Tool for Advanced Power System Modeling and Analysis. *IEEE Transactions on Power Systems*, 33(6):6894–6901, Nov 2018.

[15] Warren S McCulloch and Walter Pitts. A logical calculus of the ideas immanent in nervous activity. *The bulletin of mathematical biophysics*, 5(4):115–133, 1943.

[16] Frank Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, 65(6):386, 1958.

[17] Marvin Minsky and Seymour A Papert. *Perceptrons, reissue of the 1988 expanded edition with a new foreword by Léon Bottou: an introduction to computational geometry*. MIT press, 2017.

[18] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.

[19] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436–444, 2015.

[20] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314, 1989.

[21] Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*, pages 807–814, 2010.

[22] Léon Bottou. Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT'2010: 19th International Conference on Computational StatisticsParis France, August 22-27, 2010 Keynote, Invited and Contributed Papers*, pages 177–186. Springer, 2010.

[23] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[24] Trevor Hastie. The elements of statistical learning: data mining, inference, and prediction, 2009.

[25] Vladimir Nasteski. An overview of the supervised machine learning methods. *Horizons. b*, 4(51-62):56, 2017.

[26] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.

[27] Jerome H Friedman. Greedy function approximation: a gradient boosting machine. *Annals of statistics*, pages 1189–1232, 2001.

[28] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pages 785–794, 2016.

[29] Quang-Ha Ngo, Bang L.H. Nguyen, Tuyen V. Vu, Jianhua Zhang, and Tuan Ngo. Physics-informed graphical neural network for power system state estimation. *Applied Energy*, 358:122602, 2024.

[30] Amir Safdarian, MAHMOOD FOTUHI-FIRUZABAD, and Farrokh Aminifar. Composite power system adequacy assessment based on postoptimal analysis. *Turkish Journal of Electrical Engineering and Computer Sciences*, 21(1):90–106, 2013.

[31] TN Kipf. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

[32] Gergő Bendegúz Békési, Lilla Barancsuk, István Táczi, and Bálint Hartmann. Benchmarking various pseudo-measurement data generation techniques in a low-voltage state estimation pilot environment. *Applied Sciences*, 12(6), 2022.

[33] Tom Athay, Robin Podmore, and Sudhir Virmani. A practical method for the direct analysis of transient stability. *IEEE Transactions on Power Apparatus and systems*, (2):573–584, 2007.